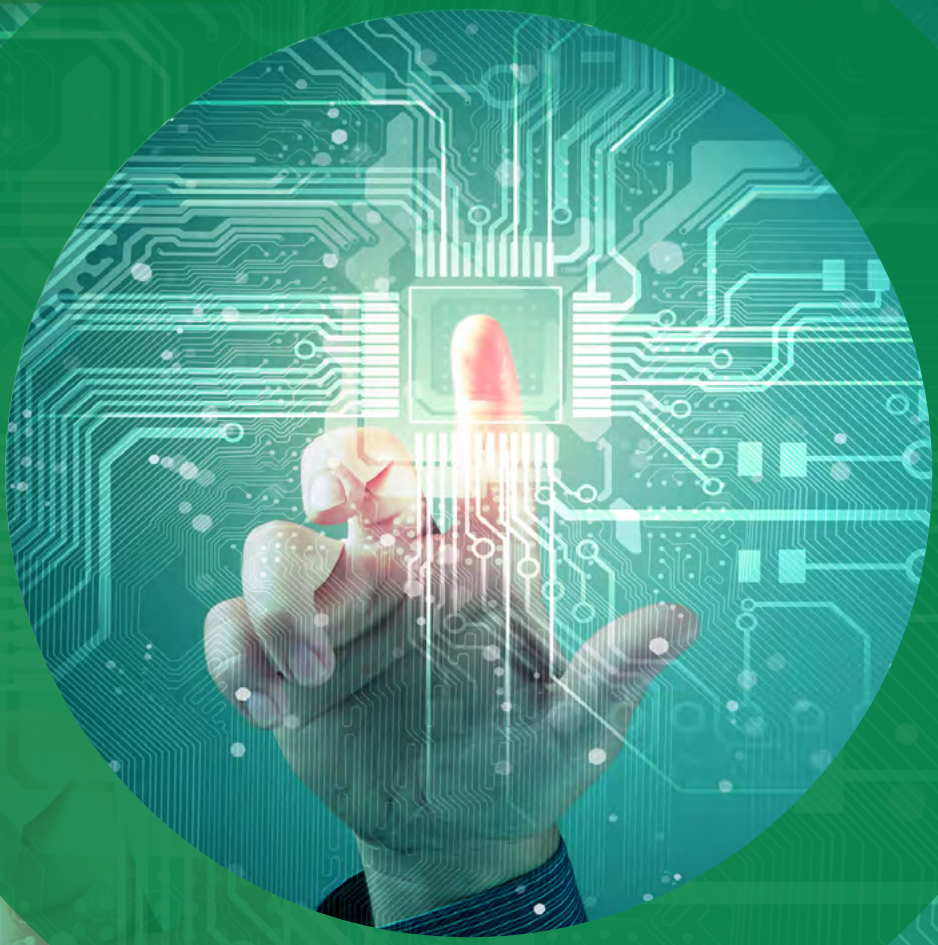
Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración



DISEÑO DE ALGORITMOS

INFORMÁTICA

Dr. Leonardo Lomelí Vanegas
Rector

Dr. Javier de la Fuente Hernández
Secretario General

Dr. Armando Tomé González
Director

Mtro. Alfonso Ayala Rico
Secretario General

Dra. Gabriela Montero Montiel
Jefa del Centro de Educación
a Distancia y Gestión del Conocimiento

Dr. Francisco Hernández Mendoza
Responsable de elaboración de material didáctico

L.A. Ramón Arcos González
Responsable de producción multimedia

Autor

Mtro. Espartaco David Kanagusico Hernández
Dr. Roberto Claudio Moreno García
Mtra. Elizabeth Hernández Sánchez

Equipo Editorial

Joel Guzmán Mosqueda
Revisión Pedagógica

Nestor Chilapa Rivas
Corrección de Estilo

Eduardo Chávez Moedano
Diseño de Portadas

Susana R. Uraga Muñoz
Diseño Editorial

Primera edición: 2023
Actualización académica: 2026

© Univerisad Nacional Autónoma de México
Univerisad Nacional Autónoma de México
Circuito Exterior s/n
Ciudad Universitaria, Coyoacán
04510, Ciudad de México.

ISBN: En Trámite
Plan de Estudios 2024

Queda prohibida la reproducción total o parcial de esta obra por cualquier medio o procedimiento, sin la autorización previa de la Universidad Nacional Autónoma de México, a traves de la Facultad de Contaduría y Administración.

Reservados todos los derechos.

Hecho en México.

DATOS DE IDENTIFICACIÓN

Diseño de algoritmos	Clave: 2125
Plan: 2023	Créditos: 8
Licenciatura: Informática	Semestre: 1°
Área o campo de conocimiento: Informática	Horas por semana: 4
Duración del programa: Semestral	Requisitos: ninguno
Tipo: Teórica Teoría: 4 Práctica: 0	
Carácter: Obligatoria (X) Optativa ( )	
Seriación: Si ( ) No (X) Obligatoria ( ) Indicativa ( )	
Asignatura con seriación antecedente: Ninguna	
Asignatura con seriación subsecuente: Ninguna	

SUGERENCIAS DE APOYO

Las siguientes sugerencias de apoyo tienen la finalidad de que puedas aprovechar de manera óptima este material didáctico.

Recomendaciones

Las siguientes recomendaciones son para los estudiantes tanto de la modalidad abierta como de la modalidad a distancia (SUAyED):

- Fomenta la interacción con tus compañeros, trata de compartir tus experiencias y comentarios sobre la asignatura para crear grupos de estudio y apoyarse mutuamente, ya sea a través de comunidades virtuales de aprendizaje, foros de discusión, correo electrónico y si te es posible de manera presencial (modalidad abierta).
- Establece un horario propicio para el estudio en el que te encuentres menos cansado, ello facilitará tu aprendizaje.
- Planifica periodos prolongados de estudio con breves descansos entre cada hora, si es necesario.
- Busca espacios adecuados donde puedas concentrarte y aprovechar al máximo tu tiempo de estudio.

Forma de trabajo

Este material didáctico ha sido diseñado específicamente para apoyar tu proceso de estudio en las modalidades abierta y a distancia de esta asignatura. Contiene un Organizador Cognitivo de Aprendizaje (OCA) que ofrece una visión general de la asignatura, los contenidos que se desarrollarán y su relación con tus conocimientos previos y posteriores. Además, plantea habilidades y aptitudes que desarrollarás y fortalecerás durante el estudio de los contenidos.

La estructura didáctica de este material te permite llevar a cabo tu aprendizaje de manera autónoma, además, cuentas con el apoyo de un asesor, en ambas modalidades, que te ayudará a alcanzar los conocimientos y habilidades necesarios para aprobar la asignatura.

Este material se encuentra conformado por las siguientes secciones:

1. Portada.
2. Datos de identificación.
3. Sugerencias de apoyo.
4. Organizador Cognitivo de Aprendizaje (OCA).
5. Objetivo general.
6. Temario.
7. Introducción general.

8. Unidad.
    8.1. Objetivo particular.
    8.2. Temario detallado.
    8.3. Actividad diagnóstica.
    8.4. Introducción.
    8.5. Contenido por unidad de conformidad con el temario
        (acompañado de imágenes, esquemas, gráficos, etc.).
    8.6. Resumen de la unidad.
    8.7. Actividades de aprendizaje.
    8.8. Actividad integradora.
    8.9. Cuestionario de reforzamiento.
    8.10. Autoevaluación.
    8.11. Respuestas de autoevaluación.
    8.12. Referencias
9. Bibliografía.
10. Contraportada.

Te sugerimos seguir los siguientes pasos:

1. Lee atentamente la presentación de la asignatura.
2. Revisa el objetivo general de la asignatura.

Por cada unidad:

1. Realiza la actividad diagnóstica para activar tus conocimientos previos.
2. Lee los contenidos educativos.
3. Revisa anticipadamente las actividades de aprendizaje para detectar su nivel de dificultad, lo que te permitirá determinar si necesitas material de apoyo adicional (otros recursos) o consultar a tu asesor.
4. Realiza las actividades correspondientes a cada tema y responde a las evaluaciones. Recuerda que las actividades a realiza dependerán de lo establecido en el plan de trabajo de tu asesor para cualquiera de las dos modalidades. Para consultar el plan de trabajo, acude a la página del SUAyED: https://suayed.fca.unam.mx. El cuestionario de autoevaluación te permitirá autoevaluarte con la finalidad de que determines tu nivel de dominio de los contenidos de la unidad y tomes medidas para mejorar tu aprendizaje. Puedes realizarlo las veces que consideres conveniente, pues normalmente no tiene ponderación para la calificación final, sin embargo, deberás prestar atención a lo que indique tu asesor al respecto.

Actividades de aprendizaje:

Las actividades de aprendizaje te ayudarán a alcanzar los objetivos planteados en cada unidad, están diseñadas para que reflexiones y analices los contenidos de forma gradual.

Recomendaciones para trabajar con las actividades:

1. El programa de la asignatura consta de **6** unidades, cada una con un número variable de actividades. En su planeación didáctica, tu asesor te indicará cuáles y cuántas son las que deberás resolver, así como su ponderación.
2. La mayoría de las unidades comienzan con la elaboración de un mapa conceptual para esquematizar el contenido total de la unidad y facilitar la comprensión.
3. Lee detenidamente cada actividad para comprender claramente qué se espera de ti. Si tienes dudas, no dudes en solicitar apoyo a tu asesor, ya sea en asesorías presenciales o por correo electrónico.
4. Te recomendamos seguir el orden de las unidades y actividades, ya que están organizadas para un aprendizaje gradual, sin embargo, deberás guiarte por lo establecido en el plan de trabajo establecido por tu asesor.
5. Las actividades de aprendizaje se resuelven directamente en plataforma y se presentan en diferentes formatos: adjuntar archivo, texto en línea, wiki, foro, autogestivas, lúdicas entre otros.
6. La forma de respuesta a cada actividad dependerá de las instrucciones proporcionadas (número de páginas, formatos, esquemas, etc.).

Se te invita a abordar estas actividades con entusiasmo ya que fueron diseñadas para apoyarte en el aprendizaje de la asignatura.

Podrás consultar o acceder a otros **recursos** a través de:

Biblioteca Digital. Para acceder a otros materiales, como libros electrónicos, es necesario registrarse en la Biblioteca Digital de la UNAM (BIDI). Puedes hacerlo desde la página principal de la FCA: http://www.fca.unam.mx/.

Ponte en línea https://ponteenlinea.fca.unam.mx/. Desde este portal podrás acceder a una gama de recursos educativos digitales, con la intención de apoyar tu aprendizaje independiente. Tales recursos se encuentran clasificados por categorías, donde la forma de realizar búsquedas es muy parecida a la que se realiza en las bibliotecas virtuales, por temas o palabras clave, etcétera. En este caso, por temas, asignatura o licenciatura. Asimismo, el portal contempla un apartado denominado Solo Apuntes, donde, además de poder consultar los materiales directamente en la plataforma Moodle, si tu deseo es consultarlos fuera de ella también lo podrás hacer. Solo necesitas acceder al portal, lo cual puedes hacer desde la propia plataforma, en Moodle, o bien, desde la página principal de la FCA.

OCA

ORGANIZADOR COGNITIVO DE APRENDIZAJE

Diseño de Algoritmos

Justificación del contexto

Este libro brinda las bases para resolver un problema lógico-matemático mediante algoritmos con el fin de formar profesionistas capaces de analizar, diseñar y programar sistemas informáticos.

<table>
<tr>
<td rowspan="4">Conocimientos previos

Matemáticas.
Bases de lógica.

Bases de programación.
Inglés básico.</td>
<td>UNIDAD 1.
Fundamentos de algoritmos</td>
<td>UNIDAD 2.
Fundamentos de programación</td>
<td rowspan="4">Conocimientos subsecuentes

Análisis, diseño y programación de sistemas informáticos.</td>
</tr>
<tr>
<td>UNIDAD 3.
Elementos básicos de un programa</td>
<td>UNIDAD 4.
Análisis de problemas</td>
</tr>
<tr>
<td>UNIDAD 5.
Diseño de algoritmos</td>
<td>UNIDAD 6.
Desarrollo de programas</td>
</tr>
<tr>
<td>Habilidades
• Capacidad de análisis.
• Pensamientos estructural y lógico para el desarrollo de algoritmos.
• Solucionar problemas.</td>
<td>Actitudes
• Proactividad.
• Creatividad e innovación.
• Disposición para investigar.</td>
</tr>
</table>

OBJETIVO GENERAL

Al finalizar el curso, el alumnado diseñará soluciones algorítmicas como respuesta a problemas de información aplicando diversas técnicas y las traducirá en un lenguaje de programación.

TEMARIO

(64 horas)

	HORAS
1. Fundamentos de algoritmos	6
2. Fundamentos de programación	6
3. Elementos básicos de un programa	10
4. Análisis de problemas	6
5. Diseño de algoritmos	20
6. Desarrollo de programas	16
TOTAL	64

INTRODUCCIÓN GENERAL

Un algoritmo es una secuencia finita de pasos para solucionar un problema. Su campo de aplicación es amplio, intervienen en la vida de las organizaciones resolviendo problemas mediante su implementación en programas de computadora en las distintas áreas de la misma. Dada su importancia, son objeto de estudio de la asignatura.

En la primera unidad, se estudian los conceptos necesarios para comprender los algoritmos y sus características, así como los autómatas y los lenguajes formales, asimismo se aborda el autómata, conocido como máquina de Turing.

En la segunda unidad, se describirá el ciclo de vida de un programa de cómputo, así como conceptos básicos de programación de computadoras, la diferencia entre un algoritmo y un programa y la descripción de los intérpretes y los compiladores.

En la tercera unidad, se describirán los elementos necesarios para el desarrollo de un programa de computadora, esto es las constantes variables, tipos de datos, expresiones y operadores.



UNIDAD 1

FUNDAMENTOS DE ALGORITMOS

OBJETIVO PARTICULAR

Al finalizar la unidad, el alumnado distinguirá los fundamentos teóricos que sustentan el concepto de algoritmo.

TEMARIO DETALLADO

(6 horas)

1. Fundamentos de algoritmos
1.1. Definición de algoritmo
1.2. Tipos de algoritmos
1.3. Autómatas y lenguajes formales
1.4. Máquina de Turing

ACTIVIDAD DIAGNÓSTICA

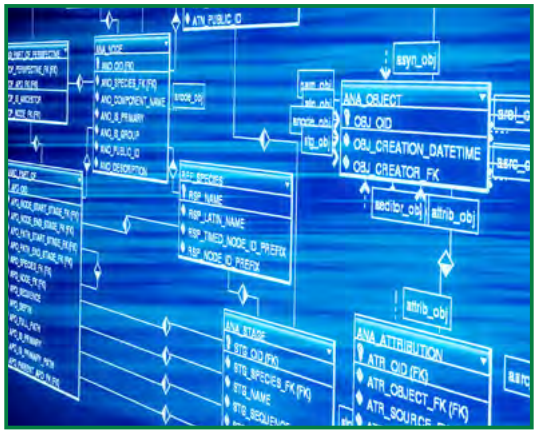


Adjuntar archivo.

Sin la necesidad de consultar ninguna fuente, contesta las siguientes preguntas:

1.- Define con tus propias palabras ¿qué entiendes por algoritmo?
2.- ¿En qué se utilizan los algoritmos?
3.- Explica de manera breve algunas características de los algoritmos.
4.- ¿En qué consiste la máquina de Turing?
5.- ¿Qué son los lenguajes formales?

INTRODUCCIÓN



Un algoritmo es una descripción ordenada de los pasos para resolver un problema. Es una manera abstracta de reducir un problema a un conjunto de pasos que le den una solución. Existen algoritmos muy sencillos y de gran creatividad, aunque también algunos conllevan un alto grado de complejidad.

Una aplicación de los algoritmos la tenemos en los autómatas, los cuales, basados en una condición de una situación dada, llevarán a cabo algunas acciones que ya se encuentran programadas en ellos. En este orden, será de gran utilidad involucrarse en su funcionamiento y terminología para entender que, en ese contexto de autómatas, los conceptos *alfabeto, frase, cadena vacía, lenguaje, gramática,* etcétera, cobran particular relevancia.

Se definirá y estudiará la máquina de Turing, un caso de autómata finito determinista, que realiza solo una actividad en una situación dada. Para conocer su diseño y funcionamiento, se desarrollan algunos ejemplos.

Así, el análisis de los algoritmos y los autómatas es medular para ejercitar un pensamiento lógico y abstracto al abordar los problemas de la informática.

1.1.
Definición de algoritmos

Según María Moliner (2002), el término algoritmo proviene "del árabe, *'hisäbu lgubar'*, cálculo mediante cifras arábigas. Procedimiento preciso para resolver un problema; por ejemplo, el que se utiliza para resolver las raíces cuadradas o las divisiones" (p. 130).

Un algoritmo es un conjunto detallado de pasos para resolver un problema. Por ejemplo, el instructivo para armar un modelo a escala. Si se atiende en forma estricta la secuencia de los pasos, se llegará al mismo resultado, no importa quien arme el modelo.

Los pasos deben ser lo más detallados posibles para la persona que arme el modelo, por lo regular, las personas tienden a dejar pasar varios aspectos de manera automática, aunque se tenga el instructivo.

Esto sería imposible en una computadora, pues requiere de indicaciones puntuales para poder ejecutarlas.

Considérese, por ejemplo, si a una persona se le pide intercambiar los números 24 y 9, con cierta lógica, responderá inmediatamente "9 y 24". En tanto, en el procesador de una computadora se tendría que indicar de qué tipo son los datos a utilizar, para este caso, números enteros, darle nombre a tres variables, digamos, *num1, num2* y *aux*; asignarle a la variable *num1* el número 24 y a *num2* el 9.



Posteriormente, señalarle que a la variable aux se le asigne el valor contenido en la variable *num1*; a *num1*, el contenido en la variable *num2*; y a esta última, el de la variable aux. Luego, se deben imprimir los valores de las variables *num1* y *num2* que exhibirán los números 9 y 24.

Se requiere, entonces, una gran cantidad de pasos para indicarle a una computadora que realice la misma tarea que a un ser humano.

Otro caso donde podemos notar la forma como el ser humano obvia pasos, es pidiendo a una persona que describa el proceso para preparar agua de limón. Seguro nos diría que toma una jarra de agua, le pone jugo de limón, azúcar, y listo. Para una computadora lo anterior no tendría sentido, ya que carece de unidades exactas y pasos detallados. Por tanto, los pasos deben tener mayor nivel de precisión, en esta secuencia como lo indica el ejemplo siguiente:

- Inicio del proceso.
- Tomar una jarra de 2 litros de capacidad.
- Llenar la jarra a 4/5 partes con agua natural.

- Tomar 4 limones.
- Cortar los limones por la mitad.
- Exprimir los limones dejando caer el jugo sobre el agua en la jarra.
- Tomar el recipiente del azúcar.
- Agregar 4 cucharadas soperas en la jarra con el agua.
- Revolver el agua con el jugo y el azúcar por 2 minutos.
- Fin del proceso.

En conclusión, cuando se elabora un algoritmo, se tomará en cuenta que la computadora es como un niño pequeño al cual se le está enseñando a realizar algo por primera vez y es necesario indicar cada paso que debe dar.

1.2. Tipos de algoritmos

Existen diversas clasificaciones para los algoritmos, una de ellas es clasificarlos como algoritmos cualitativos y algoritmos cuantitativos.

Algoritmos cualitativos

Los algoritmos cualitativos son un conjunto de instrucciones en forma de texto, para obtener la solución a un problema, cabe destacar que dicho problema no contempla cálculos matemáticos. En este tipo de algoritmos los pasos deben ser claros, concretos y con un orden riguroso. Veamos un ejemplo de este tipo de algoritmos.

Algoritmo para cambiar una llanta ponchada.

1. **INICIO**
2. Orillar el automóvil.
3. Detener el automóvil.
4. Apagar el automóvil
5. Bajarse el automóvil.
6. Abrir la cajuela.
7. Sacar señales de peligro y colocarlas a cierta distancia.
8. Sacar llanta y herramientas.
9. Introducir llave de cruz en tuercas de llanta.
10. Aflojar tuercas.
11. Levantar automóvil mediante gato hidráulico.
12. Sacar tuercas.
13. Sacar llanta ponchada.
14. Colocar llanta en buen estado.
15. Apretar tuercas.
16. Bajar automóvil con gato hidráulico.
17. Guardar llanta ponchada, señales y herramienta.
18. Cerrar cajuela.
19. Abordar el automóvil.
20. Encender automóvil.
21. Reanudar el viaje.
22. **FIN**. (Oviedo, 2015, p. 56).

Algoritmos cuantitativos

Los algoritmos cuantitativos son secuencias finitas y ordenadas de pasos que dan solución a un problema que involucra cálculos matemáticos.

Los algoritmos están compuestos por tres partes que se diagraman en la imagen. La entrada es la información dada al algoritmo por el usuario o la información generada por un conjunto de instrucciones. El proceso son los cálculos necesarios. Finalmente, la salida son los resultados, los cuales obtenemos a partir de los cálculos que se han aplicado a los datos de entrada.

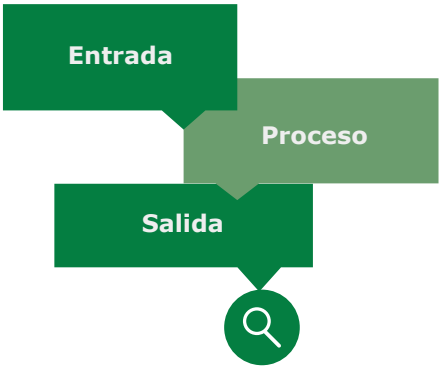


Figura 1.1. Partes de un algoritmo.

De esta forma es que los algoritmos presentan las siguientes características:

Una entrada: La entrada hace referencia a la información proporcionada al algoritmo por el usuario, la cual debe sufrir un proceso para obtener los resultados. Un algoritmo tiene cero o más datos de entrada. Estos valores le son dados por medio de una instrucción que se debe cumplir al ejecutarse el algoritmo. Son datos de entrada aquellos valores que serán almacenados en variables.

Un proceso: Un algoritmo realiza un cálculo o cálculos para obtener un resultado.

Una salida: Todo algoritmo debe proporcionar uno o más valores como resultado una vez se ha ejecutado la secuencia de pasos que lo conforman. La salida es la respuesta dada por el algoritmo.

Finitos: Todo algoritmo debe tener un número de instrucciones limitadas, es decir, la ejecución debe detenerse en un punto. No puede existir un algoritmo que se ejecute de forma indefinida.(Oviedo, 2015, p. 57)

Ordenados: Un algoritmo debe indicar el orden de realización de cada uno de sus pasos.

Claros: Todo el conjunto de pasos debe ser entendible.

Confiables: Un algoritmo siempre debe producir el mismo resultado.

1.3. Autómatas y lenguajes formales

> Un autómata es un modelo computacional consistente en un conjunto de estados bien definidos, un estado inicial, un alfabeto de entrada y una función de transición.

Este concepto es equivalente a otros como *autómata finito* o *máquina de estados finitos*. En un autómata un estado es la representación de su condición en un instante dado. El autómata comienza en el estado inicial con un conjunto de símbolos;

su paso de un estado a otro se efectúa a través de la función de transición, la cual, partiendo del estado actual y un conjunto de símbolos de entrada lo lleva al nuevo estado correspondiente.

Históricamente, los autómatas han existido desde la antigüedad, pero en el siglo XVII, cuando en Europa existía gran pasión por la técnica, se perfeccionaron las cajas de música compuestas por cilindros con púas, que fueron inspiradas por los pájaros autómatas que había en Bizancio, que podían cantar y silbar.

Así, a principios del siglo XVIII, los ebanistas Roentgen y Kintzing mostraron a Luis XVI un autómata con figura humana llamado la Tañadora de Salterio, también conocida como “La jonense de tympanon”. Por su parte, la aristocracia se apasionaba por los muñecos mecánicos de encaje, los cuadros con movimiento, y otros personajes.

Los inventores más célebres son **Pierre Jaquet-Droz**, autor del Dibujante y los Músicos; y Jacques de Vaucanson, realizador del Pato con Aparato Digestivo, un autómata que aleteaba, parloteaba, tragaba grano y evacuaba los residuos. Este autor quiso pasar de lo banal a lo útil y sus trabajos culminaron en el telar de Joseph Marie Jaquard y la máquina de Jean-Baptiste Falcon, dirigida por tarjetas perforadas.

El autómata más conocido en el mundo es la máquina de Turing, elaborado por el matemático inglés Alan Mathison Turing.

En términos estrictos, se puede decir que un termostato es un autómata, puesto que regula la potencia de calefacción de un aparato (salida) en función de la temperatura ambiente (dato de entrada), pasando de un estado térmico a otro.

Un ejemplo más de autómata en la vida cotidiana es un elevador, ya que es capaz de memorizar las diferentes llamadas de cada piso y optimizar sus ascensos y descensos.

Técnicamente, hay diferentes herramientas para definir el comportamiento de un autómata, entre las cuales se encuentra el diagrama de estado. En este se pueden visualizar los estados como círculos que en su interior registran su significado, y flechas que representan la transición entre estados y la notación de entrada/salida, que provoca el cambio entre estados.

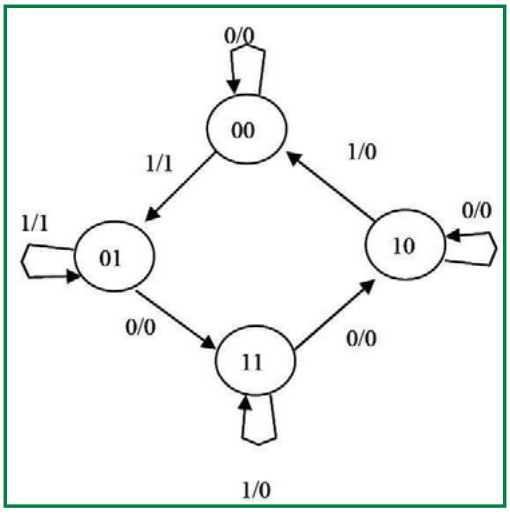


Figura 1.2. Diagrama de estado.

En el ejemplo se muestran cuatro estados diferentes de un autómata y se define lo siguiente. *Partiendo del estado "00", si se recibe una entrada "0", la salida es "0" y el autómata conserva el estado actual; pero si la entrada es "1", la salida será "1" y el autómata pasa al estado "01".*

Este comportamiento es homogéneo para todos los estados del autómata. Vale la pena resaltar que el autómata que se representa aquí tiene un alfabeto binario (0 y 1).

Otra herramienta de representación del comportamiento de los autómatas es la tabla de estado que consiste en cuatro partes: descripción del estado actual, descripción de la entrada, descripción del estado siguiente y descripción de las salidas.

A continuación, se muestra la tabla correspondiente al diagrama que se presentó en la figura anterior.

Estado actual		Entrada	Estado siguiente		Salida
A	B	X	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	1	0	1
1	1	0	0	0	0
1	1	1	1	0	1

Figura 1.3. Tabla correspondiente al diagrama de estado.

En la tabla se puede notar que el autómata tiene dos elementos que definen su estado, A y B, así como la reafirmación de su alfabeto binario. Además, se deduce la función de salida del autómata, la cual está definida por la multiplicación lógica de la negación del estado de A por la entrada x:

$$y = A' x$$

Un lenguaje formal en las ciencias de la computación, es un conjunto de símbolos y reglas. Al conjunto de los símbolos de dicho lenguaje se le llama alfabeto.

Un alfabeto es el conjunto de todos los símbolos válidos para una aplicación. En el campo de los autómatas, un lenguaje está formado por todos los caracteres utilizados para definir sus entradas, salidas y estados, el alfabeto puede ser tan simple como un alfabeto binario. Por ultimo una gramática es una colección estructurada de palabras y frases ligadas por reglas que definen el conjunto de cadenas de caracteres que representan comandos.

Una forma de representar las gramáticas es la Backus-Naur (BNF), usada para describir la sintaxis de un lenguaje dado, así como su notación.

La función de una gramática es definir y enumerar las palabras y frases válidas de un lenguaje. La forma general definida por BNF es denominada regla de producción, y se representa como:

<regla> = sentencias y frases. *

Las partes de la forma general BNF se definen como sigue:

- El “lado izquierdo” o regla es el identificador único de las reglas definidas para el lenguaje. Puede ser cualquier palabra, con la condición de estar encerrada entre los símbolos < >. Este elemento es obligatorio en la forma BNF.
- El “operador de asignación” = es un elemento obligatorio.
- El “lado derecho”, o sentencias y frases, define todas las posibilidades válidas en la gramática definida.
- El “delimitador de fin de instrucción” (punto) es un elemento obligatorio.

Un ejemplo de una gramática que define las opciones de un menú asociado a una aplicación de ventanas puede ser:

<raiz> = ARCHIVO
I EDICION
I OPCIONES
I AYUDA.

En este ejemplo, podemos encontrar claramente los conceptos de símbolo terminal y símbolo no-terminal. El primero es una palabra del vocabulario definido en un lenguaje, por ejemplo, ARCHIVO, EDICION, etc.

Y el segundo, es una regla de producción de la gramática, por ejemplo:

<raiz> = <opcion>.
<opcion> = ARCHIVO
I EDICION
I OPCIONES
I AYUDA

Otro ejemplo más complejo que involucra el uso de frases es el siguiente:

<raiz> = Hola mundo I Hola todos

En los casos anteriores se usó el símbolo I (OR), el cual denota opciones de selección mutuamente excluyentes, lo que quiere decir que sólo se puede elegir una opción entre ARCHIVO, EDICION, OPCIONES y AYUDA, en el primer ejemplo; así como “Hola mundo” y “Hola todos” en el segundo.

Un ejemplo real aplicado a una frase simple de uso común como “Me puede mostrar su licencia”, con la opción de anteponer el título Señorita, Señor o Señora, se puede estructurar de la manera siguiente en una gramática BNF:

<peticion> = <comando> I <titulo> <comando> .
<titulo> = Señor I Señora I Señorita.
<comando> = Me puede mostrar su licencia.

Hasta este momento solo habíamos definido reglas de producción que hacían referencia a símbolos terminales. Sin embargo, en el ejemplo anterior se puede observar que la regla <peticion> está formada solo por símbolos no-terminales. Otra propiedad que nos permite visualizar el ejemplo es la definición de frases y palabras opcionales, es decir, si analizamos la regla de producción <peticion>, detectamos que una petición válida puede prescindir del uso del símbolo <titulo>; mientras que el símbolo <comando> se presenta en todas las posibilidades válidas de <peticion>.

Una sintaxis para simplificar el significado de <peticion> es el operador “?”:

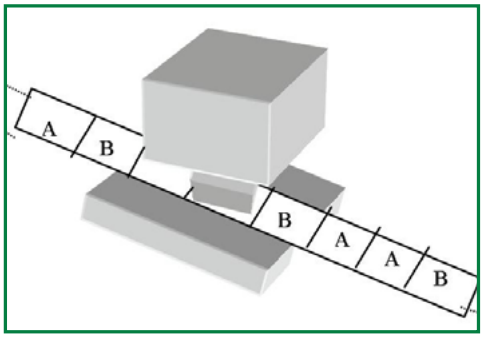
<peticion> = <titulo>? <comando>.

Con la sintaxis anterior se define que el símbolo <titulo> es opcional, o sea, que puede ser omitido sin que se pierda la validez de la <peticion>.

1.4. Máquina de Turing

El concepto de algoritmo como un conjunto de pasos lógicos y secuenciales para solucionar un problema fue implementado en 1936 por Alan Turing, matemático inglés, en la máquina de Turing. Esta se integra de tres elementos: cinta, cabeza de lectura-escritura y programa. La cinta tiene la propiedad de ser infinita (no acotada por sus extremos) y estar dividida en segmentos del mismo tamaño, los cuales almacenan cualquier símbolo o están vacíos. Asimismo, puede interpretarse como el dispositivo de almacenamiento.

La cabeza de lectura-escritura es el dispositivo que lee y escribe en la cinta. Actúa en un segmento y ejecuta sólo una operación a la vez. También tiene un número finito de estados que cambian de acuerdo con la entrada y las instrucciones definidas en el programa que lo controla.



El último elemento, el programa, es un conjunto de instrucciones que controlan los movimientos de la cabeza de lectura-escritura, indicándole hacia dónde ha de moverse y si debe escribir o leer en la celda donde se encuentre.

Actualmente, la máquina de Turing es una de las principales abstracciones utilizadas en la teoría moderna de la computación, pues auxilia en la definición de lo que una computadora puede o no puede hacer.

Máquina de Turing como función

La máquina de Turing es una función cuyo dominio se encuentra en la cinta infinita, donde se plasma su co-dominio, esto es: todos los posibles valores de entrada se hallan en la cinta y todos los resultados de su operación se expresan también ahí.

Hay problemas que pueden resolverse mediante una máquina de Turing y otros que no. Los primeros se llaman *computables* y los segundos *no computables* o indecidibles. Son *indecidibles* por no tener una solución para todas sus posibles entradas.

Un ejemplo de la máquina de Turing lo tenemos en la enumeración de binarios, como se muestra a continuación en los siguientes ejemplos:

Ejemplo 1

Diseñar una máquina de Turing que enumere los códigos binarios de la siguiente forma:

0, 1,10, 11, 110, 111,1110,...

Solución:

Se define la máquina mediante:

Q = {q1} (Conjunto de estados)
∑ = {0, 1} (Alfabeto de salida)
Γ = {0, b} (Alfabeto de entrada)
s = q1 (Estado inicial)

Y ∂ dado por las siguientes instrucciones:

∂ (q1, 0)= (q1, 1, D)

Que se lee como: si se encuentra en estado q1 y lee un cero, entonces cambia a estado q1, escribe uno y desplazar a la derecha.

∂ (q1, b) = (q1, 0, Sin Desplazamiento)

Que se lee como: si se encuentra en estado q1 y lee una cadena vacía, entonces cambia a estado *q1*, escribe un *cero y no hay desplazamiento*.

Si en esta máquina de Turing se comienza con la cabeza de lectura-escritura sobre el 0, tenemos la secuencia:

(q1, 0̲b)├ (q1, 1b̲)├ (q1, 10̲b)├ (q1, 11b̲)├(q1, 110̲b)├

NOTA: el carácter subrayado indica que la cabeza lectora/grabadora de la máquina de Turing está posicionada sobre ese carácter.

Ejemplo 2

Diseñar una máquina de Turing que acepte el lenguaje L= {$a^n b^m$| n y m >=1} por medio de la eliminación de las letras ***a*** y ***b*** que están en los extremos opuestos de la cadena. Es decir, usando *c* y *d*, la cadena *aaabbb* primero sería transformada en *caabbd*; después, en *ccabdd*; y por último, en *cccddd*.

Solución: Consideremos la máquina de Turing definida mediante:

Q = {q1, q2, q3, q4, q5} (Conjunto de estados)

∑ = {a, b, c, d} (Alfabeto de salida)

Γ = {a, b, ß} (Alfabeto de entrada)

F = {q4} (Estado final)

s = {q1} (Estado inicial)

Y ∂ dado por las siguientes instrucciones:

∂ (q1, a)	**=**	**(q2, c, D)**
∂ (q1, b)	**=**	**(q2, c, D)**
∂ (q1, c)	**=**	**(q4, d, ALTO)**
∂ (q1, d)	**=**	**(q4, d, ALTO)**
∂ (q2, a)	**=**	**(q2, a, D)**
∂ (q2, b)	**=**	**(q2, b, D)**
∂ (q2, ß)	**=**	**(q5, ß, I)**
∂ (q2, d)	**=**	**(q5, d, I)**
∂ (q3, a)	**=**	**(q3, a, I)**
∂ (q3, b)	**=**	**(q3, b, I)**
∂ (q3, c)	**=**	**(q1, c, D)**
∂ (q5, a)	**=**	**(q3, a, I)**
∂ (q5, b)	**=**	**(q3, d, I)**
∂ (q5, c)	**=**	**(q4, c, ALTO)**

Si en esta máquina de Turing se comienza con la cabeza lectora-escritora sobre la primera de la izquierda, se tiene esta secuencia de movimientos:

(q1, a̲aabbb)├	(q2, ca̲abbwb)├	(q2, caa̲bbb)├	(q2, caab̲bb)├	(q2, caabb̲b)├
(q2, caabb̲b)├	(q2, caabbb ß)├	(q5, caabbb̲ ß)├	(q3, caabb̲dß)├	(q3, caab̲bd)├
(q3, caa̲bbd)├	(q3, ca̲abbd)├	(q3, c̲aabbd)├	(q1, a̲abbd)├	(q2, cca̲bbd)├
(q2, cca̲bbd)├	(q2, ccabb̲d)├	(q2, ccabbd̲)├	(q5, ccabb̲d)├	(q3, ccab̲dd)├
(q3, cca̲bdd)├	(q3, cc̲abdd)├	(q1, cca̲bdd)├	(q2, cccb̲dd)├	(q2, cccbd̲d)├
(q5, cccb̲dd)├	(q3, ccc̲ddd)├	(q1, cccd̲dd)├	(q4, cccd̲dd)	

ALTO.

Con lo anterior queda ejemplificado el diseño de una máquina de Turing y su desarrollo.

(Moduna, 2000, pp. 68-71).

RESUMEN DE LA UNIDAD



Un algoritmo es una serie de pasos definidos y lógicos para resolver un problema, un algoritmo debe ser finito, claro, y siempre se debe obtener el mismo resultado.

Por su parte, un autómata es un modelo computacional consistente en un conjunto de estados, un estado inicial, un alfabeto de entrada y una función de transición.

Uno de los autómatas más conocidos es la máquina de Turing, aunque la máquina de Turing nunca fue construida, teóricamente es un dispositivo que manipula símbolos sobre una tira de cinta de acuerdo con una tabla de reglas.

Puede ser adaptada para simular la lógica de cualquier algoritmo en una computadora. Por ultimo también se habló del tema de los lenguajes formales. Un lenguaje formal está constituido por un alfabeto, un vocabulario y una gramática.

ACTIVIDADES DE APRENDIZAJE

Unidad 1, actividad inicial. *Adjuntar archivo.* A partir del estudio del contenido de la unidad, elabora un mapa conceptual u organizador gráfico sobre los temas vistos. Dentro del documento podrás encontrar recomendaciones de software para su elaboración.

1. **Unidad 1, actividad 1. *Adjuntar archivo.*** A partir de la lectura de la unidad, define qué es un algoritmo, y sus características principales.

2. **Unidad 1, actividad 2. *Adjuntar archivo.*** A partir de la lectura de la unidad, define qué es un autómata, y sus características principales.

3. **Unidad 1, actividad 3**. ***Adjuntar archivo.*** Describe cómo funciona la máquina de Turing.

4. **Unidad 1, actividad 4. *Adjuntar archivo.*** Elabora un algoritmo que describa los pasos necesarios para armar un modelo a escala de un automóvil, imagina que ya cuentas con todas las piezas y estas ya se encuentran pintadas. Representa el algoritmo en forma de pasos sucesivos (tipo receta de cocina) empleando un lenguaje natural; evita utilizar un lenguaje de programación.

5. **Unidad 1, actividad 5. *Foro.*** Entra al ***Foro*** titulado ***La máquina de Turing y la computación actual*** y con tus compañeros expliquen la relación que existe entre la máquina de Turing, sus aplicaciones y el funcionamiento de las computadoras actuales.

6. **Unidad 1, actividad complementaria 1. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

7. **Unidad 1, actividad complementaria 2. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

8. **Unidad 1, actividad complementaria 3. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

ACTIVIDAD INTEGRADORA

Adjuntar archivo.

En esta unidad aprendiste a distinguir los fundamentos teóricos que sustentan el concepto de algoritmo, a partir de los aprendizajes obtenidos, desarrolla la siguiente actividad integradora:

Elabora un algoritmo de ordenamiento de datos donde se genere un intercambio de posición de los números 24 y 9, dando como salida 9 y 24 en este orden. Te sugerimos utilizar tres variables: num1, num2 y aux, para que puedas realizar el intercambio. Representa el algoritmo en forma de pasos sucesivos (tipo receta de cocina) empleando un lenguaje natural; evita el lenguaje de programación y el diagrama de flujo.

CUESTIONARIO DE REFORZAMIENTO

Adjuntar archivo.

Contesta las siguientes preguntas.

1. ¿Qué es un algoritmo?
2. ¿Cuáles son las características de un algoritmo?
3. ¿Qué es un autómata?
4. ¿Cuáles son las características de un autómata?
5. En el campo de los autómatas, ¿qué es un alfabeto?
6. ¿Cuál es la definición de lenguaje formal?
7. ¿Qué es y para qué sirve una gramática?
8. ¿Qué es una máquina de Turing?
9. Explica por qué una máquina de Turing puede ser considerada un autómata.
10. ¿Qué es un proceso computable?

AUTOEVALUACIÓN

I. Contesta las siguientes preguntas, elige una V si la frase es verdadera o una F si es falsa.

1. Un algoritmo debe obtener el mismo resultado siempre.
2. Un algoritmo puede saltarse pasos y aun así obtener un resultado correcto.
3. Una forma de representar algoritmo es a través de un diagrama entidad-relación
4. Un algoritmo cualitativo contempla cálculos matemáticos
5. Un algoritmo puede representar secuencias infinitas
6. Los procesos computables y no computables pueden implementarse en un algoritmo o máquina de Turing.
7. En la máquina de Turing, la cabeza de lectura-escritura es el dispositivo que lee y escribe en la cinta. Actúa en un segmento y ejecuta sólo una operación a la vez.
8. La cinta de una máquina de Turing es necesariamente finita, ya que está definida por el tamaño del programa.
9. El programa es un conjunto de instrucciones que controlan los movimientos de la cabeza de lectura-escritura.
10. La cinta es un dispositivo solo de lectura y no de almacenamiento.

II. Contesta las siguientes preguntas, elige la letra del inciso que conteste de manera correcta.

a) Algoritmo.	b) Cualitativo.	c) Alfabeto.	d) Cuantitativo.	e) Autómata.

1. Conjunto de todos los símbolos válidos o posibles para una aplicación. ( )
2. Conjunto de pasos finitos y lógicos para resolver un problema. ( )
3. La máquina de Turing es el ejemplo de un… ( )
4. Un algoritmo que no implica cálculos matemáticos es un algoritmo… ( )
5. Un algoritmo que implica cálculos matemáticos es un algoritmo… ( )

III. Selecciona la respuesta correcta.

1. Es una característica de un algoritmo.
   a) Acepta criterios en su desarrollo.
   b) Puede omitir pasos.
   c) En ocasiones, no obtiene un resultado.
   d) Es finito.

2. Definir y enumerar las palabras y frases válidas de un lenguaje es función de.
   a) Un alfabeto.
   b) Los símbolos.
   c) La sintaxis.
   d) La gramática.

3. Autómata formado por cinta, cabeza de lectura-escritura y programa.
   a) Máquina de Turing.
   b) El Dibujante.
   c) Los Músicos.
   d) El Telar Automático.

4. Problema que no puede implementarse en una máquina de Turing por no tener solución para todas sus posibles entradas.
   a) Computable.
   b) Indecidible.
   c) Decidible.
   d) Disfuncional.

5. Es un modelo computacional consistente en un conjunto de estados bien definidos, un estado inicial, un alfabeto de entrada y una función de transición.
   a) Algoritmo.
   b) Función.
   c) Autómata.
   d) Programa.

RESPUESTAS EXAMEN DE AUTOEVALUACIÓN

En este apartado encontrarás las respuestas al examen por unidad.

UNIDAD 1	
I. Solución	
1.	V
2.	F
3.	F
4.	F
5.	F
6.	F
7.	V
8.	F
9.	V
10.	F

UNIDAD 1	
II. Solución	
1.	c
2.	a
3.	e
4.	b
5.	d

UNIDAD 1	
III. Solución	
1.	d
2.	d
3.	a
4.	b
5.	b

REFERENCIAS

Moliner María (2002). *Diccionario de uso del español*. Gredos.

Meduna Alexander. (2000). *Automata and Languages: Theory and Aplications*, Springer.

Oviedo Regino E. M. (2015). *Lógica de programación orientada a objetos*. Ecoe Ediciones.

UNIDAD 2

FUNDAMENTOS DE PROGRAMACIÓN

OBJETIVO PARTICULAR

Al finalizar la unidad, el alumnado identificará los fundamentos teóricos de la programación incluidas las etapas del ciclo de vida de los programas.

TEMARIO DETALLADO

(6 horas)

2. Fundamentos de programación
2.1. Definición de algoritmo
2.1.1. Definición del problema
2.1.2. Análisis del problema
2.1.3. Diseño de solución
2.1.4. Codificación
2.1.5. Pruebas y depuración
2.1.6. Documentación
2.1.7. Implantación
2.1.8. Mantenimiento
2.2. Conceptos básicos de programación
2.2.1. Definición de programación
2.2.2. Definición de lenguaje de programación
2.2.3. Diferencias entre algoritmo y programa
2.2.4. Traductores de lenguajes de programación
2.2.4.1. Intérpretes
2.2.4.2. Compiladores
2.2.5. Fases de compilación o emulación

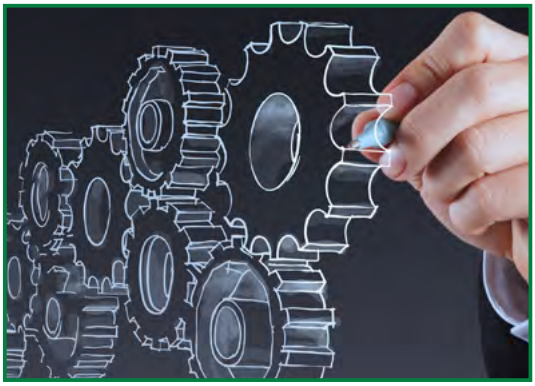
ACTIVIDAD DIAGNÓSTICA



Adjuntar archivo.

Sin consultar ninguna fuente y con tus propias palabras describe qué es un programa de computadora.

INTRODUCCIÓN



Un programa de computadora es el paso siguiente después de desarrollar un algoritmo. Un programa es la ejecución de un algoritmo en una computadora.

Un algoritmo debe ser implementado en un lenguaje de programación, sin grandes modificaciones.

Un programa de computadora es el paso siguiente después de desarrollar un algoritmo, es propiamente la ejecución de un algoritmo por medio de una computadora.

La diferencia entre un **algoritmo** y un **programa** es que el primero debe ser implementado en un lenguaje de programación, sin grandes modificaciones; mientras que el programa es desarrollado en un lenguaje de programación, tiene un ciclo de vida que inicia en el análisis del problema, continúa con la codificación, implementación, posteriormente una serie de pruebas y finalmente termina con el mantenimiento de este.

Para que el programa pueda ser funcional, necesita de un compilador o intérprete para generar un código que pueda ser ejecutado en una computadora.

2.1.
Ciclo de vida de un programa

Un programa de computadora tiene un ciclo de vida, el cual inicia con la definición de un problema para resolver. Una vez definido, se procede a analizarlo y se determina si es posible resolver el problema a través de un programa, ya que existen problemas que no pueden ser resueltos por una computadora.

Si se determina que el problema tiene una solución, entonces se realiza un diseño a través de un pseudocódigo o un diagrama de flujo. Posterior a esto, se realiza la codificación del programa en algún lenguaje de programación.

Una vez terminada la codificación se procede a probar el código y en su caso corregir el programa.

Posteriormente se desarrolla la documentación del programa y se implementa. Por último, el programa debe ser mantenido y actualizado, en caso de ser necesario.

El ciclo de vida se representa en el siguiente esquema:

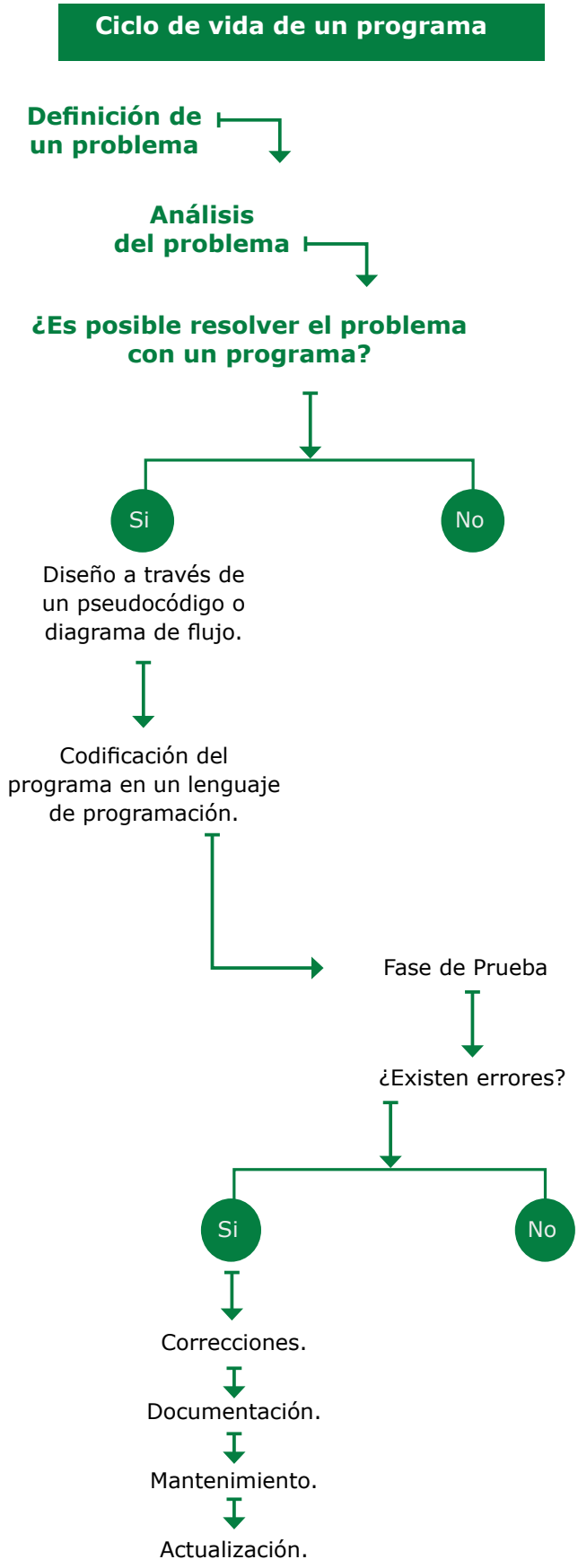


Figura 2.1. Ciclo de vida de un programa.

2.1.1. Definición del problema

El primer paso para resolver un problema es definirlo previamente. No es posible resolverlo si antes no se define de manera detallada. Además, Si el problema no se acota, no será posible resolverlo, porque en ocasiones suele ocurrir que lo que aparentemente es un problema resulta ser un conjunto de problemas, por lo cual será más difícil de resolver.

Entonces hay que definir y acotar el problema, veamos este concepto de forma más detallada.

Definición y acotación del problema: consiste en dar una descripción del problema con los datos de entrada y el resultado. Esta descripción se puede hacer con un texto descriptivo, el problema debe ser concreto y no involucrar otros problemas.

Veamos un ejemplo con el siguiente problema:

Una farmacia desea controlar el inventario de los medicamentos que vende. Actualmente la farmacia lleva el control de sus inventarios a través de una hoja de cálculo electrónica donde el vendedor anota los productos vendidos.

La definición del problema es que no existe un control preciso del inventario de los productos, ya que a veces el vendedor no registra la venta y compra de productos de manera correcta.

Para acotar el problema, se debe mencionar que la gestión de inventarios es a través de una hoja de cálculo y no se realiza de forma óptima.

2.1.2. Análisis del problema

> El análisis del problema es un proceso para recabar la información necesaria para emprender una acción que lo solucione.

Una vez definido y acotado el problema, el paso siguiente es determinar si el problema puede ser resuelto por un programa computacional.

Hay que tomar en cuenta, que no todos los problemas se pueden resolver con un programa de cómputo; por ejemplo, desarrollar un programa que pueda predecir con precisión cuándo ocurrirá un terremoto. Actualmente no existe un programa computacional que pueda realizar dicha predicción.

En el caso de que el problema se pueda resolver por medio de un programa de cómputo, se debe realizar un análisis del problema, fruto del cual se obtendrá una documentación en donde se especificará de forma escrita qué tiene que hacer el programa, tanto en la descripción de los responsables que realizan los programas como las interfaces de los usuarios.

Por lo tanto, es necesario utilizar diferentes técnicas para recolectar la información, como entrevistas, cuestionarios u observaciones directas del problema a resolver.

Se deben conocer e identificar a las personas afectadas por el proceso, desde el cliente que paga el proyecto hasta usuarios finales y terceras personas o partes interesadas. Es importante que los usuarios finales participen adecuadamente en esta fase del proyecto.

Diversos problemas requieren soluciones diferentes. Un problema puede llegar a tener más de una forma de solucionarlo, la dificultad se centra en saber cuál es la mejor solución, ya sea la más económica, la más fácil de desarrollar, o la más rápida de implementar. Entonces, dependiendo de las necesidades encontramos la solución para elegir.

Si la solución del problema es impostergable, entonces se debe optar por la solución más rápida de implementar. Si los recursos son escasos, entonces se debe optar por la solución más económica.

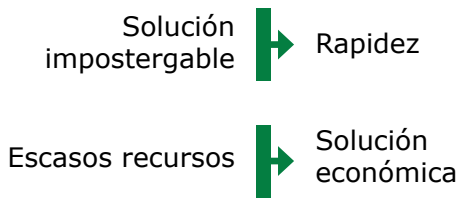


Figura 2.2. Toma de decisiones en el análisis del problema. Basado en: (Joyanes, 2020, pp. 43-44).

2.1.3. Diseño de solución

En la fase de diseño es necesario representar las características del sistema de información. Esto incluye, los algoritmos, estos pueden ser representados en pseudocódigo.

El pseudocódigo es la representación escrita de un algoritmo, muestra en forma de texto los pasos para solucionar un problema. El pseudocódigo posee una sintaxis para realizar la representación del algoritmo, veamos dicha sintaxis.

El pseudocódigo está limitado por las etiquetas de:

INICIO y FIN.
Dentro de estas etiquetas se escriben las instrucciones del algoritmo. Las palabras reservadas se escriben con mayúsculas. Se recomienda una sangría para que el código sea más fácil de leer.
Para la entrada de datos se utiliza la palabra:

LEER.
Para indicar el despliegue de datos se utiliza la palabra:

ESCRIBIR.
La entrada de datos se realiza, por defecto, desde el teclado, que es la entrada estándar del sistema. El despliegue de datos se realiza, por defecto, en la pantalla, que es la salida estándar del sistema.

Veamos un ejemplo de un algoritmo en pseudocódigo que suma dos números:

```
ALGORITMO Suma
VAR
ENTERO Numero1, Numero2, Resultado;
INICIO
    ESCRIBIR("Introduce dos números: ");
        LEER(Numero1, Numero2);
      Resultado <- Numero1 + Numero2;
    ESCRIBIR("La suma es:", Resultado);
FIN
```

Veamos el código:

ALGORITMO Suma

En esta línea se indica el nombre del algoritmo.

VAR

ENTERO Numero1, Numero2, Resultado;

En esta línea se indican tres variables y su tipo de dato, en este caso, tipo entero.

INICIO

En esta línea se indica el inicio del algoritmo.

```
ESCRIBIR("Introduce dos números: ");
    LEER(Numero1, Numero2);
```

En las líneas anteriores se muestra un texto que pide dos números, y la función LEER, los captura en las variables Numero1 y Numero2.

```
    Resultado <- Numero1 + Numero2;
ESCRIBIR("La suma es:", Resultado);
FIN
```

En la variable Resultado se almacena la suma de las dos variables y se muestra el resultado en pantalla, por último, la etiqueta FIN indica el fin del programa (Joyanes, 2020, pp. 44-47).

2.1.4. Codificación

La codificación es la fase donde se captura el programa utilizando un lenguaje de programación. Existen diversos lenguajes de programación, como ejemplos de estos lenguajes se encuentra el lenguaje C, Lenguaje C++, Java, PHP y Python.

La codificación se realiza escribiendo el código fuente, el código fuente son líneas de texto que indican qué debe realizar un programa. Dicho código fuente contiene información, como las funciones que realizará el programa, en qué orden las realizará, y en caso de un error, debe indicar qué pasos se seguirán para dar aviso del error.

Así los pasos son los siguientes:

1	2	3
Escritura del código fuente	Compilación o interpreta-ción, según el lenguaje de programación elegido	Pruebas y depuración del programa

Figura 2.3. Pasos a seguir en la fase de codificación.

Veamos un ejemplo de un programa desarrollado en el lenguaje C, cabe destacar que existen varios IDE (por sus siglas en inglés, *Integrated Development Environment*) para la edición y ejecución de programas en dicho lenguaje, ejemplos de estos IDE son: Eclipse, Dev-C++ y NetBeans.

```
/* Escribe un mensaje en pantalla */
/* incluye biblioteca donde se define E/S */
# include <stdio.h>
void main ( )
{
printf("\nIntroducción a la programación en lenguaje C");
return (0);
}
```

Veamos el programa línea por línea para explicarlo. El programa muestra un mensaje en pantalla, veamos las primeras dos líneas.

```
/* Escribe un mensaje en pantalla */
/* incluye biblioteca donde se define E/S */
```

Las anteriores líneas son comentarios, no tienen efecto sobre el programa, veamos la siguiente línea.

```
# include <stdio.h>
```

Esta línea indica que hay que incluir una librería, una librería es un conjunto de funciones, en este caso esta librería contiene funciones de entrada y salida, es decir funciones para la entrada y despliegue de datos. Veamos la siguiente línea:

```
void main ( )
```

Esta línea indica que es la función principal del programa y no devuelve ningún valor. Veamos la siguiente línea:

```
{
```

La llave indica que inicia el programa. Veamos las siguientes líneas:

```
printf("\nIntroducción a la programación en lenguaje C");
return (0);
}
```

La primera línea indica que se imprima en pantalla el texto "Introducción a la programación en lenguaje C", la segunda línea indica que el programa no devuelve ningún valor y con la llave se cierra el programa.

La codificación es la escritura de las instrucciones de un algoritmo, en un lenguaje de programación (en el ejemplo se utiliza el lenguaje C), en el caso del pseudocódigo es importante respetar la sintaxis y el orden de las instrucciones. Si hubiera un error de sintaxis en el pseudocódigo, sería un error, pero aun así las instrucciones pueden interpretarse.

Pero en el caso de la codificación, no se puede cometer un solo error de sintaxis, o de orden en las instrucciones, ya que de lo contrario el programa se ejecutaría de forma incorrecta o simplemente no se ejecutaría (Joyanes, 2020, pp. 47-48).

2.1.5.
Pruebas y depuración

Depuración

Una anécdota sobre el origen de este término (del inglés *debugging*, 'eliminación de bichos') cuenta que, en la época de la primera generación de computadoras constituidas por bulbos, encontraron una polilla entre los circuitos que era la responsable de la falla del equipo (Muñoz, 2016). De allí nació la expresión para indicar que el equipo o los programas presentan algún problema.

La depuración, entonces, es el proceso de identificación y corrección de errores de programación. Para depurar el código fuente, el programador se vale de herramientas de *software* que le facilitan la localización y eliminación de errores. Los compiladores son un ejemplo de estas herramientas.

Se dice que un programa está depurado si está libre de errores. Cuando se depura un programa se hace un seguimiento de su funcionamiento y se van analizando los valores de sus distintas variables, así como los resultados obtenidos de los cálculos del programa.

Una vez depurado el programa, se solucionan los posibles errores encontrados y se procede a depurar nuevamente. Estas acciones se repiten hasta que el programa no contiene errores.

Los errores más sencillos de detectar son de sintaxis, presentes cuando alguna instrucción está mal escrita o existen errores en la puntuación.

Existen también errores lógicos, en este caso, aunque el programa no contenga fallas de sintaxis, no realiza la tarea para la que fue creado. Pueden presentarse valores erróneos de las variables, ejecuciones de programa que no terminan, imprecisiones en los cálculos, etcétera.

Estos últimos son los más difíciles de detectar, por lo que se debe realizar un seguimiento detallado del programa.

Pruebas

El propósito de las pruebas es asegurar que el programa produce los resultados definidos en las especificaciones funcionales. El programador utilizará los datos de prueba para comprobar que el programa genera los resultados correctos.

En caso de que los datos sean erróneos debe indicar el error, y por qué se produjo dicho error; así como una acción correcta en el caso de datos incorrectos.

Concluida la programación, el desarrollador volverá a usar los datos de prueba para verificar que el programa dará los resultados correctos. En esta ocasión, concentrará su atención también en la interacción correcta entre los diferentes programas y el funcionamiento completo del programa.

Verificará lo siguiente:

- Todos los valores incluidos en los datos de prueba.
- Todos los valores que el programa de prueba actualice.
- Todos los cálculos efectuados por el programa.
- Los casos en que haya interacción con una base de datos.
- Los casos en que haya interacción con otro programa.
- Todas las condiciones del programa.
- Se cotejará la lógica del programa.

Figura 2.4. Interacción correcta para el buen funcionamiento del programa.

Las pruebas consisten en introducir datos a un programa para determinar si los resultados que arroja dicho programa son correctos. En caso de que existan errores, esto es que los resultados que arroja el programa son incorrectos, se procede a la depuración. La depuración es el proceso de encontrar y corregir los errores, entonces se procede a la realización de nuevas pruebas.

El proceso de prueba se repite hasta que todos los errores se han eliminado (Joyanes, 2020, pp. 48-49).

2.1.6. Documentación

La documentación de los programas es parte de la documentación que debe hacerse de un sistema de información. Se deben documentar los nombres de los programas, el nombre del programador, la fecha de creación, la función del programa, los datos de entrada y otros datos que el programador considere adecuados. Veamos los datos que debe contener la documentación.

- ***Nombre del programa y programador.*** Indica el nombre del programa y el nombre del programador que lo realizó.
- ***Descripción.*** Señala la función del programa.
- ***Variables y constantes.*** Describe las variables y constantes que se utilizan en el programa.
- ***Fecha de creación y/o modificación.*** Fecha a partir de la cual se libera el programa y si se modifica, la fecha de la modificación del programa.
- ***Datos de entrada***. Describe el dato o datos que se deben introducir en el programa.
- ***Datos de salida.*** Describe el dato o datos que debe arrojar el programa.
- ***Mensajes de ejecución correcta.*** Describe los mensajes que indican que la ejecución del programa es correcta.
- ***Mensajes de error.*** Describe los mensajes que indican la ejecución errónea del programa.

Figura 2.5. Datos que debe contener la documentación. Fuente: Basado en (Joyanes, 2020, p. 49).

La importancia de la documentación radica en que describe el funcionamiento de un programa, las variables, constantes y funciones que lo componen, así como los datos con que debe ser alimentado el programa y los datos que debe arrojar el mismo, los mensajes de una ejecución correcta y una ejecución incorrecta, así como quién realizó o modificó el programa y cuándo se liberó o modificó el programa. En caso de que se requiera realizar un mantenimiento al programa, toda esta información será útil para dicha tarea, de lo contrario, el programador a cargo de dicho mantenimiento tendría que comenzar desde cero (Joyanes, 2020, p. 49).

2.1.7. Implantación

Una vez terminado el programa, y comprobado que no existen errores en el mismo, es el momento de preparar el lanzamiento del programa mediante la instalación del *software* en su versión final.

La implantación del programa es la instalación del programa en una computadora, si fuera necesario hay que realizar las adecuaciones de *hardware* necesarias en la computadora donde será instalado, por ejemplo, si el programa requiere ciertos requisitos de memoria principal o secundaria, también las modificaciones necesarias de *software*, por ejemplo, una actualización del sistema operativo.

Cabe destacar en esta fase que se incluye la capacitación a los usuarios del programa, en la que, por lo general, como apoyo, se elaboran una serie de manuales en los que se explica a detalle el funcionamiento del programa.

La etapa de implantación es la fase culminante del desarrollo de un programa, en esta fase se establecen plazos para la capacitación de los usuarios y fechas para la liberación del programa, cabe destacar que es muy importante que se involucren todos los interesados en el nuevo programa para facilitar y garantizar el éxito del mismo.

2.1.8. Mantenimiento

A medida que el cliente utiliza el nuevo programa surgirá la necesidad de hacer cambios y mejoras. Con los nuevos requisitos se necesitará una funcionalidad adicional. El mantenimiento del *software* se realiza con la finalidad de añadir nuevas funciones.

Los usuarios de los programas solicitarán los cambios necesarios al área de sistemas, con el fin de que estos continúen operando de forma adecuada. Para ello, se le debe dar mantenimiento de forma periódica a los programas.

El mantenimiento presenta las modalidades descritas a continuación:

PREVENTIVO
Cuando los programas no presentan errores, pero hay necesidad de realizar respaldos, verificar la integridad de los datos o verificar si los resultados son correctos.
CORRECTIVO
Los programas presentan algún error en algún cálculo, por lo que es necesario revisar la codificación para corregirlos. Se deben realizar las pruebas al programa, imprimiendo los reportes que generan y verificar si los cálculos que estos ofrecen son correctos.

ADAPTATIVO

Los programas no tienen errores, pero se requiere alguna actualización por una nueva versión del programa, una nueva versión del sistema operativo o un nuevo equipo de cómputo con ciertas características. Es decir, hay que adaptar los programas a las nuevas necesidades o a la nueva tecnología de software o de hardware.

Figura 2.6. Modalidades de mantenimiento.

En cualquier caso, el usuario debe realizar la solicitud por escrito puntualizando el tipo de mantenimiento que requiere y remitirla al área de sistemas para su revisión y valoración. En tanto, el personal del área de sistemas hará una orden de trabajo para proceder a realizar el servicio pedido.

La etapa de mantenimiento es importante porque posibilita dar mayor vida útil al desarrollo de un programa, ya que generalmente se requieren realizar actualizaciones. Es muy recomendable tomar en cuenta los tipos de mantenimiento que se describieron anteriormente para ofrecer una mejor calidad en mantenimiento de los programas (Joyanes, 2020, p. 49).

A continuación, se estudiarán algunos conceptos básicos de programación.

2.2. Conceptos básicos de programación

Para entender la programación de computadoras, es necesario conocer algunos conceptos básicos que a continuación se describen:

Programa: es un conjunto de instrucciones precisas, en un lenguaje “entendible” por la computadora. Hay que notar que la computadora solo comprende el lenguaje máquina, es decir, un lenguaje basado en el sistema binario que consiste en la combinación de 0 y 1.

Programación: se denomina programación a todo el proceso que conlleva la construcción de programas.

Lenguaje de programación: un lenguaje de programación es un lenguaje diseñado para representar algoritmos de forma inteligible para una computadora. Existen muchos lenguajes de programación, pero todos deben transformar el código fuente en el lenguaje máquina, cuyas instrucciones están codificadas en forma de secuencias de ceros y unos (bits).

Los lenguajes de programación se encuentran formados por varios elementos, entre los más importantes se encuentran:

Librería: es un conjunto de funciones codificadas en un lenguaje de programación.

Palabras reservadas: es una palabra que tiene un significado especial y no puede ser utilizada como un identificador de objetos, un objeto puede ser una variable.

Variables: las variables permiten acceder a la memoria para almacenar y recuperar los datos utilizados.

Operadores: estos son aritméticos, lógicos y relacionales.

Constantes: son objetos cuyo valor no cambia.

Tipos de datos: es un atributo de los datos que indica la clase de datos a utilizar, por ejemplo: entero, carácter, flotantes.

Comentarios: elemento destinado a realizar anotaciones legibles en el código fuente.

Sentencias: Estructuras que condicionan el flujo de un programa (Joyanes, 2020, pp. 89-102).

2.2.1. Definición de programación

La programación es el proceso que conlleva la construcción de programas. Para construir un programa hay que seguir los siguientes pasos: el diseño del programa, que se puede realizar mediante el pseudocódigo, la codificación que es la escritura del programa en un lenguaje de programación, después de la captura se procede a realizar pruebas para determinar si el programa cumple con las especificaciones y, por último, el mantenimiento del programa, si se requiere modificarlo debido a nuevas necesidades, como ya se describió anteriormente.

2.2.2. Definición de lenguaje de programación

Un lenguaje de programación es un lenguaje de computadora utilizado para que un programador pueda indicarle a una computadora qué tareas debe realizar, un lenguaje de programación produce programas de *software*.

Un lenguaje de programación está diseñado para representar algoritmos en un lenguaje diseñado para ejecutar una serie de sentencias en una computadora.

Un lenguaje de programación permite escribir instrucciones que una computadora puede interpretar, cabe destacar que las instrucciones en el lenguaje de programación deben ser transformadas en lenguaje máquina mediante un compilador o un intérprete. Existen muchos lenguajes de programación, pero el único lenguaje que entiende la computadora es el lenguaje máquina, cuyas instrucciones están codificadas en forma de secuencias de ceros y unos o sea bits.

Veamos algunos ejemplos del código de dos lenguajes de programación, que envían un mensaje a pantalla. El mensaje a pantalla es “Hola Mundo”.

Programa en PHP

```
<html>
<head>
        <title>Prueba de PHP</title>
</head>
        <body>
        <?php
                echo ‘<p>Hola Mundo</p>’;
         ?>
        </body>
</html>
```

El programa en PHP utiliza etiquetas en HTML como las siguientes para mostrar un texto en un navegador web, las etiquetas HTML son las siguientes:

```
<html>
<head></head>
<title></title>
<body></body>
</html>
```

El programa en PHP inicia y termina con las siguientes etiquetas:

```
<?php
?>
```

En esta parte se envía un mensaje a pantalla.

```
echo '<p>Hola Mundo</p>';
```

Programa en Java

```
public class HolaMundo {
public static void main(String[] args) {
System.out.println("Hola Mundo");
  }
}
```

En esta línea se indica el nombre de la clase de nombre HolaMundo

```
public class HolaMundo {
```

En esta línea se encuentra el método principal.

```
public static void main(String[] args) {
```

Por último, esta línea muestra en pantalla el mensaje "Hola Mundo" (Joyanes, 2020, pp. 22-23).

```
System.out.println("Hola Mundo");
```

2.2.3. Diferencias entre algoritmo y programa

Un algoritmo indica los pasos para solucionar un problema. Algunos problemas tienen distintas soluciones. Otros problemas no tienen solución algorítmica, por ejemplo, un algoritmo que determine de manera precisa cuándo ocurrirá una pandemia.

Un programa es la implementación de un algoritmo que posea una solución, la implementación de un algoritmo en una computadora es un programa.

2.2.4. Traductores de lenguajes de programación

Un traductor de un lenguaje de programación es un programa que toma como entrada un texto escrito en un código fuente y proporciona como salida un código en lenguaje máquina. Ejemplos de estos traductores son los intérpretes y los compiladores.

2.2.4.1. Intérpretes

Un intérprete es un programa que ejecuta directamente las instrucciones escritas en un lenguaje de programación, las sentencias se ejecutan línea por línea, esto trae ventajas y desventajas, la ventaja es que el código del programa se puede modificar con facilidad, ya que se tiene acceso al código fuente, la desventaja es que el desempeño del programa es menor comparado con un programa que fue implementado con un compilador. Un ejemplo de un lenguaje de programación que utiliza un intérprete es Perl, PHP, Python, JavaScript y Ruby, por mencionar algunos de los más populares.

2.2.4.2. Compiladores

Un compilador es un programa que transforma el código fuente de un lenguaje de programación a su equivalente en código máquina. Un lenguaje de programación que utiliza un compilador es el lenguaje C, C++, Pascal, Delphi y Fortran, por mencionar algunos de los más populares.

2.2.5. Fases de compilación o emulación

FASES DE UN COMPILADOR:

Los compiladores son programas de computadora que traducen un lenguaje de programación a lenguaje máquina. Un compilador toma un código fuente y produce un código objeto, le agrega las librerías necesarias y, por último, genera un código ejecutable.

La compilación se compone de varias etapas, las cuales son las siguientes:

1.-Analizador léxico

Esta fase consiste en leer el texto del código fuente carácter por carácter e ir generando los tokens (caracteres relacionados entre sí). Los tokens son la entrada para el siguiente proceso de análisis (análisis sintáctico). Dichos caracteres son palabras reservadas, operadores, constantes, variables, signos de puntuación llaves y paréntesis.

2.-Análisis sintáctico

La tarea del analizador sintáctico es procesar los tokens que le suministra el analizador léxico, comprobar que el orden sea correcto. Si la ordenación es correcta, obtiene la estructura jerárquica del programa en forma de árbol.

3.-Análisis semántico

Realiza las comprobaciones necesarias sobre el árbol sintáctico para determinar el correcto significado del programa, si el árbol es correcto obtiene un árbol semántico.

4.-Generación de código intermedio

El código intermedio genera un código mejorado, más fácil de traducir a lenguaje máquina. El código intermedio representa las operaciones a llevar a cabo en un lenguaje más cercano a la máquina final, aunque no a una máquina concreta sino a una máquina abstracta.

5.-Generador de código final

La generación del código final toma como entrada la representación intermedia y genera el código final. La generación de código final es un proceso más mecánico, ya que consiste en ir pasando las instrucciones del código intermedio al lenguaje ensamblador de la máquina a utilizar, más adelante, se puede ensamblar el código y obtener un código ejecutable, pero este proceso ya es automático. Al generarse un código ejecutable optimizado para un hardware en específico la ejecución del código es más veloz.



6.-Tabla de símbolos

La tabla de símbolos no es parte del proceso de compilación, pero es una estructura de datos donde se guarda información sobre las variables declaradas, funciones, sus parámetros y cuanta información vaya a ser necesaria para realizar el proceso de compilación.

El compilador se vale de esta estructura de datos para almacenar dicha información a lo largo del proceso de compilación.

7.-Gestor de errores

Detecta e informa de los errores que se produzcan, generalmente aparecen en la fase de análisis sintáctico y semántico. Este debe generar mensajes significativos para poder corregirlos (Ruiz, 2010, pp. 3-14).

RESUMEN DE LA UNIDAD

La programación es la definición de instrucciones en un lenguaje de programación, el cual, posteriormente, deberá ser transformado en lenguaje máquina para que pueda ser interpretado por una computadora.

El resultado es un conjunto de instrucciones al que llamamos programa. Al igual que un algoritmo, el objetivo del programa es resolver un problema, pero mientras el algoritmo lo resuelve de forma teórica, el programa resuelve un programa de forma práctica.

Existen distintos elementos para la creación de un programa, los dos elementos más importantes son los lenguajes de programación y los traductores.

Una computadora solo comprende el lenguaje máquina, o una secuencia de instrucciones en 1 y 0, pero para un ser humano es complicado entender el lenguaje máquina, por esto se desarrollaron los lenguajes de programación, lenguajes que un ser humano puede entender y utilizar para el desarrollo de un programa computacional.



Una vez desarrollado el programa, se requiere de un traductor para cambiar el código fuente en lenguaje máquina.

Hay dos tipos de traductores. Los intérpretes leen línea por línea el código fuente y lo transforman en lenguaje máquina. El compilador transforma el código fuente en un código ejecutable, el cual esta optimizado para el hardware donde fue compilado. El intérprete permite el desarrollo de programas de forma más sencilla y es fácil modificarlos, por otro lado, el compilador permite la generación de programas más veloces.

ACTIVIDADES DE APRENDIZAJE

Unidad 2, actividad inicial. *Adjuntar archivo.* A partir del estudio del contenido de la unidad, elabora un mapa conceptual u organizador gráfico sobre los temas vistos. Dentro del documento podrás encontrar recomendaciones de software para su elaboración.

1. **Unidad 2, actividad 1. *Adjuntar archivo.*** Después de haber dado lectura a la unidad, contesta lo siguiente: ¿Cuál es el ciclo de vida de un programa de computadora?

2. **Unidad 2, actividad 2. *Adjuntar archivo.*** Después de haber dado lectura a la unidad, contesta lo siguiente: Describe cuál es la importancia de la documentación en un programa de cómputo.

3. **Unidad 2, actividad 3. *Adjuntar archivo.*** Después de haber dado lectura a la unidad, contesta lo siguiente: Realiza un algoritmo que realice la división de dos números, utiliza el pseudocódigo.

4. **Unidad 2, actividad 4. *Adjuntar archivo.*** Después de haber dado lectura a la unidad, contesta lo siguiente: Describe las diferencias entre un compilador y un intérprete.

5. **Unidad 2, actividad 5. *Adjuntar archivo.*** Después de haber dado lectura a la unidad, contesta lo siguiente: Describe cómo realizar el análisis de un problema.

6. **Unidad 2, actividad complementaria 1. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

7. **Unidad 2, actividad complementaria 2. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

8. **Unidad 2, actividad complementaria 3. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

ACTIVIDAD INTEGRADORA

Adjuntar archivo.

En esta unidad aprendiste a identificar los fundamentos teóricos de la programación incluidas las etapas del ciclo de vida de los programas, a partir de los aprendizajes adquiridos, elabora la siguiente actividad integradora.

Mediante un diagrama de flujo, puedes hacer uso de software como Visio o cualquier otro de tu preferencia o el que tu asesor te indique a fin de describir las fases de la compilación de un programa de computadora.

CUESTIONARIO DE REFORZAMIENTO

Adjuntar archivo.

Contesta las siguientes preguntas.

1. ¿Qué es un programa de computadora?
2. ¿Qué es la programación?
3. ¿Qué es un lenguaje de programación?
4. ¿Qué es una variable?
5. ¿Qué es una constante?
6. ¿Qué es la codificación?
7. ¿Qué es un compilador?
8. ¿Qué es un intérprete?
9. Menciona dos lenguajes de programación que utilicen un intérprete.
10. Menciona dos lenguajes de programación que utilicen un compilador.

AUTOEVALUACIÓN

I. Contesta las siguientes preguntas, elige una V si la frase es verdadera o una F si es falsa.

1. El lenguaje C utiliza un compilador.
2. El lenguaje PHP utiliza un compilador.
3. Una entrevista es una forma de obtener información en la fase de diseño.
4. Una librería contiene funciones de un lenguaje de programación.
5. *Debugging* es sinónimo de depuración.
6. Las variables permiten acceder a la memoria para almacenar y recuperar los datos utilizados, su valor es constante.
7. Un ejemplo de tipo de dato es el signo de “+”
8. Una palabra reservada es una palabra que tiene un significado especial y no puede ser utilizada como un identificador de objetos.
9. Los errores lógicos se deben a fallas en la sintaxis.
10. Un ejemplo de lenguaje máquina es el lenguaje C.

II. Contesta las siguientes preguntas, elige la letra del inciso que conteste de manera correcta.

a) Preventivo	b) Correctivo	c) Depuración	d) Definición	e) Análisis

1. Es el proceso de identificación y corrección de errores de programación.

2. El primer paso para resolver un problema es la _____ del mismo.

3. A recabar la información necesaria para resolver un problema, se le conoce como.

4. Es el tipo de mantenimiento en el que entre sus tareas se encuentra la verificación de la integridad de los datos.

5. Es el tipo de mantenimiento que se ejecuta cuando los programas presentan algún error.

III. Selecciona la respuesta correcta.

1. Es una estructura tipo diccionario con operaciones de inserción, borrado y búsqueda.

a) Tabla de símbolos.
b) Lenguaje C.
c) Diagrama de flujo.
d) Pseudocódigo.

2. Es la una fase del ciclo de vida de un programa de computadora, donde se hacen adecuaciones al mismo.

a) Análisis.
b) Codificación.
c) Diseño.
d) Mantenimiento.

3. Es un programa que lee línea por línea el código fuente y lo transforma en un código objeto, Perl lo utiliza.

a) Máquina de Turing.
b) Debugger.
c) Interprete.
d) Navegador.

4. Es un programa que transforma un código fuente en un código ejecutable, C++ lo utiliza.

a) Diagrama de flujo.
b) Compilador.
c) Explorador de archivos.
d) Algoritmo.

5. El código <stdio.h> del lenguaje C es un ejemplo de

a) Constante.
b) Librería.
c) Palabra reservada.
d) Variable.

RESPUESTAS EXAMEN DE AUTOEVALUACIÓN

En este apartado encontrarás las respuestas al examen por unidad.

UNIDAD 2	
I. Solución	
1.	V
2.	F
3.	F
4.	V
5.	V
6.	F
7.	F
8.	V
9.	F
10.	F

UNIDAD 2	
II. Solución	
1.	c
2.	d
3.	e
4.	a
5.	b

UNIDAD 2	
III. Solución	
1.	a
2.	d
3.	c
4.	b
5.	b

REFERENCIAS

Muñoz, A. (1 de octubre de 2016). ¿Qué es un bug? *Computer* hoy https://computerhoy.com/noticias/software/que-es-bug-51858

Ruiz, J. (2010). *Ciudad de México*. Compiladores. Teorías e implementa. Alfaomega.

Joyanes, L. (2020). *Algoritmos, estructuras de datos y objetos*. McGraw-Hill Interamericana.

UNIDAD 3

ELEMENTOS BÁSICOS DE UN PROGRAMA

OBJETIVO PARTICULAR

Al finalizar la unidad, el alumnado empleará los elementos sintácticos para el desarrollo de programas.

TEMARIO DETALLADO

(10 horas)

- 3. Elementos básicos de un programa
  - 3.1. Estructura de un programa
  - 3.2. Identificadores y palabras reservadas
  - 3.3. Manejo de memoria
    - 3.3.1. Constantes
    - 3.3.2. Variables
    - 3.3.3. Arreglos
    - 3.3.4. Registros
  - 3.4. Tipos de datos
    - 3.4.1. Numéricos
    - 3.4.1.1. Enteros
    - 3.4.1.2. Reales o flotantes
    - 3.4.2. No numéricos
    - 3.4.2.1. Caracteres
    - 3.4.2.2. Booleanos
  - 3.5. Declaraciones
  - 3.6. Expresiones y operadores
    - 3.6.1. Expresiones
    - 3.6.2. Operadores aritméticos
    - 3.6.3. Operadores relacionales
    - 3.6.4. Operadores lógicos
    - 3.6.5. Operador de asignación
    - 3.6.5.1. Literales numéricas
    - 3.6.5.2. Literales de carácter
    - 3.6.5.3. Literales de cadena
    - 3.6.5.4. Literales booleanas
    - 3.6.6. Operadores Asociativos
    - 3.6.7. Precedencia de operadores

ACTIVIDAD DIAGNÓSTICA



Adjuntar archivo.

Sin consultar ninguna fuente, realiza la actividad:

Describe con tus propias palabras lo que es un programa de computadora.

INTRODUCCIÓN



Los programas de computadora poseen una estructura que contiene elementos que se utilizan como bloques constructivos. Los elementos constitutivos de un programa son los siguientes:

- Las etiquetas de inicio y fin del programa.
- Las variables.
- Constantes.
- Palabras reservadas.
- Las estructuras de control de flujo del programa (inicio, fin, si-entonces, mientras).
- Las funciones y los caracteres especiales.

Todos estos elementos se encuentran en un lenguaje de programación.

Los elementos mencionados permiten la construcción de un programa de computadora.

Es por eso que en esta unidad se detallarán los elementos constitutivos de un programa para poder comprender su utilidad y así desarrollar un programa de cómputo. Se analizarán fragmentos de código utilizando el lenguaje C.

3.1.
Estructura de un programa

Un programa de computadora tiene una estructura que contiene los siguientes elementos:

- Inicio y fin.
- Declaración de variables.
- Constantes.
- Instrucciones para las operaciones.
- Programa de lectura, proceso y generación de datos de salida.

Figura 3.1. Elementos de un programa de cómputo.

El programador especifica qué tipos de datos de entrada son válidos, los datos procederán de un dispositivo de entrada, por ejemplo: un teclado, una pantalla *touch screen*, un lector de código de barras o cualquier otro dispositivo, que permita la entrada de datos.



Figura 3.2. Dispositivos de entrada.

Por otro lado, la salida de datos se presenta en dispositivos de salida, como una pantalla, una impresora, una bocina o cualquier otro dispositivo que permita la salida de datos (Mancilla, 2014, pp. 17-20).

Las instrucciones en un programa pueden ejecutarse de forma **secuencial**, esto es, todas las sentencias se ejecutan sin posibilidad de omitir alguna. Como se expone a continuación:

Estructura Secuencial
Inicio
Instrucción 1
Instrucción 2
Instrucción n
Fin

En cambio, si las instrucciones no son secuenciales y se interrumpe la secuencia mediante instrucciones de bifurcación, entonces la estructura es **selectiva**. Observa el ejemplo:

Estructura Selectiva
Inicio
Si Condición
Instrucción
De lo Contrario
Instrucción
Fin Si
Fin

Ahora bien, un programa es iterativo si se ejecuta la instrucción mientras se cumpla una condición. Veamos un ejemplo de estructura **iterativa**.

Estructura Iterativa
Inicio
Si Condición Ejecuta mientras
Instrucción
Fin Si
Fin

Veamos a detalle la estructura de un programa en lenguaje C, con el siguiente ejemplo (Joyanes, 2020, p. 86):

#include <stdio.h> **<-BIBLIOTECA**
#include <conio.h> **<-BIBLIOTECA**

Las bibliotecas son repositorios de funciones, la biblioteca **stdio.h** contiene funciones para la entrada y salida de datos, un ejemplo de esto es la función **printf.**

void main(void) **<-FUNCION PRINCIPAL**

La función main es la función principal de todo programa en C.

{ LLAVE DE INICIO

Las llaves de inicio y de fin indican cuando inicia y termina un programa

float num1=25.75, num2=10.15;
<-DECLARACION DE VARIABLES

int total=0; **<-DECLARACION DE VARIABLES**

Es esta parte se declaran las variables que se van a usar, su tipo, y, en su caso, se inicializan.

total=(int)num1+(int)num2;
<-DESARROLLO DEL PROGRAMA

printf(“Total: %d”,total);
<-DESARROLLO DEL PROGRAMA

getch(); **<-DESARROLLO DEL PROGRAMA**

En esta parte, como su nombre lo indica, se desarrolla el programa.

} **LLAVE DE FIN**

Estas son las partes más importantes de un programa en C (Kalicharan, 2015, pp. 6-7).

3.2. Identificadores y palabras reservadas

Los **identificadores** son secuencias de caracteres usadas para dar nombre a los objetos de un programa, como variables. Por ejemplo, el nombre de una variable en un algoritmo es un **identificador**.

VAR

ENTERO Numero1, Numero2, Resultado;

En lenguaje C las variables se declaran de la siguiente manera:

int numero1, numero2, resultado;

El código anterior indica que las tres variables son de tipo entero.

Los identificadores tienen la misma función que las palabras reservadas, con la diferencia de que los identificadores son definidos por el programador (Kalicharan, 2015, pp. 27-28).

Las **palabras reservadas** son textos cuyo significado está predefinido y no se pueden usar para otro fin, por ejemplo, para definir el inicio y terminación de un algoritmo se utiliza la palabra **INICIO** y **FIN**.

En el caso del lenguaje C tenemos algunas **palabras reservadas** como:

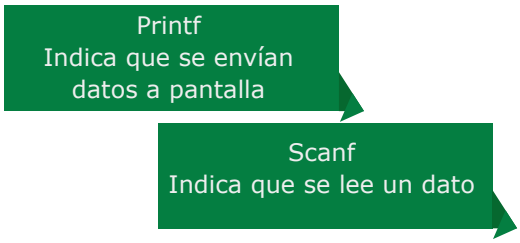


Figura 3.3. Palabras reservadas.

Las palabras reservadas tienen una función específica en un lenguaje de programación y no pueden ser utilizados para otro fin (Mancilla, 2014, p. 282).

3.3. Manejo de memoria

El manejo de la memoria es el proceso de gestionar la memoria principal de un equipo de cómputo. En la memoria principal de una computadora se almacenan los datos necesarios para la ejecución de las aplicaciones, se le conoce comúnmente como *Random Access Memory* o RAM, el almacenamiento es temporal, porque cuando se apaga el equipo de cómputo la información almacenada se borra.

La gestión de las aplicaciones en la memoria principal se realiza de forma ágil, porque la memoria principal está diseñada para que el acceso a los datos sea veloz. Aunque no es una regla, entre más memoria principal posea un equipo de cómputo más rápido ejecutará las aplicaciones.

A pesar de que la mayoría de los equipos de cómputo cuentan con una alta capacidad de memoria principal, las aplicaciones actuales tienen altos requerimientos de memoria, por ejemplo, las aplicaciones que utilizan gráficas consumen una gran cantidad de la capacidad de un equipo.

Cabe desatacar que la memoria principal es de mayor costo que la memoria secundaria de una computadora, la memoria secundaria es conocida comúnmente como el disco duro de una computadora, y el acceso a la información de un disco duro es más lenta que el acceso a la memoria RAM.

Otro término importante es el concepto de memoria virtual, que se explica a continuación.

La memoria virtual es utilizar la memoria secundaria cuando la memoria principal es insuficiente. Cuando en la memoria principal no caben las aplicaciones en ejecución, se utiliza la memoria secundaria para almacenar las aplicaciones que no caben en la memoria principal. Aparentemente es una buena idea; sin embargo, un disco duro es más lento que la memoria RAM, por lo tanto, la ejecución de las aplicaciones que usan memoria virtual se realizará de forma más lenta (Mancilla, 2014, pp. 20-23).

3.3.1. Constantes

Los programas contienen ciertos valores que no pueden cambiar durante la ejecución del programa. Tales valores se llaman **constantes**. Por ejemplo:

```
REAL PI <- 3.141592;
```

El valor de PI, nunca cambia, es una constante.

En el lenguaje C, las constantes se crean de la siguiente manera (Joyanes, 2020, pp. 93-95):

```
#define constante1=7
```

Las constantes son valores que nunca cambian durante la ejecución del programa y son definidas por el programador (Kalicharan, 2015, pp. 80-83).

3.3.2. Variables

Una variable es una unidad básica de almacenamiento, la creación de una variable es la **combinación** de un **identificador** y un **tipo**. En el ejemplo siguiente, la variable ***numero1*** es de tipo ***entero*** (Joyanes, 2020, pp. 93-95).

```
VAR
ENTERO numero1;
numero1=1;
```

En el caso del lenguaje C, la declaración de una variable es la siguiente:

```
int numero1=1;
```

Las variables, como su nombre indica son espacios de almacenamiento cuyo valor cambia, durante la ejecución del programa (Kalicharan, 2015, pp. 18-20).

3.3.3. Arreglos

Un arreglo es un conjunto ordenado de elementos homogéneos. El término ordenado significa que el elemento primero, segundo, tercero y enésimo de un arreglo puede ser identificado. Por su parte, los términos elementos homogéneos quieren decir que los elementos utilizan el mismo tipo de datos. Veamos un ejemplo de la declaración de un arreglo de nombre **x** de tipo entero (Osorio, 2008, p. 365).

ENTERO x[100];

En el caso del lenguaje C, el arreglo se declara de la siguiente manera:

int x[100];

A un arreglo también se le da el nombre de vector. Los vectores se almacenan en la memoria principal en orden adyacente. Así, un vector de cincuenta números denominado NUMEROS se representa gráficamente por cincuenta posiciones de memoria sucesivas.

Vector NUMEROS	Dirección Memoria
NUMEROS[1]	Dirección X
NUMEROS[2]	Dirección X+1
NUMEROS[3]	Dirección X+2
.	.
NUMEROS[50]	Dirección X+49

Tabla 3.1. Ejemplo de representación de valores en un vector.

Cada elemento de un vector se puede procesar como si fuese una variable al ocupar una posición de memoria.

NUMEROS[25]←72;

La instrucción anterior almacena el valor 72 en la posición 25a del vector **NUMEROS**. La instrucción de salida es la siguiente:

ESCRIBIR (NUMEROS[25]);

Visualiza el valor almacenado en la posición 25a, en este caso 72.

Veamos el siguiente algoritmo (Joyanes, 2020, pp. 257-259):

```
ALGORITMO VECTOR
VAR
ENTERO Cont;
ENTERO X[100];
INICIO
        DESDE Cont<-1 HASTA 100
Cont <- Cont+1 HACER
                X[Cont] <- Cont;
        FIN_DESDE
ESCRIBIR (X[4]);
FIN
```

El algoritmo anterior declara un arreglo de 100 elementos de tipo entero y una variable de nombre **Cont**, utilizando un ciclo de tipo **DESDE HASTA**, le asigna un valor consecutivo que se incrementa en 1 a la variable Cont. Terminado el ciclo se muestra en pantalla el valor de la cuarta posición del arreglo **X**.

Veamos este algoritmo en lenguaje C, cabe destacar que en el lenguaje C se empieza a contar desde 0, no desde 1 (Kalicharan, 2015, pp. 199-200).

```
#include <stdio.h>
#include <conio.h>

void main(void)
{
   int x[100];
   int cont;
   for(cont=0;cont<100;cont++)
                x[cont]=cont;
   printf("%d",x[4]);
    getch();
}
```

Además de los arreglos unidimensionales se pueden utilizar arreglos de más de una dimensión, como ejemplo de esto tenemos a los arreglos bidimensionales. En un arreglo bidimensional, los datos son representados en forma de matriz con dos subíndices. Ejemplos típicos de matrices son: tablas de distancias kilométricas entre ciudades, cuadros horarios de trenes o aviones, informes de ventas periódicas (mes/unidades vendidas), etcétera.

El arreglo bidimensional contiene dos subíndices, que hacen referencia al renglón y la columna una matriz. Veamos el siguiente ejemplo:

MATRIZ [renglon][columna]

	1	2	3	4
1	1	2	3	4
2	5	6	7	8
3	9	10	11	12

Si me quiero referir al primer elemento entonces lo hago de la siguiente manera:

```
MATRIZ [1][1];
```

Si le quiero asignar el valor de 1, lo hago de la siguiente manera;

```
MATRIZ [1][1] <- 1;
```

Si quiero asignar el valor de 1 a todos los elementos de la matriz, lo hago de la siguiente manera (Joyanes, 2020, pp. 265-266):

```
ALGORITMO MATRIZ
VAR
ENTERO i,j;
ENTERO R<-3 C<-4;
ENTERO MATRIZ [3][4];
INICIO

DESDE i<-1 HASTA i<=R i+1 HACER

   DESDE j<-1 HASTA j<=C j+1 HACER

        MATRIZ[i] [j] <- 1;
  FIN_DESDE
FIN_DESDE
FIN
```

Si este algoritmo se implementa en C, se hace de la siguiente manera:

```
# include <stdio.h>
#define R 5
#define C 5
main()
{
        int matriz[R][C];
        int i,j;
        for(i=0;i<R;i++)
                for(j=0;j<C;j++)
                        matriz[i][j] = 1;
return(0);
}
```

Un arreglo es una colección de variables del mismo tipo que se referencian por un nombre en común. El tipo más simple de arreglo es el arreglo unidimensional, o vector (matriz de una dimensión), pero se pueden crear arreglos de más de una dimensión (Kalicharan, 2015, pp. 225-226).

3.3.4. Registros

Un registro es un dato estructurado, formado por elementos relacionados que pueden ser del mismo tipo o de distintos tipos, a los que se les denomina campos, veamos un ejemplo (Joyanes, 2020, pp. 272-273):

```
REGISTRO NOMBRE_REGISTRO
        ENTERO Toneladas
        REAL Kilos
        REAL Gramos
FIN_REGISTRO
```

En el caso del lenguaje C, un registro tiene el nombre de estructura y se declara de la siguiente manera:

```
struct nombre{
        int toneladas;
        float kilos;
        float gramos;
};
struct nombre etiqueta;
```

Los miembros de la estructura se referencian utilizando la etiqueta de la estructura, el operador punto(.) y la variable a la que se hace referencia. Los miembros de la estructura deben ser inicializados fuera de ella, si se hace en el interior da error de compilación.

Veamos el ejemplo de un registro en lenguaje C. El siguiente programa almacena información acerca de automóviles, debido a que se maneja información de distintos tipos sería imposible manejarla en un arreglo, por lo que es necesario utilizar una estructura.

El programa le pide al usuario que introduzca los datos necesarios para almacenar información de automóviles, cuando el usuario termina la captura, debe presionar las teclas ctrl.+z.

```
/* programa en C que muestra la forma de crear una estructura */
# include <stdio.h>

struct coche {
  char fabricante[15];
  char modelo[15];
  char matricula[20];
  int antiguedad;
  long int kilometraje;
  float precio_nuevo;
  } miauto;

main()
{
  printf("Introduzca el fabricante del coche.\n");
  gets(miauto.fabricante);
  printf("Introduzca el modelo.\n");
  gets(miauto.modelo);
  printf("Introduzca la matricula.\n");
  gets(miauto.matricula);
  printf("Introduzca la antiguedad.\n");
  scanf("%d",&miauto.antiguedad);
  printf("Introduzca el kilometraje.\n");
  scanf("%ld",&miauto.kilometraje);
  printf("Introduzca el precio.\n");
  scanf("%f",&miauto.precio_nuevo);
```

```
   getchar(); /*vacia la memoria
intermedia del teclado*/

   printf("\n\n\n");
   printf("Un %s %s con %d años de
antiguedad con número de matricula
#%s\n",miauto.fabricante,miauto.
modelo,miauto.antiguedad,miauto.
matricula);
   printf("actualmente con %ld
kilómetros",miauto.kilometraje);
   printf(" y que fue comprado por
$%5.2f\n",miauto.precio_nuevo);
   return(0);
}
```

Un registro permite la gestión de datos de distintos tipos, lo que le da versatilidad a un programa para resolver problemas, donde se requiere gestionar datos de tipos diferentes (Kalicharan, 2015, pp. 280-282).

3.4. Tipos de datos

El objetivo de un sistema de cómputo es la gestión de datos. Si los datos se gestionan de forma adecuada el sistema de información transforma los datos en información. Ejemplo de información son las cifras de ventas de un supermercado, o las calificaciones de un grupo. Una computadora puede utilizar varios tipos de datos. Los tipos de datos pueden ser números, letras, cadenas de caracteres o valores booleanos. Tomando en cuenta lo mencionado, un tipo de dato define un conjunto de valores y las operaciones posibles sobre dichos valores (Joyanes, 2020, pp. 90-92).

3.4.1. Numéricos

Los tipos de datos numéricos comprenden dos categorías, los enteros y los números con punto flotante. Los enteros no poseen partes fraccionales y los números con punto flotante poseen una parte fraccional.

3.4.1.1. Enteros

Los **tipos de datos enteros** son aquellos que representan solo números sin partes fraccionales. Estos pueden poseer un signo o no tener un signo.

Los números enteros se pueden representar en 8, 16 o 32 bits, o su tamaño en bytes, que serían 1 byte, 2 bytes o 4 bytes. Eso da origen a una escala de enteros cuyos rangos dependen de cada computadora. Veamos el siguiente ejemplo de tipo de dato entero en el lenguaje C.

TIPO	RANGO DE VALORES	TAMAÑO EN BYTES	DESCRIP-CIÓN
int	-32.768 a 32.767	**2**	Para números enteros.
unsigned int	0 a 65.535	**2**	Para números enteros.
long int	±2.147.483.647	**4**	Para números enteros.
unsigned long int	0 a 4.294.967.295	**4**	Para números enteros.

Tabla 3.2. Ejemplo de tipos de datos enteros en lenguaje C.

Los enteros se denominan en ocasiones números de punto o coma fija. Y son muy útiles para realizar operaciones con elementos que son indivisibles (Kalicharan, 2015, p. 29).

3.4.1.2. Reales o flotantes

Los números reales son el conjunto de números naturales, enteros, racionales e irracionales que siempre tienen un punto decimal y pueden ser positivos o negativos. Un número real consta de un entero y una parte decimal.

En el lenguaje C el rango de valores de los números reales es el siguiente:

TIPO	RANGO DE VALORES	TAMAÑO EN BYTES	DESCRIP-CIÓN
float	3.4E-38 decima-les(6)	**6**	Para números con decimales
double	1.7E-308 decima-les(10)	**8**	Para números con decimales
long double	3.4E-4932 decima-les(10)	**10**	Para números con decimales

Tabla 3.3. Rango de valores de tipos de datos flotantes en lenguaje C.

En aplicaciones científicas se requiere el manejo de números muy grandes, como la masa de la Tierra, o muy pequeños, como la masa de un electrón (Kalicharan, 2015, pp. 33-34).

3.4.2. No numéricos

Los tipos de datos no numéricos son tipos de datos que comprenden caracteres y tipos de datos booleanos.

3.4.2.1. Caracteres

El tipo **carácter**, como su nombre lo indica, contiene un solo carácter. Los caracteres que reconocen las diferentes computadoras no son estándar; sin embargo, la mayoría reconoce los siguientes caracteres: alfabéticos, numéricos y caracteres especiales:

- Caracteres alfabéticos (A, B, C,...,Z) (a, b, c,...,z).
- Caracteres numéricos (1, 2,...,9, 0).
- Caracteres especiales (+, –, *, /, ^,...,;,<,>,$,...).

Una **cadena de caracteres** es una sucesión de caracteres que se encuentran delimitados por una comilla o comillas dobles. La longitud de una cadena de caracteres es el número de ellos comprendido entre los separadores, o delimitadores (Joyanes, 2020, pp. 92-94).

Veamos una descripción de los tipos de dato carácter en lenguaje C en la siguiente tabla:

TIPO	RANGO DE VALORES	TAMAÑO EN BYTES	DESCRIPCIÓN
char	-128 a 127	**1**	Para una letra o un dígito.
unsigned char	0 a 255	**1**	Letra o número positivo.

Tabla 3.4. Tipos de datos de tipo carácter en lenguaje C.

Los tipos de dato carácter son útiles cuando solo se desea utilizar una letra o un número, por ejemplo, un abecedario, una numeración o todo programa que requiera de un carácter (Kalicharan, 2015, pp. 38-40).

3.4.2.2. Booleanos

El tipo booleano también denominado lógico es aquel dato que solo puede tomar uno de dos valores: **cierto** o **falso**.

Este tipo de datos se utiliza para representar las alternativas (sí/no) a determinadas condiciones. Por ejemplo, cuando se pide si un valor entero es par, la respuesta será verdadera o falsa, según sea par o impar (Joyanes, 2020, p. 92).

El lenguaje C no cuenta con tipos de datos booleanos, en este caso se tienen que definir, esto se puede realizar con estas instrucciones.

```
typedef int bool;
enum { false, true };
```

Las formas de organizar los datos están determinadas por los **tipos de datos**.
Un tipo de dato determina el **rango de valores** que puede tomar un objeto, las operaciones a que puede ser sometido y el **formato** de almacenamiento en memoria (Kalicharan, 2015, p. 283).

3.5. Declaraciones

Las declaraciones son una parte de un algoritmo o programa donde se definen las constantes y variables que serán utilizadas.

En la sección de declaraciones de un algoritmo se declaran las constantes, los tipos de datos y las variables que se usan en el algoritmo. La sintaxis es la siguiente (Joyanes, 2020, p. 94):

```
Declaraciones de constantes
VAR
Declaraciones de variables
```

En lenguaje C, las declaraciones se realizan de la siguiente forma:

```
#define peso 1;
int i, j, k;
float largo, ancho, alto;
```

Se declara una constante y variables de tipo entero y real o flotante. En lenguajes de programación como C es indispensable declarar las constantes y variables antes de utilizarlas. No todos los lenguajes de programación lo requieren, por ejemplo, en el lenguaje de programación PHP no es necesario hacerlo, aunque por orden, es mejor realizar las declaraciones de variables y constantes antes de utilizarlas (Kalicharan, 2015, pp. 24-25).

3.6. Expresiones y operadores

Una expresión se forma combinando constantes y variables. Una expresión consta de **operadores** y **operandos**. Según sea el tipo de datos que utilizan.
Las expresiones realizan acciones específicas, según un operador, por ejemplo, la suma de dos operandos con el operador **SUMA (+)**.

A + B

Un operando puede ser una constante o una variable. Los operadores son aritméticos, relacionales, lógicos y de asignación. Una expresión es una secuencia de operadores y operandos que representa un valor obtenido a través de un cálculo.

3.6.1. Expresiones

Una **expresión** representa un valor, el resultado de realizar las operaciones indicadas siguiendo las reglas de evaluación establecidas en el lenguaje.
Cada expresión toma un valor que se determina, tomando los valores de las variables y constantes implicadas y la ejecución de las operaciones indicadas (Kalicharan, 2015, pp. 18-20).

Un ejemplo de una expresión es la siguiente:

Var1 <- Var1 + Var2;

Var1 es la suma de Var1 más Var2.

En lenguaje C la expresión se escribiría de la siguiente forma (Joyanes, 2020, pp. 94-96):

var1 = var1 + va2;

3.6.2. Operadores aritméticos

Los operadores aritméticos, como su nombre lo indica, permiten la realización de operaciones aritméticas, como sumas, restas multiplicaciones y divisiones.
Los operadores aritméticos pueden aplicarse a todo tipo de expresiones. Son utilizados para realizar operaciones matemáticas sencillas, aunque uniéndolos se puede realizar cualquier tipo de operaciones. En la siguiente tabla se muestran todos los operadores aritméticos.

OPERADOR	DESCRIPCIÓN
-	**Resta**
+	**Suma**
*	**Multiplica**
/	**Divide**
%	**Módulo (resto de una división)**

Tabla 3.5. Tipos de operadores.

- Corresponden a las operaciones matemáticas de suma, resta, multiplicación, división y módulo.
- Son binarios porque cada uno tiene dos operandos.

La división de enteros devuelve el cociente entero y desecha la fracción restante (Joyanes, 2020, pp. 94-96):

1/2 tiene el valor 0
3/2 tiene el valor 1
-7/3 tiene el valor -2

El operador módulo se aplica así: con dos enteros positivos, devuelve el resto de la división (Kalicharan, 2015, pp. 29-30).

12%3 tiene el valor 0
12%5 tiene el valor 2

3.6.3. Operadores relacionales

Los operadores relacionales hacen referencia a la relación entre unos valores y otros. Lo podemos ver en la siguiente tabla:

OPERADORES RELACIONALES	
OPERADOR	DESCRIPCIÓN
<	**Menor que**
>	**Mayor que**
<=	**Menor o igual**
>=	**Mayor o igual**
= =, =	**Igual**
! =, <>	**Distinto**

Tabla 3.6. Operadores relacionales.

Veamos un ejemplo de su utilización.

EJEMPLO	DESCRIPCIÓN
A < B	**A Menor que B**
A > B	**A Mayor que B**
A <= B	**A Menor o igual que B**
A >= B	**A Mayor o igual que B**
A == B	**A Igual que B**
A != B	**A Distinto de B**

Tabla 3.7. Ejemplo de utilización de operadores relacionales.

Los operadores relacionales se usan para comparar dos valores. Si el resultado de la comparación es correcto la expresión es verdadera, en caso contrario es falsa (Kalicharan, 2015, pp. 65-66).

3.6.4. Operadores lógicos

Los **operadores lógicos** se usan para combinar dos valores Booleanos y devolver un resultado verdadero o falso. **Los operadores lógicos** también se denominan **operadores** Booleanos.

OPERADORES LÓGICOS	
OPERADOR	DESCRIPCIÓN
AND	**Y**
OR	**O**
NOT	**NO**

Tabla 3.8. Operadores lógicos.

Ejemplo de su utilización.

EJEMPLO	RESULTADO
A AND B	Verdadero si A y B son verdaderos
A OR B	Verdadero si A o B son verdaderos
NOT A	Si A es verdadero entonces el resultado es falso

Tabla 3.9. Ejemplo de utilización de operadores lógicos.

En este tipo de operadores los resultados son Verdadero o Falso (Joyanes, 2020, pp. 100-102).

3.6.5. Operador de asignación

Un **operador de asignación** asigna un valor a su operando izquierdo basándose en el valor de su operando derecho. El **operador de asignación**, en el caso de seudocódigo, es el siguiente: **(<-)**, este operador asigna un valor al operando izquierdo.

En el caso del lenguaje C, la asignación es el signo de igual **(=)** que asigna el valor de su operando derecho a su operando izquierdo. Es decir, **x = y** asigna el valor de **y a x** (Joyanes, 2020, pp. 105-106).

3.6.5.1. Literales numéricas

Las literales permiten almacenar en el programa un valor que permanecerá constante a lo largo de la ejecución del programa.

Una literal es un valor constante formado por una secuencia de caracteres.

Son ejemplos de literales los números, los caracteres, las cadenas de caracteres y los valores booleanos.

Ejemplo de una literal numérica.

PI <- 3.1416;

En lenguaje C se realiza de la siguiente manera:

PI = 3.1416;



3.6.5.2. Literales de carácter

Las literales de caracteres se expresan por un solo carácter entre comillas sencillas.

literal_caracter <- ‘a’;

En lenguaje C se realiza de la siguiente manera:

literal_caracter = ‘a’;

3.6.5.3. Literales de cadena

Una cadena es una combinación de caracteres. Las literales de cadena se representan por una secuencia de caracteres entre comillas dobles.

literal_caracter <- “hola”;

En lenguaje C se realiza de la siguiente manera:

literal_caracter = “hola”;

3.6.5.4.
Literales booleanas

Las literales booleanas consisten en las palabras reservadas **true** y **false**, o falso y verdadero. Veamos un algoritmo que, dependiendo del valor de dos variables, asigna el valor VERDADERO o FALSO.

```
SI A > B ENTONCES
        X <- VERDADERO;
SI_NO
        X <-FALSO;
FIN_SI
```

En esencia, una expresión lógica es una expresión que solo puede tomar dos valores: verdadero y falso.

Las literales numéricas, de carácter, cadena y lógicas caben en la categoría de constantes, ya que conservan su valor durante la ejecución del programa (Kalicharan, 2015, pp. 41-42).

3.6.6.
Operadores Asociativos

El único **operador asociativo** es el paréntesis ( ), el cual permite indicar en qué orden deben realizarse las operaciones. Cuando una expresión se encuentra entre paréntesis, indica que las operaciones que están dentro de ellos deben realizarse primero. Veamos un ejemplo:

En este caso por prioridad de operadores, la multiplicación se ejecuta primero que la suma y la resta.

4 + 2 * 5 = 14

En cambio, la siguiente expresión:

(4 + 2) * 5 = 30

En este caso por prioridad de operadores, la suma se ejecuta primero y después la multiplicación.

Existe una precedencia de operadores, primero se realizan las operaciones dentro del paréntesis (Kalicharan, 2015, pp. 30-31).

3.6.7.
Precedencia de operadores

La ejecución de los operadores se realiza de acuerdo con una prioridad, la cual es la siguiente. Todas las expresiones entre paréntesis se evalúan primero. Las expresiones con paréntesis anidados se evalúan de dentro hacia afuera, el paréntesis más interno se evalúa primero.

Dentro de una misma expresión los operadores se evalúan en el siguiente orden.

1. () Paréntesis
2. ^ Exponenciación
3. *, /, mod Multiplicación, división, módulo. (El módulo o mod es el resto de una división)
4. +, - Suma y resta.

Los operadores en una misma expresión con igual nivel de prioridad se evalúan de izquierda a derecha.

Ejemplos:

a. 4 + 2 * 5 = 14	Primero se multiplica y después se suma
b. 23 * 2 / 5 = 9.2	46 / 5 = 9.2
c. 3 + 5 * (10 - (2 + 4)) = 23	3 + 5 * (10 - 6) = 3 + 5 * 4 = 3 + 20 = 23
d. 2.1 * (1.5 + 3.0 * 4.1) = 28.98	2.1 * (1.5 + 12.3) = 2.1 * 13.8 = 28.98

Los operadores poseen una precedencia, esto es una prioridad de ejecución. Primero se ejecutan las operaciones dentro de los paréntesis, después las divisiones y multiplicaciones y por ultimo las sumas y las restas (Kalicharan, 2015, pp. 30-31).

RESUMEN DE LA UNIDAD

Un programa de computadora posee una estructura diseñada para la utilización de variables, si se utilizan valores cambiantes y constantes, si se utilizan valores que no cambian durante la ejecución del programa, además de estructuras de control que permiten la ejecución de instrucciones de forma secuencial, o con bifurcaciones, dependiendo del cumplimiento de una condición, o estructuras cíclicas si se desea que una instrucción se repita un número determinado o indeterminado de veces. Por último, los programas están diseñados para leer y transformar los datos introducidos por un usuario, en información desplegada en una pantalla.

Los elementos de un programa son inicio y fin, declaración de variables, constantes, instrucciones para las operaciones y programa de lectura, proceso y generación de datos de salida.

Las estructuras de los programas son: secuenciales, selectivas, e iterativas.

En el diseño de un programa es importante considerar los identificadores y las palabras reservadas.

Para que los programas puedan ejecutarse de manera rápida es necesario que se gestione y maneje la memoria RAM o memoria principal, entre más memoria se tenga más rápido se ejecutarán las aplicaciones y la memoria virtual o secundaria.

Los programas contienen valores que se llaman constantes, estos valores no son cambiantes.

Las variables, como se vio en el desarrollo del contenido, son unidades básicas de almacenamiento y son cambiantes, como su nombre lo indica.

Los arreglos (array en inglés) es un conjunto ordenado de elementos de datos, comúnmente se les conoce también como vectores. Es muy común su uso en la programación orientada a objetos.

En todos los programas se usan los registros, mismos que son datos estructurados y elementos relacionados.

La finalidad de un programa es la gestión de datos y generación de información, por lo que es necesario el conocimiento de los tipos de datos para lograr una buena gestión de estos, en cuanto a sus tipos pueden ser numéricos, enteros, reales o flotantes, no numéricos, de caracteres y booleanos.

Las expresiones son partes importantes de un programa, representan valores.

Los operadores pueden ser aritméticos, relacionales lógicos, de asignación y asociativos, todos ellos con características importantes para la ejecución de un programa.

Las literales son de gran valor para un programa, ya que son valores constantes, pueden ser de carácter, de cadena y booleanas.

ACTIVIDADES DE APRENDIZAJE

Unidad 3, actividad inicial. ***Adjuntar archivo.*** A partir del estudio del contenido de la unidad, elabora un mapa conceptual u organizador gráfico sobre los temas vistos. Dentro del documento podrás encontrar recomendaciones de *software* para su elaboración.

1. **Unidad 3, actividad 1.** ***Adjuntar archivo.*** Después de haber dado lectura a la unidad, realiza un algoritmo en seudocódigo que solicite dos números, los sume y muestre el resultado. Describe el proceso.

2. **Unidad 3, actividad 2.** ***Adjuntar archivo.*** Después de haber dado lectura a la unidad, investiga los tipos de datos que se utilizan en un lenguaje de programación, puedes seleccionar cualquier lenguaje de programación, excepto el lenguaje C.

3. **Unidad 3, actividad 3.** ***Adjuntar archivo.*** Después de haber dado lectura a la unidad, realiza un algoritmo en seudocódigo que determine si un número introducido por el usuario es par o impar. Describe el proceso.

4. **Unidad 4, actividad 4.** ***Adjuntar archivo.*** Después de haber dado lectura a la unidad, realiza un algoritmo en seudocódigo que contenga 5 números de tipo entero, almacenados en 5 variables. Ordena los 5 valores de forma ascendente o descendente y muestra el resultado del ordenamiento (puedes utilizar el algoritmo de ordenamiento de la burbuja). Describe el proceso.

5. **Unidad 5, actividad 5.** ***Adjuntar archivo.*** Realiza un algoritmo en seudocódigo que pregunte por el sexo de un usuario, el algoritmo solo permite la entrada de las letras ‘M’ y ‘H’, si el usuario introduce una letra diferente el algoritmo enviará un mensaje de error, si introduce una letra permitida, el algoritmo indicará que la captura es correcta. Describe el proceso..

6. **Unidad 2, actividad complementaria 1.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

7. **Unidad 2, actividad complementaria 2.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

8. **Unidad 2, actividad complementaria 3.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

ACTIVIDAD INTEGRADORA

Adjuntar archivo.

En esta unidad aprendiste sobre el empleo de los elementos sintácticos para el desarrollo de programas. A partir de los conocimientos adquiridos, desarrolla la actividad integradora:

Realiza un algoritmo en seudocódigo que sume 10 números de tipo entero, contenidos en un arreglo de 10 elementos.

Describe el proceso en un documento en Word y súbelo a la plataforma para revisión y evaluación por parte del asesor.

CUESTIONARIO DE REFORZAMIENTO

Adjuntar archivo.

Contesta las siguientes preguntas.

1. Describe qué es un programa de cómputo.
2. Describe un programa con estructura secuencial.
3. Describe un programa con estructura selectiva.
4. Describe un programa con estructura iterativa.
5. Define qué es una constante.
6. Define qué es una variable.
7. Describe qué es una palabra reservada.
8. Describe qué es un identificador.
9. ¿Qué es una declaración?
10. ¿Qué es una expresión?

AUTOEVALUACIÓN

I. Contesta las siguientes preguntas, elige una V si la frase es verdadera o una F si es falsa.

1. La estructura secuencial ejecuta todas las sentencias sin posibilidad de omitir alguna. ○

2. La estructura selectiva utiliza ciclos con un número determinado de iteraciones. ○

3.La estructura iterativa ejecuta una sentencia mientras se cumpla una condición. ○

4. Una palabra reservada es un texto predefinido y que no se puede usar para otro fin. ○

5. Un ejemplo de una palabra reservada en lenguaje C es INICIO. ○

6. El nombre de una variable también es un identificador. ○

7. La memoria principal es lo mismo que un disco duro de computadora. ○

8. Las constantes son valores que nunca cambian durante la ejecución del programa. ○

9. Las variables son valores que nunca cambian durante la ejecución del programa. ○

10. Un arreglo es un conjunto ordenado de elementos heterogéneos, o sea de distintos tipos de datos.. ○

II. Contesta las siguientes preguntas, elige la letra del inciso que conteste de manera correcta.

a) Matriz	b) Vector	c) Enteros	d) Caracteres	e) Registro

1. Un conjunto de elementos heterogéneos o de distinto tipo, se le conoce como: ◯
2. Un ____ es un arreglo unidimensional. ◯
3. Un arreglo bidimensional es también conocido como: ◯
4. Los tipos de datos numéricos comprenden dos categorías, los ____ y los de punto flotante. ◯
5. Los tipos de datos no numéricos son tipos de datos que comprenden ___ y tipos de datos booleanos. ◯

III. Selecciona la respuesta correcta.

1. Las _____ son una parte de un algoritmo donde se definen las constantes y variables.

a) Variables.
b) Expresiones.
c) Declaraciones.
d) Constantes.

2. La combinación de constantes y variables forma una:

a) Declaración.
b) Expresión.
c) Instancia.
d) Biblioteca.

3. Un ejemplo de un operador relacional es el siguiente:

a) +
b) *
c) %
d) >

4. Una literal es un valor:

a) Constante
b) Numérico
c) Sin signo
d) Positivo

5. De acuerdo con la precedencia de operadores, el _____ tiene mayor precedencia.

a) +
b) /
c) ()
d) -

RESPUESTAS EXAMEN DE AUTOEVALUACIÓN

En este apartado encontrarás las respuestas al examen por unidad.

UNIDAD 2	
I. Solución	
1.	V
2.	F
3.	V
4.	V
5.	F
6.	V
7.	F
8.	V
9.	F
10.	F

UNIDAD 2	
II. Solución	
1.	e
2.	b
3.	a
4.	c
5.	d

UNIDAD 2	
III. Solución	
1.	c
2.	b
3.	d
4.	a
5.	c

REFERENCIAS

Joyanes, A. (2020). *Algoritmos, estructuras de datos y objetos*. McGraw-Hill Interamericana.

Kalicharan, N. (2015). *Learn to Program with C*. Apress.

Mancilla, H. (2014). *Diseño y construcción de Algoritmos*. Universidad del Norte.

Osorio, R. (2008). *Lógica y programación orientada a los objetos: un inicio al desarrollo de software*. ITM.

UNIDAD 4

ANÁLISIS DE PROBLEMAS

OBJETIVO PARTICULAR

Al finalizar la unidad, el alumnado establecerá el alcance de un problema en términos algorítmicos.

TEMARIO DETALLADO

 **(6 horas)**

4. Análisis de problemas
4.1. Definición de objetivo
4.2. Definición de datos de salida y postcondiciones
4.3. Definición de datos de entrada y precondiciones

ACTIVIDAD DIAGNÓSTICA



Adjuntar archivo.

Con base en tus conocimientos adquiridos en las unidas previas redacta con tus propias palabras una definición de algoritmo y menciona sus elementos más importantes.

INTRODUCCIÓN

En la presente unidad de aprendizaje se abordan los conceptos básicos para el análisis de problemas, las fases en la resolución de problemas desde la perspectiva algorítmica y su relación con la programación (programas) y su metodología.

Las fases previas a la codificación de programas son un pilar para obtener soluciones eficaces y eficientes. Como primer punto a desarrollar se debe definir el objetivo del problema que se desea resolver, este punto debe de ser revisado por todas las partes interesadas en el proyecto, como lo pueden ser los clientes, áreas de negocio, usuarios finales, operadores de sistemas, áreas de tecnologías de información, analistas de negocio y sistemas, consultores, entre otros. La participación de todos proporcionará los elementos necesarios para identificar los requerimientos funcionales y no funcionales a considerar en el desarrollo de la solución, así como los aspectos tecnológicos del software o sistemas que se desarrollarán.



Por otra parte, una vez definido nuestro objetivo, con la participación de todos los interesados en el proyecto, el equipo encargado en la construcción del software iniciará las fases de análisis del problema, diseño del algoritmo, codificación de un programa, compilación, ejecución, verificación, depuración, mantenimiento y documentación. En cada fase se deberán tener puntos de control y revisión para garantizar la calidad del software y su nivel de cumplimiento con el objetivo definido.

Adicionalmente se abordarán los elementos de entrada y salida que deben ser considerados en el desarrollo de programas, así como el desarrollo de precondiciones y postcondiciones dentro de los mismos.

4.1.
Definición de objetivo

Como primer paso en el análisis de problemas, desde la perspectiva algorítmica, se debe iniciar definiendo nuestro objetivo, el cual será la meta por alcanzar, esto se realizará a través de un proceso de resolución de problemas, el cual hace uso de computadoras y a su vez conduce a la escritura o codificación de programas y a su ejecución. Dentro del proceso de análisis, diseño y codificación de programas las habilidades más importantes son la creatividad, procesos mentales imaginativos para afrontar un problema y proponer alternativas para alcanzar el objetivo o solución de este. Sin embargo, estos procesos creativos deben considerar una serie pasos genéricos que son esenciales y que todos los desarrolladores de software y programadores deben de seguir.

Las fases de resolución de un problema y su codificación a un programa son:

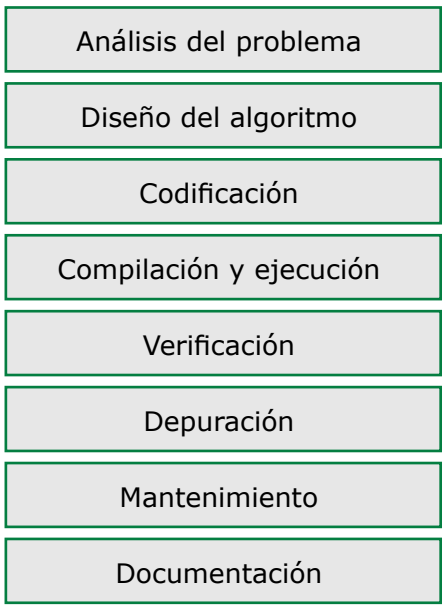


Figura 4.1. Fases de resolución de un problema.

A continuación, se describirán los aspectos más relevantes dentro de cada fase del proceso de resolución de problemas:

Análisis	Diseño	Codificación
• Primera fase en la cual se analizan los requisitos y requerimientos proporcionados por el cliente o empresa, los cuales especifican las condiciones y reglas que se deben de cumplir para alcanzar el objetivo o solución del problema.	• Una vez analizado el problema, se empieza la construcción (diseño) de las posibles soluciones las cuales se desarrollan en algoritmos, pseudocódigos o diagramas de flujo.	• Una vez definida la solución se inicia la etapa de escritura de código en un lenguaje de programación de alto nivel, por ejemplo; Java, Python, C, etc. Una vez finalizada la codificación se obtendrá como resultado un código fuente que deberá ser compilado o interpretado.
Ejecución	**Mantenimiento**	**Documentación**
• Con el código fuente resultante de la fase anterior, este deberá ser compilado o interpretado para su ejecución, una vez ejecutado se deberán realizar las pruebas de verificación, validación y depuración necesarias para garantizar el cumplimiento de los requisitos y requerimientos expuesto es la fase de análisis así para la identificación de errores y mejora de este.	• Una vez verificado y validado el programa se inicia la fase de mantenimiento en la cual se podría actualizar o modificar cada vez que sea necesario considerando los posibles cambios en las reglas de negocio, normatividad o legislación.	• Esta fase se encuentra presente y se trabaja de forma paralela durante todas las fases. Se cumple con la escritura de todas las fases especificando los detalles más relevantes del ciclo de vida de software.

Figura 4.2. Fases para la resolución de problemas. Fuente: (Custodio, 2022, p. 22)

Como se puede observar en las primeras fases (análisis y diseño), anteriormente expuestas, el uso y manejo de los conceptos de algoritmos se hace presente.

> La palabra algoritmo se deriva de la traducción al latín de la palabra Alkhô-warîzmi, nombre de un matemático y astrónomo árabe que escribió un tratado sobre manipulación de números y ecuaciones en el siglo IX. Un algoritmo es un método para resolver un problema mediante una serie de pasos precisos, definidos y finitos. (Joyanes, 2019, p. 46)

La primera fase de la resolución de un problema es el análisis del problema. En esta fase se requiere tener un claro entendimiento de la definición del problema, el objetivo al que se desea alcanzar, así como los requerimientos y requisitos (reglas de negocio) expresados por el cliente y/o la empresa, en esta fase se busca determinar el ¿qué hace el programa? Como resultado de esto se obtendrán las especificaciones necesarias donde se contempla lo que el programa debe hacer y las posibles soluciones.

Adicionalmente a los requisitos y requerimientos, se deben precisar las especificaciones relacionadas con los datos de entrada y de salida ya que son aspectos clave para una adecuada codificación. En la siguiente imagen se muestran las primeras fases para la resolución de problemas:

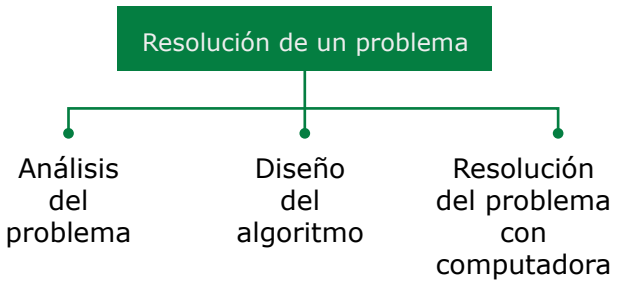


Figura 4.3. Primeras fases para la resolución de problemas. Fuente: (Joyanes, 2008, p. 47).

Algunas de las herramientas que se pueden utilizar para identificar, definir y precisar los elementos de un problema son las siguientes preguntas:

- ¿Cuál es el objetivo o resultado qué se quiere alcanzar?
- ¿Qué datos de entrada se requieren?
- ¿Qué datos de salida se esperan?
- ¿Cuáles son las reglas de negocio?
- ¿Se tienen restricciones para la solución?

Una vez definido el ¿qué hace el programa?, en la fase de análisis, el siguiente paso es iniciar el proceso de diseño, en el cual el objetivo es definir el ¿cómo se realizará el programa? o cómo se realizarán las tareas necesarias para resolver el qué.

Dentro de la metodología que estamos utilizando para la resolución de problemas, en la fase de diseño se cuenta con varios métodos, como los es el método Divide y vencerás (Martí, 2004, p. 311). La resolución de problemas a través de este método se basa en la filosofía de dividir un problema complejo en problemas más pequeños o subproblemas y a su vez dividir estos subproblemas en subproblemas más pequeños hasta llegar a un punto en que la complejidad del problema original sea más fácil de resolver. Este método también es conocido como diseño descendente (*top-down*) o modular.

El proceso de dividir el problema en cada etapa y expresar cada subprograma en forma más detallada y con menor complejidad se denomina refinamiento sucesivo. Cada subprograma es resuelto mediante un módulo (subprograma) que tiene un solo punto de entrada y un solo punto de salida.

Cualquier programa bien diseñado consta de un programa principal (el módulo de nivel más alto) que llama a subprogramas (módulos de nivel más bajo) que a su vez pueden llamar a otros subprogramas.

Los programas estructurados de esta forma se dice que tienen un diseño modular y el método de romper el programa en módulos más pequeños se llama programación modular. Los módulos se planean, codifican, comprueban y se depuran de manera independiente (incluso por diferentes programadores) y después se combinan entre sí.

El proceso implica la ejecución de los siguientes pasos hasta que el programa se concluye:



Figura 4.4. Proceso de programación modular.

Una vez finalizada la fase de análisis del problema, se realiza la fase de diseño, la cual es independiente de cualquier lenguaje de programación.

Durante la fase de diseño de algoritmos se pueden utilizar diagramas de flujo, los cuales son una representación gráfica de un algoritmo. Los símbolos utilizados han sido normalizados por el Instituto Norteamericano de Normalización (ANSI, por sus siglas en inglés, *American Nacional Standards Institute*) y los más frecuentemente empleados se muestran en la siguiente imagen:

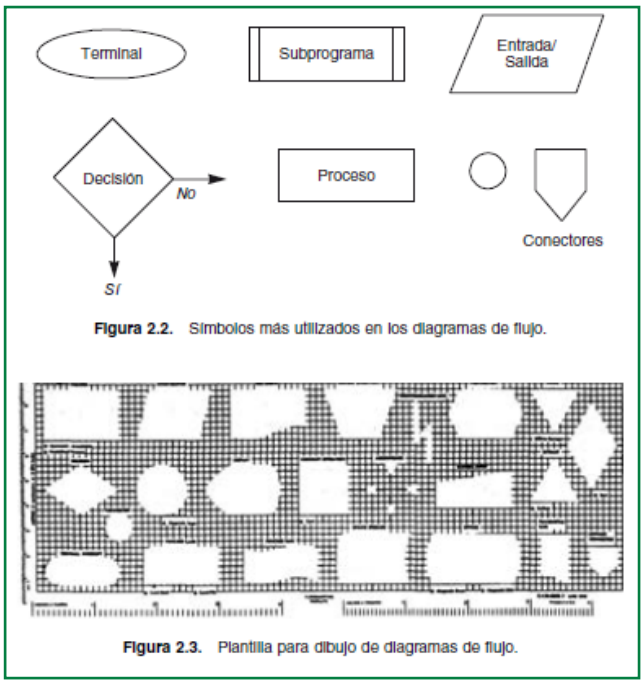


Figura 4.5. Simbología ANSI. Fuente: (Joyanes, 2008, p. 49).

Otra herramienta para el diseño es el pseudocódigo, en el cual se realiza una descripción textual de las instrucciones a realizar o instrucciones que resuelven el problema, las palabras utilizadas en esta herramienta son similares al inglés o español las cuales facilitaran la compresión a los programadores para desarrollar las instrucciones en algún lenguaje de programación. Básicamente un pseudocódigo es una serie de especificaciones de algoritmos.

Las palabras reservadas básicas se representan en letras negritas minúsculas. Estas palabras son traducción libre de palabras reservadas de lenguajes como C, Pascal, etc. A continuación, se muestra un ejemplo de la escritura de un pseudocódigo:

Suma de dos números
Introducir primer número
Introducir segundo número
Realizar operación suma
primer número más segundo
número
imprimir resultado

La fase de codificación es la etapa en el cual los programadores realizan la traducción de diagramas de flujos, algoritmos y pseudocódigos a un lenguaje de programación. Una de las ventajas de utilizar herramientas de diseño como diagramas de flujo y pseudocódigos es que, sin importar la representación de estos, podrán ser escritos en cualquier lenguaje de programación derivado de su independencia de estos últimos.

A continuación, se muestra un ejemplo de la codificación de una calculadora en un lenguaje de programación (Python):

```
# Programa que realiza operaciones básicas
# Esta función suma dos números
def add(x, y):
    return x + y
# Esta función resta dos números
def subtract(x, y):
    return x - y
# Esta función multiplica dos números
def multiply(x, y):
    return x * y

# Esta función divide dos números
def divide(x, y):
    return x / y

print("Selecciona un operacion.")
print("1.suma")
print("2.resta")
print("3.multiplicacion")
print("4.división")

while True:
    # Selección de la opción del usuario
    choice = input("Selecciona una opción(1/2/3/4): ")

    # Validar la opción del usuario
    if choice in ('1', '2', '3', '4'):
        num1 = float(input("Ingresar el primer número: "))
        num2 = float(input("Ingresa el segundo número: "))
        if choice == '1':
            print(num1, "+", num2, "=", add(num1, num2))
        elif choice == '2':
            print(num1, "-", num2, "=", subtract(num1, num2))
        elif choice == '3':
            print(num1, "*", num2, "=", multiply(num1, num2))
        elif choice == '4':
            print(num1, "/", num2, "=", divide(num1, num2))

        # Valida si el usuario desea realizar otra operación
        next_calculation = input("¿Quieres hacer otra
operación? (sí/no): ")
        if next_calculation == "no":
          break
    else:
        print("Valor invalido")
```

En el ejemplo anterior observamos cómo un algoritmo se puede convertir en un programa fuente en un lenguaje de programación o código fuente, el cual deberá ser almacenado en disco duro y después deberá llevar un proceso de compilación y ejecución. El código fuente, en el proceso de compilación, será transformado en lenguaje máquina, este proceso se lleva a cabo con ayuda de un software llamado compilado. Durante el

proceso de compilación se presentan errores en el código fuente, el software notificará de los errores encontrados y será necesario realizar las modificaciones necesarias, una vez realizadas las correcciones se realizará un nuevo proceso de compilación.

Una vez que el compilador realice su proceso sin encontrar errores, el resultado será la obtención de un programa objeto, el cual es una fase previa para obtener un programa ejecutable. Una vez que se obtiene el programa fuente, el sistema operativo será el encargado de realizar la fase de montaje o enlace, en el cual se carga el programa objeto con todas las bibliotecas del programa compilador y del sistema operativo, durante este proceso se obtendrá un programa ejecutable. Una vez obtenido el programa ejecutable este se puede ejecutar desde una Shell, línea de comando o una interfaz gráfica. En la ejecución del programa, el usuario podrá interactuar o visualizar las funciones y cumplimiento de objetivos por los cuales fue creado el software. Durante esta etapa se pueden validar las funciones del software así como su comportamiento.

Las instrucciones u órdenes para compilar y ejecutar un programa en C, C++,... o cualquier otro lenguaje dependerá de su entorno de programación y del sistema operativo en que se ejecute Windows, Linux, Unix, etc.

En la siguiente imagen se describe el proceso completo de compilación/ejecución de un programa:

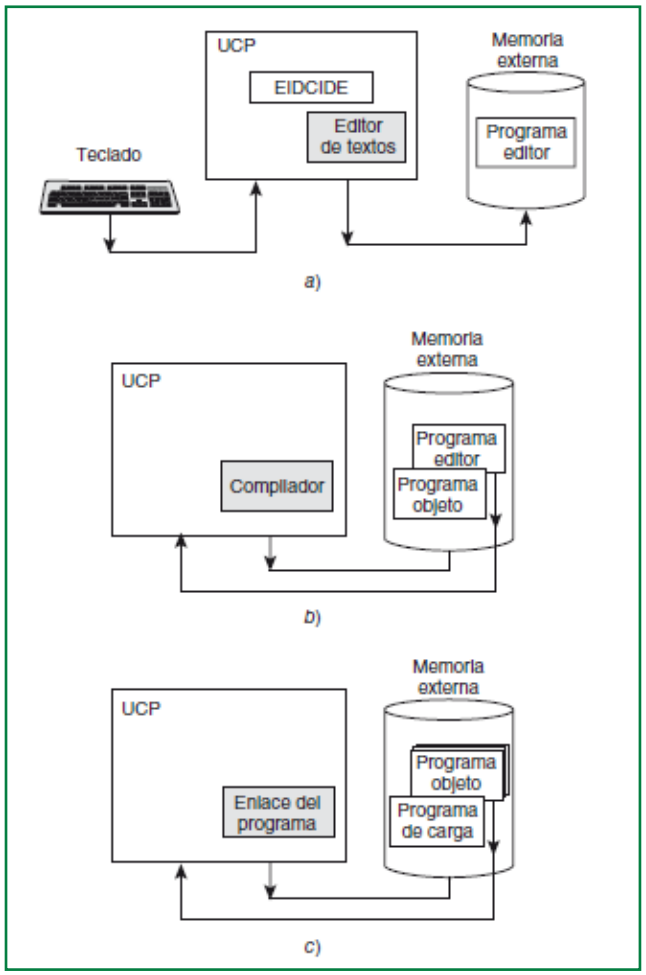


Figura 4.6. Proceso completo de compilación/ ejecución de un programa. Fuente: (Joyanes, 2008, p. 53).

Otra fase en la construcción de software es la documentación, en la cual se deberán describir y plasmar tanto, a nivel del código fuente como en documentos externos, los detalles del software, como son sus requerimientos funcionales, sus requerimientos no funcionales, el algoritmo o algoritmos utilizados, los diagramas de flujo utilizados, casos de usos, desviaciones, casos de negocio, etc.

Un software no documentando o documentado parcialmente presentará problemas en el futuro para su mantenimiento, revisión, procesos de auditoría, modificaciones, depuraciones, entre otros.

Como se mencionó, la documentación deberá ser tanto a nivel de código fuente como en la documentación externa. Cuando es como código fuente nos referimos a la documentación interna, en la cual, a través de comentarios de código fuente, se describen los aspectos más relevantes de las líneas de programación escrita por los programadores, explicaciones breves de variables o constantes o funciones principales del código fuente. Por otra parte, la documentación externa incluye, los manuales de usuarios, descripción de los algoritmos, diagramas de flujo, requerimientos funcionales y no funcionales, así como notas importantes o memorias técnicas que surgieron en el camino de la construcción del software.

La documentación deberá realizarse desde las primeras fases de construcción del software, desde el levantamiento y entendimiento de requerimientos y necesidades del cliente. Contar con una documentación detallada y completa ayudará en el futuro para el mantenimiento del software, así como el cumplimiento de controles internos y procesos de auditoría.

Durante las posibles modificaciones del software o mantenimiento de este, será necesario llevar un proceso de generación de versiones del software, así como la documentación de cada versión. En este último aspecto se deberá especificar los cambios realizados, fechas, pruebas y conservar un respaldo de las versiones anteriores u obsoletas del código fuente y su documentación relacionada.

4.2. Definición de datos de salida y postcondiciones

Cualquier equipo de cómputo, así como el software que estos utilizan, hace uso de datos de salida y postcondiciones. En términos generales los datos de salida son presentados a través de algunos dispositivos periféricos de salida estándar como lo son: impresoras, impresoras 3D, dispositivos de almacenamiento externo como discos duros externos, cintas magnéticas, USB, etc. Las operaciones de salida o datos de salida normalmente son el resultado de la finalización de una tarea específica del software, o finalización de este, normalmente estos procesos de salida de datos son proceso de escritura en archivos.

Durante las primeras fases de ejecución del software, este utilizará datos de entrada o precondiciones que serán necesarias para el inicio de ejecución o realización de funciones y/o instrucciones, después de la realización de dichas instrucciones u operaciones el resultado obtenido deberá ser mostrado o desplegado en algún lado, lo cual son nuestros datos de salida.

En forma general, observamos que la ejecución de un programa se debe de tener entradas (datos de entrada), tareas o instrucciones (operaciones del software) y nos dará como resultado una salida (datos de salida). Estos datos de salida dependiendo

de la naturaleza del software o del sistema podrán ser los datos de entrada para otro software, por otra parte, estos datos de salida podrían brindar las postcondiciones necesarias para la toma de alguna decisión o inicio de otro operación o función.

A continuación, se muestran algunos ejemplos de cómo expresar datos de salida en un programa. Estas instrucciones mostrarán en pantalla o escribirán en un archivo los datos de algunas variables:

A ← 100
B ← 200
C ← 300
escribir (A, B, C)

Se visualizarían en la pantalla o imprimirían en la impresora los valores 100, 200 y 300 que contienen las variables A, B y C.

4.3. Definición de datos de entrada y precondiciones

Como se mencionó en el subtema anterior, un programa es, en términos prácticos, un proceso conformado por un conjunto de entradas (datos de entrada), tareas o instrucciones (operaciones del software) y que dará como resultado una salida (datos de salida). En este caso se deben de abordar todos aquellos datos necesarios para iniciar la ejecución del programa además de cumplir una serie de precondiciones necesarias para iniciar las instrucciones o funciones.

Una analogía de esta fase sería imaginar la preparación de un pastel, si bien conocemos el resultado o salida que es el pastel ya terminado y sus proceso o instrucciones para llegar a dicho resultado, no podemos realizar o iniciar la preparación sino contamos con las entradas, en este caso los ingredientes como leche, harina, huevos, etc. Además, es importante considerar las precondiciones necesarias, como podrían ser el contar con gas, un horno, haber tenido una revisión del proceso, etc.

Con base en el ejemplo anterior se confirma que el programa es un proceso para la solución de problemas, en el cual cada elemento es importante para llegar al resultado deseado.

En el proceso de programación y/o construcción del código fuente, los desarrolladores de software establecen los datos de entrada necesarios para que el software pueda iniciar sus funciones y ejecute de forma adecuada sus instrucciones. Estos datos de entrada se pueden presentar de varias formas, la primera es que pueden ser datos definidos como constantes dentro del programa, otra forma es a través de la consulta de información externa al programa, como la consulta de información de una base de datos o la consulta de una hora. La última forma es a través de los datos proporcionados directamente por el usuario, ya sea vía el teclado, la carga de archivos, consulta a un disco duro interno o externo o consulta de información en memoria.

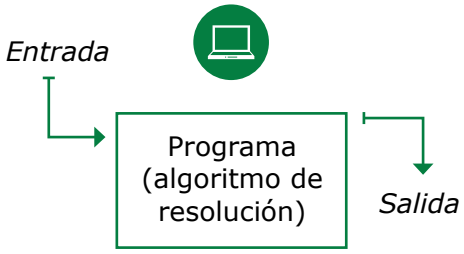
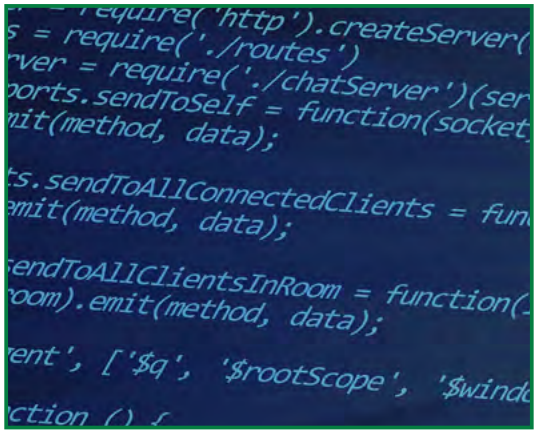


Figura 4.7. Proceso de programación.
Fuente: (Joyanes, 2008, p. 85).



En los siguientes ejemplos se muestran algunas formas de expresar datos de entrada dentro de un programa, en el primer ejemplo se necesitan ingresar tres valores para calcular la ganancia de una inversión.

a) leer (IINVERSION, TIEMPO, TASA)

Este software solicitará al usuario que ingrese los valores para las tres variables, estos datos se ingresaran vía el teclado el cual es la entrada de datos estándar para cualquier equipo de cómputo. Una vez que los datos sean ingresados, el software asignará estos valores a las variables definidas y los almacenará durante la ejecución de las siguientes instrucciones.

INVERSIÓN = 10000
TIEMPO = 365
TASA = 10.0

RESUMEN DE LA UNIDAD



En esta unidad se analizó lo correspondiente a la introducción a la metodología que hay que seguir para la resolución de problemas con computadoras. La resolución de un problema con una computadora se hace escribiendo un programa que exige al menos los siguientes pasos:

1. Definición o análisis del problema.
2. Diseño del algoritmo.
3. Transformación del algoritmo en un programa.
4. Ejecución y validación del programa.

También se revisaron los primeros pasos a seguir para que un desarrollador de software cuente con las herramientas necesarias que le permitan tener una interacción con una computadora, esto a través de instrucción escritas en un lenguaje de programación en el cual se generará un programa.

Por último, se revisaron los primeros elementos que necesita un programa desde sus datos de entrada y/o precondiciones para iniciar sus instrucciones, así como los datos de salidas los cuales serán el resultado de las operaciones internas.

ACTIVIDADES DE APRENDIZAJE

Unidad 4, actividad inicial. *Adjuntar archivo.* A partir del estudio del contenido de la unidad, elabora un mapa conceptual u organizador gráfico sobre los temas vistos. Dentro del documento podrás encontrar recomendaciones de software para su elaboración.

1. **Unidad 4, actividad 1. *Adjuntar archivo.*** A partir de los aprendizajes obtenidos en esta unidad, identifica los elementos más importantes del siguiente problema (elementos de entrada, elementos de salida, proceso principal y objetivo). Se quiere realizar una tabla con las depreciaciones acumuladas y los valores reales anuales de un automóvil durante los seis años posteriores a la compra, que se llevó a cabo en el año 2005. El valor del vehículo fue de 20 000 dólares, suponiendo un valor de recuperación o rescate de 2 000. Elabora el análisis del problema, si se conoce la fórmula de la depreciación anual constante para cada año de vida útil.

2. **Unidad 4, actividad 2. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza un pseudocódigo: explica con tus propias palabras el proceso de compilación, ejecución, verificación y depuración de un programa.

3. **Unidad 4, actividad 3. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza el diagrama de flujo (formato libre) en el cual muestres el proceso completo de resolución de problemas.

4. **Unidad 4, actividad complementaria 1. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

5. **Unidad 4, actividad complementaria 2. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

6. **Unidad 4, actividad complementaria 3. *Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

ACTIVIDAD INTEGRADORA

Adjuntar archivo.

En esta unidad aprendiste sobre el alcance de un problema en términos algorítmicos. A partir de los conocimientos adquiridos desarrolla la siguiente actividad integradora:

Realizar un cuadro conceptual en el cual expliques con tus palabras los aspectos más importantes de cada fase del proceso de resolución de problemas.

CUESTIONARIO DE REFORZAMIENTO

Adjuntar archivo.

Contesta las siguientes preguntas.

1. ¿Cuál es el objetivo del proceso de resolución de problemas?
2. Menciona las fases del proceso de resolución de problemas.
3. Describe con tus palabras la fase de análisis del problema.
4. Describe con tus palabras la fase de diseño de algoritmos.
5. Describe con tus palabras la fase de codificación.
6. Describe con tus palabras la fase de compilación, ejecución, verificación y depuración.
7. Describe con tus palabras la fase de mantenimiento y documentación.
8. Menciona cuáles son las herramientas básicas para el diseño de algoritmos.
9. Describe qué es un pseudocódigo y menciona sus características.
10. Describe qué es un diagrama de flujo y menciona sus características.

AUTOEVALUACIÓN

1.- Relaciona las columnas con la respuesta correcta. Elige el inciso que responda correctamente la frase.

a) Análisis	b) Mantenimiento	c) Diseño	d) Compilación y ejecución	e) Codificación

1 El programa se ejecuta, se comprueba rigurosamente y se eliminan todos los errores (en inglés denominados bugs) que puedan aparecer.

2. El programa se actualiza y modifica, cada vez que sea necesario, de modo que se cumplan todas las necesidades de cambio de sus usuarios.

3. El problema se analiza teniendo presente la especificación de los requisitos dados por el cliente de la empresa o por la persona que encarga el programa.

4. La solución se escribe en la sintaxis del lenguaje de alto nivel (por ejemplo, C++ o Python) y se obtiene un programa fuente que se compila a continuación.

5. Una vez analizado el problema, se diseña una solución que conducirá a un algoritmo que resuelva el problema.

II.- Elige la letra (V) si la respuesta es verdadera o falso (F) si es falsa.

1. El diagrama de flujo y el pseudocódigo son herramientas de diseño de algoritmos. ◯

2. Un programa de computadora es un conjunto de instrucciones, órdenes dadas a la máquina que producirán la ejecución de una determinada tarea. ◯

3. Las pantallas, impresoras, discos son ejemplos de entradas de datos. ◯

4. La escritura de un programa es la implementación de un algoritmo. ◯

5. El proceso de programación es, por consiguiente, un proceso de solución de problemas. ◯

RESPUESTAS DE AUTOEVALUACIÓN

En este apartado encontrarás las respuestas al examen por unidad.

UNIDAD 4	
I. Solución	
1.	d
2.	b
3.	a
4.	e
5.	c

UNIDAD 4	
II. Solución	
1.	v
2.	v
3.	f
4.	v
5.	v

REFERENCIAS

Custodio, C. (2022). *Algoritmos, diagramas de flujo y pseudocódigo: Los fundamentos de la programación*. Play books.

Joyanes, L. (2008), *Fundamentos de programación. Algoritmos, estructuras de datos y objetos*. McGraw Hill.

Martí, N. (2004). E*structuras de datos y métodos algorítmicos: ejercicios resueltos*. Pearson.

UNIDAD 5

DISEÑO DE ALGORITMOS

OBJETIVO PARTICULAR

Al finalizar la unidad, el alumnado diseñará soluciones algorítmicas de problemas con el apoyo de los elementos sintácticos de un lenguaje de programación.

TEMARIO DETALLADO

(20 horas)

- 5. Diseño de algoritmos
  - 5.1. Estrategias de diseño de algoritmos
    - 5.1.1. Fuerza bruta
    - 5.1.2. Divide y vencerás
    - 5.1.3. Recursividad
    - 5.1.4. Programación dinámica
    - 5.1.4. Algoritmos ávidos o voraces
    - 5.1.5. Backtracking
    - 5.2. Herramientas de diseño de algoritmos
    - 5.2.1. Pseudocódigo
    - 5.2.2. Diagrama de flujo
    - 5.2.3. Diagrama Nassi-Schneiderman
    - 5.3. Sentencias
    - 5.3.1. Secuenciales
    - 5.3.2. Selectivas
    - 5.3.2.1. Selección simple (SI ENTONCES)
    - 5.3.2.2. Selección doble (SI ENTONCES DE LO CONTRARIO)
    - 5.3.2.3. Anidamiento
    - 5.3.2.4. Selección múltiple (EN CASO DE)
    - 5.3.3. Iterativas
    - 5.3.3.1. Iteración con precondición (MIENTRAS)
    - 5.3.3.2. Iteración con postcondición (HACER MIENTRAS)
    - 5.3.3.3. Iteración para número determinado de veces (DESDE HASTA)
    - 5.3.3.4. Clasificación de variables de acuerdo a su función
    - 5.3.3.4.1. Contadores
    - 5.3.3.4.2. Acumuladores
    - 5.3.3.4.3. Banderas
    - 5.3.3.4.4. Centinelas

ACTIVIDAD DIAGNÓSTICA



Adjuntar archivo.

Con base a lo visto en la unidad anterior, realiza un mapa mental del proceso de resolución de problemas.

INTRODUCCIÓN



En la presente unidad se revisarán las diferentes estrategias de diseño de algoritmos que un desarrollador de software o analista de sistemas puede utilizar para la resolución de problemas, su funcionamiento, así como las herramientas de diseño de algoritmos las cuales puede utilizar para expresar las propuestas de solución planteadas.

Las estrategias de diseño que se presentarán permitirán tener una percepción de las diferentes formas en las cuales se puede abordar y resolver un problema, así como las ventajas de utilizar cada estrategia, permitiendo ampliar la visión y determinar cuál estrategia sea la mejor forma de responder a un problema en específico. Cada estrategia plantea un paradigma para la resolución de problemas, dichos paradigmas permiten encontrar soluciones eficientes, así como eficaces, dependiendo tanto de las condiciones de los problemas como de los requerimientos de este.

Por otra parte, se revisarán los diferentes tipos de sentencia que se utilizan dentro de un programa, las cuales darán la estructura del programa, así como los casos en los cuales pueden ser utilizados.

En cada tipo de sentencia presentado se podrá entender cómo es posible representar las diferentes instrucciones de un algoritmo a un lenguaje de programación, de igual forma se abordan diferentes situaciones dentro de un programa: el envío de un mensaje, resolver una situación en específico, evaluar precondiciones y/o postcondiciones, determinar que camino a seguir dentro de una condición, etc. Cada sentencia es la representación en la cual una computadora podrá resolver un problema.

5.1.
Estrategias de diseño de algoritmos

En la fase de análisis de problemas, se pueden presentar varias soluciones para cumplir con el objetivo, sin embargo, podemos hacer uso de varias estrategias de diseño de algoritmos, las cuales nos proporcionarán algunos enfoques para hacer frente a cualquier tipo de problema. En la siguiente sección se mostrarán las estrategias de diseño de algoritmos más utilizados exponiendo sus beneficios, ventajas y en qué circunstancias es más favorable utilizarlas.

5.1.1.
Fuerza bruta

Dentro de las técnicas y/o estrategias para el diseño de algoritmos podemos hacer uso de la técnica llamada fuerza bruta o Búsqueda exhaustiva, los algoritmos basados en este tipo de técnica se caracterizan en explorar todas las posibles combinaciones de elementos para encontrar una solución.

Uno de los principales criterios para hacer uso de esta técnica es cuando se aborden problemas de optimización, ya que este tipo de problema se deberá resolver explorando todas las posibles soluciones utilizando todos los elementos (variables y constantes). Una vez exploradas todas las posibles soluciones se escogerá aquella que optimice una determinada función o cumpla cierto objetivo como tiempo de ejecución, optimización de memoria (volátil o no volátil), etc.

Una de las características al desarrollar algoritmos a través de fuerza bruta es que durante la primera ejecución llevará un tiempo más amplio dado que buscará todas aquellas soluciones posibles. Después de la primera ejecución, y bajo ciertas condiciones, las ejecuciones subsecuentes podrían llevar más tiempo dado que ya se tiene un registro de soluciones previas. Si la naturaleza del problema requiere o conlleva un cambio se tendrá que volver a realizar la búsqueda de todas las soluciones.

5.1.2.
Divide y vencerás

Otra técnica común en el diseño de algoritmos es divide y vencerás, que consta de dos partes:

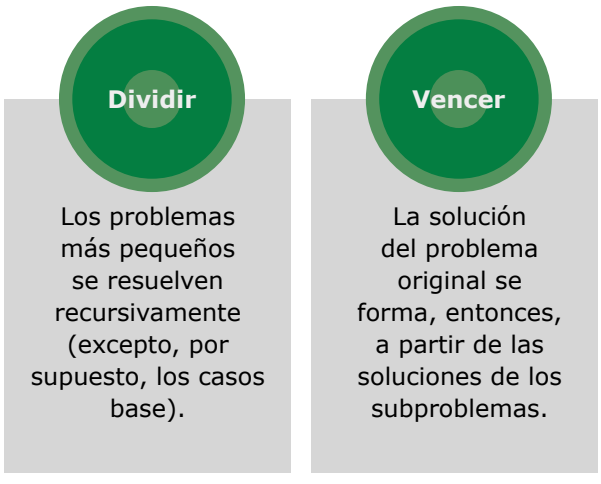


Figura 5.1. Técnica de diseño de algoritmos.

Las rutinas en las cuales el texto contiene al menos dos llamadas recursivas se denominan ***algoritmos de divide y vencerás***; no así aquellas cuyo texto solo comprende una.

La idea de la técnica divide y vencerás es dividir un problema en subproblemas del mismo tipo y, aproximadamente, del mismo tamaño; resolver los subproblemas recursivamente y combinar la solución de los subproblemas para dar una solución al problema original.

La recursión finaliza cuando el problema es pequeño y la solución fácil de construir directamente.

5.1.3. Recursividad

Otra estrategia de diseño de algoritmos es utilizar las funciones de recursividad, las cuales tienen la propiedad de llamarse a sí mismas, esto significa que una sección o función del código tendrá la capacidad de ejecutarse a sí misma hasta que sea necesario o se cumpla una condición de salida, en términos de programación un subprograma puede llamar a otro subprograma, pero en este caso se llamaría de nuevo, como se muestra a continuación:

A **llamar_a** A
o bien
A **llamar_a** B, B **llamar_a** A

En primera instancia, parece incorrecta. Sin embargo, existen lenguajes de programación Pascal, C, entre otros, en que un subprograma puede llamarse a sí mismo.

Una función o procedimiento que se puede llamar a sí mismo se llama recursivo. (Bisbal, 2009, p. 15). La recursión (recursividad) es una herramienta muy potente en algunas aplicaciones, sobre todo de cálculo. La recursión puede ser utilizada como una alternativa a la repetición o estructura repetitiva. El uso de la recursión es particularmente idóneo para la solución de aquellos problemas que pueden definirse de modo natural en términos recursivos.

La escritura de un procedimiento o función recursiva es similar a sus homónimos no recursivos; sin embargo, para evitar que la recursión continúe indefinidamente es preciso incluir una condición de terminación. La razón de que existan lenguajes que admiten la recursividad se debe a la existencia de estructuras específicas tipo pilas (*stack*, en inglés) para este tipo de procesos y memorias dinámicas. Las direcciones de retorno y el estado de cada subprograma se guardan en estructuras tipo pilas.

5.1.4. Programación dinámica

La programación dinámica fue trabajada inicialmente por el matemático Richard Bellman en 1953, consiste en un método que reduce el tiempo de ejecución de un algoritmo mediante la utilización de subproblemas superpuestos y subestructuras óptimas. Una subestructura óptima significa que soluciones óptimas de subproblemas pueden ser usadas para encontrar las soluciones óptimas del problema en su conjunto.

En general, se pueden resolver problemas con subestructuras óptimas siguiendo estos pasos:

1. Dividir el problema en subproblemas más pequeños.
2. Resolver estos problemas de la mejor manera usando este proceso de tres pasos recursivamente.
3. Aplicar estas soluciones óptimas para construir una solución óptima al problema original.

Los subproblemas se resuelven, a su vez, dividiéndolos en subproblemas más pequeños, hasta alcanzar el caso fácil, donde la solución al problema es trivial.

5.1.4. Algoritmos ávidos y voraces

Algoritmos voraces

Suelen utilizarse en la solución de problemas de optimización y se distinguen porque son:

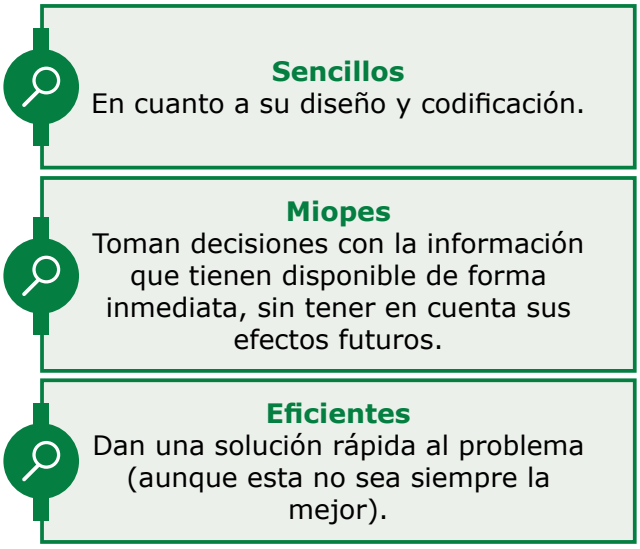


Figura 5.2. Tipos de algoritmos voraces.

Tienen las propiedades siguientes:

- Tratan de resolver problemas de forma óptima.
- Disponen de un conjunto o lista de candidatos.

A medida que avanza el algoritmo, se acumulan dos conjuntos:

- Candidatos considerados y seleccionados.
- Candidatos considerados y rechazados.

Existe una función que comprueba si un cierto conjunto de candidatos constituye una solución del problema, ignorando si por el momento es óptima o no.

Los algoritmos voraces son simples, por lo común se emplean para resolver problemas de optimización. Por mencionar un ejemplo, encontrar la secuencia óptima para procesar un conjunto de tareas por una computadora; o hallar el camino mínimo de un grafo, entre otras.

Por lo regular, intervienen los siguientes elementos:

- Un conjunto o lista de candidatos (tareas a procesar, vértices del grafo, etcétera).
- Una función que determina si un conjunto de candidatos es una solución al problema (aunque no tiene por qué ser la óptima).
- Una función que determina si un conjunto es completable, es decir, si añadiendo a este conjunto nuevos

candidatos es posible alcanzar una solución al problema, suponiendo que esta exista.
- Una función de selección que escoge el candidato, aún no seleccionado, que es más prometedor.
- Una función objetivo que da el valor/costo de una solución (tiempo total del proceso, longitud del camino, etcétera) y es la que se pretende maximizar o minimizar.

5.1.5. *Backtracking*

El término *backtrack* fue acuñado por el matemático estadounidense D. H. Lehmer, en la década de 1950. Es una estrategia para encontrar soluciones a problemas que satisfacen restricciones.

Los problemas que deben satisfacer un determinado tipo de restricciones son completos, donde el orden de los elementos de la solución no importa. Consisten en un conjunto o lista de variables en la que a cada una se le debe asignar un valor sujeto a las restricciones del problema.

La técnica va creando todas las combinaciones de elementos posibles para llegar a una solución. Su principal virtud es que en la mayoría de las implementaciones se pueden evitar combinaciones estableciendo funciones de acotación (o poda) y reduciendo el tiempo de ejecución.

La vuelta atrás está muy relacionada con la búsqueda combinatoria. La idea es encontrar la mejor combinación en un momento determinado, por eso se dice que este tipo de algoritmo es una "búsqueda en profundidad". Si se halla una alternativa incorrecta, la búsqueda retrocede hasta el paso anterior y toma la siguiente alternativa. Cuando se han terminado las posibilidades, se vuelve a la elección anterior y se toma la siguiente opción. Si no hay más alternativas, la búsqueda falla.

Normalmente, se suele implementar este tipo de algoritmos como un procedimiento recursivo. Así, en cada llamada al procedimiento se toma una variable y se le asignan todos los valores posibles, llamando a su vez al procedimiento para cada uno de los nuevos estados.

La diferencia con la búsqueda en profundidad es que se suelen diseñar funciones de cota, de modo que no se generen algunos estados si no van a conducir a ninguna solución, o a una solución peor de la que ya se tiene. De esta forma se ahorra espacio en memoria y tiempo de ejecución.

Es una técnica de programación para hacer una búsqueda sistemática a través de todas las configuraciones posibles dentro de un espacio de búsqueda. Para lograrlo, los algoritmos de tipo *backtracking* construyen posibles soluciones candidatas de manera sistemática. En general, dada una solución candidata:

1. Verifican si *s* es solución. Si lo es, hacen algo con ella (depende del problema).
2. Construyen todas las posibles extensiones de *s* e invocan recursivamente al algoritmo con todas ellas.

A veces, los algoritmos *backtracking* se aprovechan para encontrar una solución nada más, pero otras veces conviene que las revisen todas, por ejemplo, para ubicar la más corta.

5.2. Herramientas de diseño de algoritmos

Como se ha visto en las unidades anteriores, una vez realizado el análisis correspondiente para solucionar un problema, el siguiente paso a realizar es desarrollar y describir la solución o las posibles soluciones en algoritmos. Un algoritmo será la descripción paso a paso en un lenguaje común y natural. Retomando lo visto anteriormente, un algoritmo es un método que define una serie de pasos para alcanzar un objetivo o resolver un problema, los algoritmos están compuestos por las siguientes propiedades:

1.- Es finito, lo que implica que tiene un inicio y un fin.
2.- Cada vez que se ejecuta se llega al mismo resultado.
3.- Tiene una secuencia lógica (serie de pasos), sin ambigüedad.

Adicionalmente a las características mencionadas, un algoritmo nos ayuda a responder la pregunta “¿Qué hacer?”, por lo regular siempre ejecutará sus instrucciones de forma secuencial, si bien se pueden describir diferentes escenarios dependiendo de las entradas o circunstancias del problema en específico, este siempre cumplirá con un flujo determinado.

Al escribir algoritmos normalmente se cumplen ciertos criterios o reglas, la primera de ellas es el sangrado o justificación, en la cual, dependiendo del tipo de descripción y/o instrucción, estas sentencias tendrán una justificación diferente, permitiendo que sea entendible cada una y observar las estructuras de control fundamentales, como lo son estructuras selectivas, secuenciales y repetitivas.

La escritura de algoritmos permite además de plasmar la solución en un modelo entendible, tanto para analistas como programadores, facilita la identificación de puntos de mejora, reducción de pasos, reutilización de soluciones, entre otras.

5.2.1. Pseudocódigo

Como primera herramienta de diseño de algoritmos tenemos el pseudocódigo, el cual se define como una descripción detallada expresada en un lenguaje natural. El uso del lenguaje utilizado en pseudocódigos permite ser el antecedente a la fase de codificación y/o escritura de los algoritmos a un lenguaje de programación de alto nivel, los cuales son lenguajes de programación parecidos al lenguaje humano. Algunos ejemplos de lenguajes de programación de alto nivel son; JAVA, C, Python, etc.

Los pseudocódigos nacen con la intención de plasmar en un lenguaje similar a un lenguaje de programación real. Esta representación en forma de descripción y/o narración simplifica la forma de traducir un algoritmo a un lenguaje natural y dar un punto de entrada a la codificación a un lenguaje de programación. Cabe aclarar que un pseudocódigo puede ser entendido en términos generales por cualquier persona y podría incluir en su estructura palabras reservadas a una estructura similar de un lenguaje de programación, sin embargo, un pseudocódigo no puede ser interpretado ni ejecutado por una computadora.

Por otra parte, una de las ventajas de utilizar los pseudocódigos por los programadores es que estos pueden tener una visión general de la ejecución, funciones y tareas a realizar, así como tener la posibilidad de centrarse en la lógica y las estructuras de control que deberán utilizarse. Adicionalmente al desarrollar y revisar una solución en pseudocódigo nos permite encontrar algunos errores en etapas tempranas del desarrollo de software y las modificaciones en este son más fáciles de corregir y disminuyen el esfuerzo en la realización de cambios, además de que dependiendo de la experiencia de los analistas y programadores del proyecto, un pseudocódigo representa una forma sencilla para traducir sus instrucciones a un lenguaje de programación como JAVA, C, Python, JavaScript, etc.

Dentro de la descripción que se plantea en un pseudocódigo se pueden utilizar palabras en inglés que señalan actividades o acciones que comúnmente son las mismas que cualquier lenguaje de programación utiliza: *start, end, stop, go, if, if-else, while, do-while, until, for, etc.*

A continuación, se presenta un ejemplo de un pseudocódigo en el cual se resuelve el problema de cálculo del salario neto de un trabajador:

```
start
//cálculo de impuestos y salarios, esta línea
es utilizada como comentarios
read nombre, horas, precio
salario = horas*precio
tasas = .25*salario
salario_neto = salario – tasas
write nombre, salario, tasas, salario_neto
end
```

En el algoritmo anterior se observa que el inicio comienza con la palabra start y finaliza con la palabra end, en estas sentencias solo se escribe una sola instrucción o una acción por línea.

Otro aspecto importante para destacar son las líneas que inician con //, las cuales son líneas denominadas de comentarios. Estas líneas, dentro del pseudocódigo, son notas informáticas para el lector y/o también son líneas que nos ayudan a documentar el proyecto. Estas secciones de comentarios no realizan ningún tipo de instrucción o acción para el funcionamiento del programa, una práctica que normalmente se realiza en la etapa de codificación es documentar mediante estos comentarios dentro del mismo código. Algunos autores utilizan algunos otros símbolos para especificar comentarios como lo son comillas, apostrofes, corchetes,

entre otros, si bien se puede establecer el símbolo dependiendo del autor de consulta o las practicas que utilicen los miembros del proyecto, se debe de especificar y dar a conocer estas características ya que si se incorpora un miembro nuevo al equipo podría causar confusión en el momento de realizar la codificación.

En el siguiente ejemplo se muestra el uso del pseudocódigo podría ser un sencillo algoritmo del arranque de un coche.

```
inicio
//arranque matinal de un coche
introducir la llave de contacto
girar la llave de contacto
pisar el acelerador
escuchar el ruido del motor
pisar de nuevo el acelerador
esperar unos instantes a que se caliente el
motor
fin
```

En el ejemplo anterior se observa el uso de las palabras inicio y fin, las cuales están en español, la ventaja de la utilización de pseudocódigos es la flexibilidad de la lengua natural a utilizar. Otros términos que pueden ser sustituidos del inglés al español se observan en las instrucciones para las estructuras de control de programación como; *leer, escribir, si, si-entonces, mientras, hasta que, repetir*, etc. La utilización de pseudocódigos en el idioma español ha permitido facilitar la compresión de estos temas y, en consecuencia, facilita la codificación a un lenguaje de programación.

5.2.2. Diagrama de flujo

Los diagramas de flujo son la representación gráfica de los algoritmos. Elaborarlos implica diseñar un diagrama de bloque que contenga un bosquejo general del algoritmo y, con base en este proceder, a su ejecución con todos los detalles necesarios.

Reglas para construir diagramas de flujo:

1. Debe diagramarse de arriba hacia abajo y de izquierda a derecha. Es una buena costumbre en la diagramación que el conjunto de gráficos tenga un orden.
2. El diagrama solo tendrá un punto de inicio y uno final. Aunque en el flujo lógico se tomen varios caminos, siempre debe existir una sola salida.
3. Usar notaciones sencillas dentro de los gráficos, si se requieren notas adicionales colocarlas en el gráfico de anotaciones a su lado.
4. Se deben inicializar todas las variables al principio del diagrama. Esto es muy recomendable, pues ayuda a recordar todas las variables, constantes y arreglos que van a ser utilizados en la ejecución del programa. Además, nunca sabemos cuándo otra persona modificará el diagrama y necesitará saber de estos datos.

5. Procurar no cargar demasiado una página con gráficos; si es necesario, utilizar más hojas, emplear conectores. Cuando los algoritmos son muy grandes, se pueden utilizar varias hojas, con conectores de hoja para cada punto en donde se bifurque a otra hoja.
6. Todos los gráficos estarán conectados con flechas de flujo. Jamás debe dejarse un gráfico sin que tenga alguna salida, a excepción del que marque el final del diagrama.

Terminado el diagrama de flujo, se realiza la prueba de escritorio. Es decir, se le da un seguimiento manual al algoritmo, llevando el control de variables y resultados de impresión en forma tabular.



Figura 5.3. Ventajas y desventajas del desarrollo de programas.

Algunos gráficos usados en los diagramas:

SÍMBOLO	DESCRIPCIÓN
	Terminal. Indica el inicio y el final del diagrama de flujo.
	Entrada/Salida. Marca la entrada y salida de datos.
	Proceso. Señala la asignación de un valor en la memoria y/o la ejecución de una operación aritmética.
	Decisión. Simboliza la realización de una comparación de valores.
	Proceso predefinido. Representa los subprogramas.
	Conector de página. Indica la continuidad del diagrama dentro de la misma página.
	Conector de hoja. Representa la continuidad del diagrama en otra página.
	Impresión. Indica la salida de información por impresora.
	Pantalla. Marca la salida de información en el monitor de la computadora.
	Flujo. Simbolizan la secuencia en que se realizan las operaciones.

Figura 5.4 Símbolos utilizados en los diagramas. Basado en: (Cairó, 2006, p. 5).

A continuación, se presenta un ejemplo de diagrama de flujo.

Algoritmo: imprimir los factoriales para los números del 1 al 10.

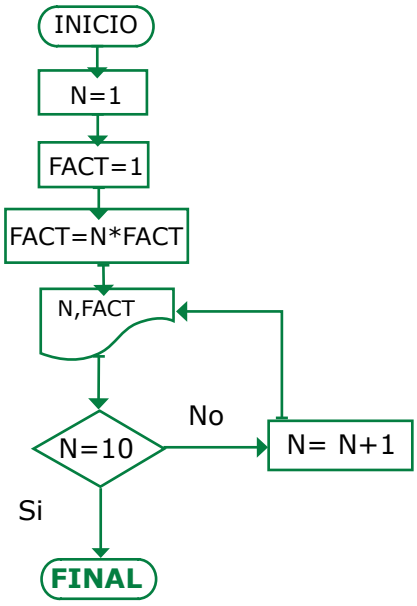


Figura 5.5. Ejemplo de diagrama de flujo.

Puede haber varias soluciones, pero una será la óptima.

5.2.3. Diagrama Nassi-Schneiderman

Diagrama de Nassi/Shneiderman (N/S)

El diagrama estructurado N/S, también conocido como diagrama de Chapin, (Oviedo, 2004, p. 37), es parecido a uno de flujo en el que se omiten las flechas de unión y las cajas son contiguas. Las acciones sucesivas se pueden escribir en cajas sucesivas y, como en los diagramas de flujo, se anotan diferentes acciones en cada caja. Un algoritmo se representa en la siguiente forma:

```
Inicio
 Accion1
 Accion2
 . . .
Fin
```

A continuación, se ejemplifica el uso de este tipo de diagrama.

Algoritmo: cálculo del salario neto a partir de las horas laboradas, el costo por hora y la tasa de impuesto del 16% sobre el salario.

```
Inicio
 Leer
 Nombre,Hrs,Precio
 Calcular
 Salario = Hrs * Precio
 Calcular
 Imp = Salario* 0.16
 Calcular
 Neto = Salario + Imp
 Escribir
 Nombre, Imp, SNeto
Fin
```

5.3.

Sentencias

A medida que ha pasado el tiempo en el área de las ciencias computacionales se han observado un gran número de avances, desde el mejoramiento en el rendimiento de los equipos de cómputo, la miniaturización de los componentes, el número de dispositivos, las interfaces, la interacción hombre-maquina y, por su parte, la programación no ha sido la excepción. La programación en la actualidad ha evolucionado desde lo que fue en sus inicios, se ha simplificado cada vez más, en sus inicios la programación era en su totalidad a través de un lenguaje máquina, todo a través de ceros y unos que se le indicaban a un equipo de cómputo conectado y desconectado un *switch* o a través de impulsos eléctricos. Después a través de tarjetas perforadas, uso de mnemónicos, lenguaje ensamblador, hasta la llegada de lenguajes de programación de alto nivel, de los cuales los primeros fueron PASCAL, FORTRAN, C, hasta los más modernos como GO, JAVA, Python, etc.

La llegada de los lenguajes de programación a finales de los años sesenta posibilitó que cualquier tipo de algoritmo, sin importar su grado de complejidad, podría ser construido en lenguajes de programación más parecidos al lenguaje humano y con la facilidad de poder construir las soluciones a través de la combinación de las tres estructuras de control (secuencial, selección y repetitiva o iterativa), algunos autores mencionan una estructura de control llamada “invocación” o “jump”.

Las sentencias de selección **if** o si, y **switch** o según-sea; las sentencias iterativas son: **for** o desde, **while** o mientras, **do-while** o hacer-mientras y **repeat-until** o repetir-hasta que. Por último, las de salto son: **break** o romper, **goto** o ir-a y **return** o volver.

Cuando se utiliza el término flujo de control se refiere al orden en el cual se ejecutan las sentencias (instrucciones y/o acciones) del programa, este flujo de control se podría definir como la secuencia de instrucciones en un orden definido por el algoritmo, ya que un flujo podría ser secuencial o con “saltos” o un orden no secuencial.

Un flujo secuencial significa que cada sentencia (instrucción y/o acción) se ejecuta en un orden descendente o línea tras línea (en secuencia), una sentencia después de la otra, en el orden que el programador haya definido. Por otra parte, cuando se utilizan las estructuras de selección, repetición e invocación es posible que el flujo secuencial del programa cambie o sea modificado y no siga la secuencia descendente. Las estructuras de repetición permiten ejecutar, en más de una ocasión, un conjunto de instrucciones o sentencias hasta que se cumpla una condición. Para estructuras de selección nos permiten ejecutar aquellas instrucciones o sentencias específicas sin la necesidad que se encuentren en el orden descendente.

En un programa en el cual su código fuente cuente solamente con sentencias secuenciales, siempre se ejecutará en orden descendente y al ejecutarlo varias veces siempre realizará las mismas acciones, si bien este comportamiento es el esperado en este tipo de sentencias, podría presentar un gran número de limitantes ya que, bajo ciertas condiciones o circunstancias, como evaluaciones o comparaciones, no podría realizar acciones bajo estos escenarios. Este aspecto es importante de mencionar ya que en los problemas de la vida real es muy complicado encontrar soluciones que utilicen solo acciones de forma secuencial, lo que implica que en una solución a un problema este necesite evaluar más condiciones, considerar precondiciones, así como postcondiciones, cambiar el flujo del programa, repetir ciertas instrucciones por un tiempo determinado, esperar una respuesta o mensaje, esperar cambios en datos o valores, dado estos escenarios es necesario hacer uso de otras estructuras de control.

5.3.1. Secuenciales

Como se mencionó en el subtema anterior, una estructura de tipo de secuencial es aquella que realiza o ejecuta su conjunto de instrucciones en forma de secuencia (una acción seguida de otra en forma descendente). Una vez iniciado el programa, este se ejecutará de la misma forma la n-cantidad de veces que se ejecuta, sin presentar ningún cambio. En la siguiente imagen se muestran algunos ejemplos de una estructura secuencial utilizando pseudocódigos, diagrama de flujo y diagrama N-S:

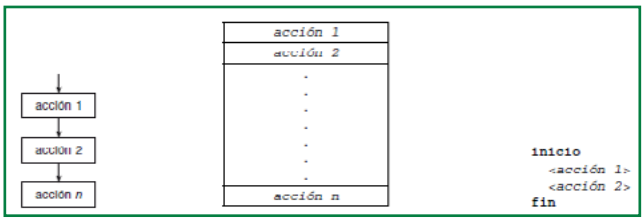


Figura 5.6. Ejemplos de estructura secuencial.
Fuente: (Joyanes, 2008, p. 128)

A continuación, se presenta un ejemplo haciendo uso de estructuras secuenciales, así como se representación en diagrama de flujo y su pseudocódigo correspondiente:

Objetivo: Calcular la suma y el producto de dos números.

Descripción: La suma S de dos números (A y B) es S = A+B y el producto P es P = A*B.

Pseudocódigo:

```
Start //inicio del programa
Leer A
Leer B
S = A + B
P = A * P
Mostrar S
Mostrar P
End //fin del programa
```

Diagrama de flujo:

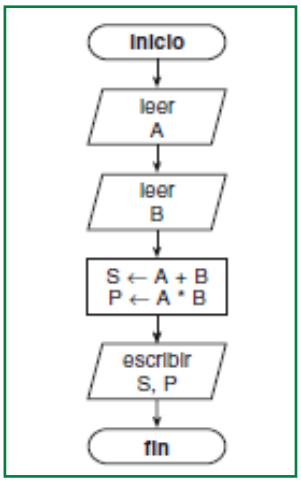


Figura 5.7. Ejemplo de diagrama del Pseudocódigo.

5.3.2. Selectivas

Como se mencionó anteriormente, las estructuras secuenciales podrían presentar limitantes ante problemas complejos, ya que son pocos los casos en los cuales la solución a un problema no considere precondiciones, postcondiciones o una solución totalmente lineal.

Cuando nuestra solución contiene una gran cantidad de posibles alternativas, flujos de control a seguir o la ejecución de la evaluación de ciertas condiciones, se pueden utilizar las estructuras selectivas, las cuales toman decisiones lógicas. En alguna literatura podrían referirse a ellas como **estructuras de decisión**.

Al utilizar estructuras selectivas, se parte del principio de la evaluación de una condición y en función del resultado obtenido de esta condición se tomará cierto camino, alternativa u opción.

Las condiciones se definen a través de expresiones y/o evaluaciones lógicas. Las estructuras selectivas se expresan en pseudocódigo a través de palabras reservadas como; **If** o si, **then** o entonces y **else** o si no.

Para el caso de diagramas de flujo se hace el uso de la figura geométrica rombo o se puede utilizar un triángulo en el interior de una caja o un rectángulo. Las estructuras selectivas pueden ser de varios tipos: simples, dobles y múltiples. Dependiendo de la complejidad del problema, las condiciones del programa o las evaluaciones que se deban de hacer se puede utilizar los diferentes tipos.

5.3.2.1. Selección simple (SI ENTONCES)

Dentro de las estructuras de selección, la estructura de tipo selección simple **if-then**, o si-entonces, es la estructura más sencilla para utilizar, en la cual se ejecutará una determinada acción siempre y cuando se cumpla una condición lógica, la cual podría ser verdadera o sea un valor 1 o falsa o un valor 0.

La condición para evaluar si-entonces deberá considerar:

- Si la condición es verdadera, entonces ejecutara la o las acciones S1 o S-n
- Si la condición es falsa, entonces no se ejecutará ninguna acción o no se realizará nada.

A continuación, se presentan de forma figue con algunos ejemplos de estructuras de selección simple en diagrama de flujo y pseudocódigo:

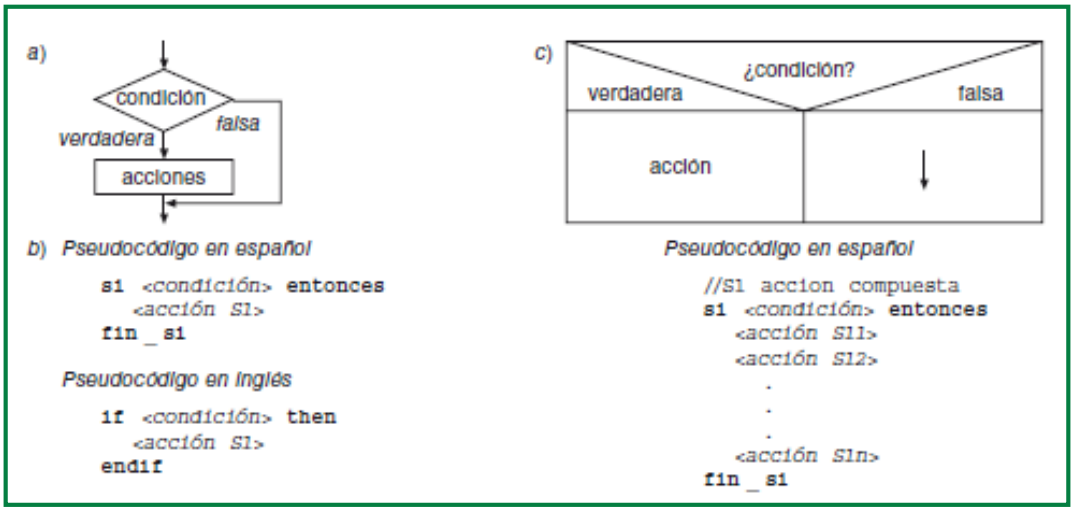


Figura 5.8. Estructura de selección simple en diagrama de flujo. Fuente: (Joyanes, 2008, p. 131).

5.3.2.2.
Selección doble
(SI ENTONCES DE LO CONTRARIO)



El segundo tipo de estructura de selección es la selección doble, si-entonces de lo contrario. La selección simple, si bien nos ayuda a realizar ciertas acciones bajo una condición, sigue siendo limitada ya que solo considera un escenario o una sola opción, en problemas más complejos se necesitará elegir entre dos opciones o alternativas.

La condición para evaluar si-entonces de lo contrario deberá considerar:

- Si la condición es verdadera, entonces ejecutara la o las acciones S1 o S-n
- De lo contrario, entonces se ejecutarán las acciones de S2

A continuación, se presentan de forma gráfica algunos ejemplos de estructuras de selección dobles en diagrama de flujo y pseudocódigo:

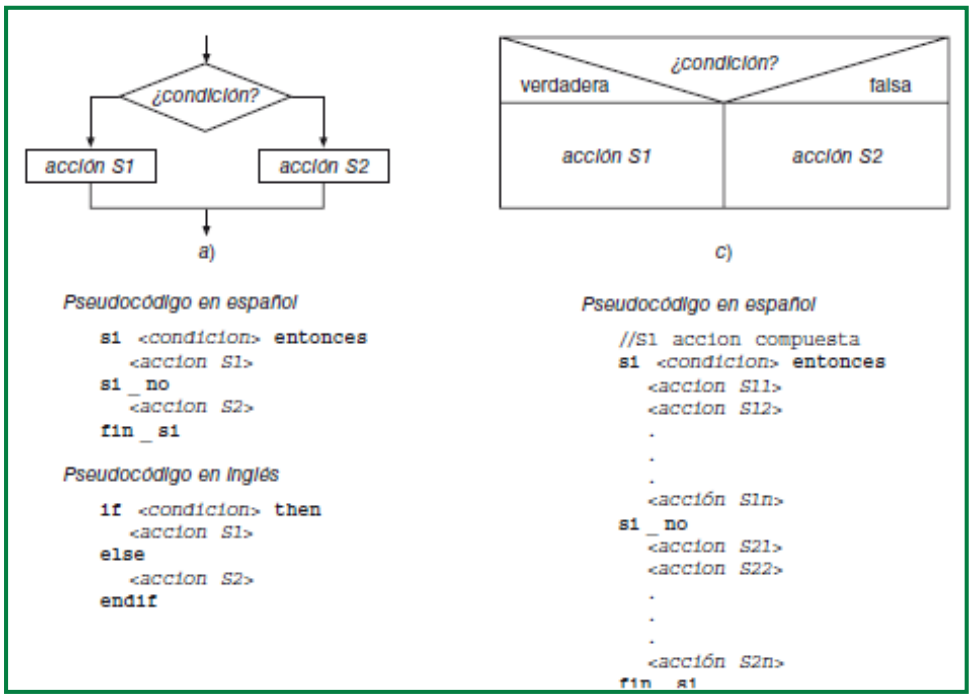
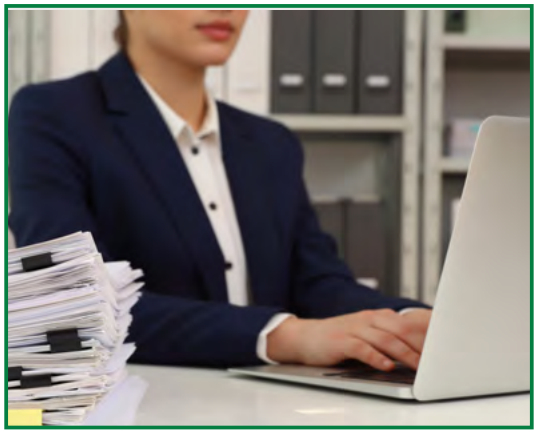


Figura 5.9. Ejemplos de estructura de selección doble en diagrama de flujo y pseudocódigo. Fuente: (Joyanes, 2008, p. 137).

5.3.2.3. Anidamiento



Dentro del tipo de estructura de selección simple y doble, se puede aplicar un conjunto de evaluaciones adicionales o, dicho de otra forma, añadir estructuras de selección simple o doble dentro de una selección simple o doble. Al concepto de aplicar estructuras dentro de otras estructuras del mismo tipo se le conoce como anidamiento.

Un ejemplo de condición con anidamiento para una estructura de selección doble o si-entonces de lo contrario deberá considerar:

- *Si* la condición es verdadera, entonces ejecutara la o las acciones S1 o S-n
  - *Estructura de selección simple anidada*, *Se evalúa la condición anidada.* Si la condición es verdadera, entonces ejecutara la o las acciones S1'-anidada o S-n- anidada
  - *De lo contrario, entonces* se ejecutará las acciones de S1"-anidada
- *De lo contrario, entonces* se ejecutará las acciones de S2
  - *Estructura de selección simple* anidada, Se evalúa la condición anidada. Si la condición es verdadera, entonces ejecutara la o las acciones S2'-anidada o S-n- anidada
  - *De lo contrario, entonces* se ejecutará las acciones de S2"-anidada

Como se observa en las condiciones a considerar anteriormente expuestas, el anidamiento de estructuras de selección dentro de una estructura de selección principal o maestra podría ser tan compleja y continuar de forma indefinida. Este tipo de estructuras se utilizan en problemas de mayor grado de complejidad y en circunstancias en las cuales se deban de evaluar precondiciones para determinar qué flujo seguir. Es importante mencionar que, a medida que se agreguen niveles de anidamiento, la estructura del algoritmo, así como la codificación del mismo, implicará cometer errores, por lo cual se recomienda siempre validar todos los flujos del programa y comprobar las salidas de estos.

5.3.2.4. Selección múltiple (EN CASO DE)

Como se ha observado, en las estructuras de selección anteriores se pueden decidir los flujos de control que se requiere y hasta tener sub-flujos o estructuras dentro de estructuras para condiciones específicas, al momento se han visto estructuras en las cuales solo se tienen como máximo dos flujos o la selección de dos posibles elecciones; sin embargo, en ocasiones será necesario contar con más de dos opciones o alternativas de flujos. Por ejemplo, si un usuario requiere elegir entre 3 o más opciones, como escoger un piso en un elevador, elegir un refresco en una máquina expendedora o escoger un camino dentro de los n-posibles en un juego, el grado opciones y complejidad aumenta y las opciones con las estructuras de selección simple, doble y anidadas en ambos casos se ven limitadas. Si bien podría resolverse bajo estas estructuras, la complejidad y el tamaño del algoritmo y del código fuente podría ser demasiado grande y poco manejable además de causar problemas de legibilidad.

Para este tipo de problemas y condiciones se puede hacer uso de la estructura de control de decisión múltiple, en la cual se evaluará una expresión o condición que podría tomar una decisión dentro de un universo de valores distintos desde 0,1,2,3… hasta n.

Dependiendo de la opción que se elija dentro de la condición, se realizarán las instrucciones o acciones indicadas dentro de ese flujo seleccionado.

A continuación, se muestran algunos modelos de la implementación de la estructura múltiple:

```
Modelo 1:

según_sea expresion (E) hacer
     el: accion S11
         accion S12
         .
         .
         accion S1a
     e2: accion S21
         accion S22
         .
         .
         accion S2b
     .
     .
     en: accion S31
         accion S32
         .
         .
         accion S3p
     si-no
         accion Sx
fin_según

Modelo 2 (simplificado):

según E hacer
       .
       .
       .
fin_según
```

```
Modelo 3 (simplificado):

opción E de
  .
  .
fin_opción

Modelo 4 (simplificado):

caso_de E hacer
  .
  .
  .
fin_caso

Modelo 5 (simplificado):

si E es n hacer
  .
  .
  .
fin_si
```

Figura 5.10. Modelos de la implementación de la estructura múltiple. Fuente: (Joyanes, 2008, p. 137).

5.3.3.
Iterativas

Otro tipo de estructuras de control a revisar son las iterativas, las cuales tienen una relación directa con el funcionamiento de las tareas de una computadora ya que la ejecución repetida de una o varias instrucciones, aplicaciones y software tienen dentro de su código el repetir una actividad una y otra vez hasta el cumplimiento de una condición, postcondición o la interacción del usuario.

Dependiendo del tipo de problema que tengamos que resolver, en ocasiones una solución puede plantear una solución en la cual una serie de instrucciones debe de repetirse n-veces hasta lograr alcanzar un valor, cumplir una condición o esperar una respuesta especifica del usuario o de otro programa. Un ejemplo de este tipo de algoritmos sería la lectura de una serie de números dados por el usuario, esto con el fin de llenar una lista, una matriz, una tabla, ante dicha situación el programa solicitará los datos una y otra vez hasta que el usuario indique que se ha completado la tarea o que el usuario haya terminado de ingresar los datos en una lista predefinida.

Las estructuras iterativas repiten una secuencia de instrucciones un número determinado de veces, a este tipo de eventos se les denomina bucles o ciclos. Para ejemplificar esto, se plantea la siguiente cuestión: un programa tiene como objetivo realizar la suma a partir de una lista de números, los cuales serán ingresados por el usuario desde el teclado, esto podría aplicar para registrar las calificaciones de un grupo de alumnos. Como primer punto a considerar es la lectura de los números y añadir dichos valores a una variable denominada SUMA que contenga las sumas parciales sucesivas. En este caso, la variable Suma parte de un valor en cero y a medida que se ingresan los valores o las calificaciones esta variable incrementa su valor cada vez que se ingresa un nuevo valor.

A continuación, se presenta el algoritmo en pseudocódigo para la solución de este problema:

```
Algoritmo suma sucesiva
Start //inicio del programa
Var
Entero: SUMA, NUMERO
SUMA = 0
Leer (NUMERO)
SUMA = SUMA + NUMERO
Leer (NUMERO)
SUMA = SUMA + NUMERO
Leer (NUMERO)
SUMA = SUMA + NUMERO
End // fin del programa
```

Como se observa en el pseudocódigo anterior, la SUMA y lectura del NUMERO ingresado por el usuario podría ser tan grande como sea necesario, en este caso se observan las sentencias que se repiten varias veces (Lectura y SUMA), este tipo de sentencias son bucles o ciclos.

Para definir y crear un bucle de forma correcta deben plantearse las siguientes preguntas:

¿qué contiene el bucle?, ¿qué acciones se realizarán dentro de él?, ¿cuántas veces debe repetirse?, ¿en qué condición deberá finalizar?

En el ejemplo anterior uno de los primeros aspectos a considerar es conocer el número de valores a ingresar o el número de datos de lista, ya que nos indicará cuántos números se sumarán y cuántas veces se solicitará al usuario ingresar y leer los números. Esto indicaría una forma o condición para finalizar el bucle, la primera, el bucle finalizaría cuando se terminen de ingresar todos los datos de la lista, otra forma sería preguntar al usuario si desea finalizar el ingreso de datos.

De esta forma se puede utilizar una variable denominada TOTAL, la cual inicia con la cantidad de números a ingresar, lo cual definirá nuestra cantidad de repeticiones o interacciones; después, se deberá establecer el procedimiento para disminuir o decrementar el valor de la variable TOTAL cada vez que se ingrese un valor y el bucle se ejecute, planteado de otra manera sería: TOTAL = TOTAL – 1.

Otra alternativa sería establecer la variable en cero e ir incrementando el valor en uno cada vez que se ingrese un número y se ejecute el bucle, donde TOTAL = TOTAL + 1, esta acción se realiza hasta que TOTAL sea igual al número de valores.

5.3.3.1. Iteración con precondición (MIENTRAS)

Dentro de las estructuras de control iterativas se encuentran varios tipos, la primera es la estructura iterativa **mientras** o while, en este tipo de iteración el cuerpo del bucle o ciclo se repite mientas se esté cumpliendo la condición definida.

Cuando se ejecuta la instrucción **mientras**, el primer aspecto que sucede es la evaluación de la condición (que por lo regular son expresiones de tipo booleano), si dicha evaluación resulta ser verdadera, la siguiente acción sería ejecutar las acciones o instrucciones definidas dentro del bucle, una vez finalizada la ejecución de las acciones, se vuelve a realizar una evaluación de la condición. Este proceso se realiza n-veces mientras la condición siga siendo verdadera. Por otra parte, si la evaluación resultara ser falsa, el bucle no se ejecuta si es la primera vez o es finalizada, si ya se ha ejecutado varias veces.

A continuación, se presenta un ejemplo con su representación en pseudocódigo y diagrama de flujo:

Objetivo: Leer desde teclado un número que definirá la cantidad de números a ingresar por el usuario y después se realizará la suma de todos los números:

Pseudocódigo

```
Algoritmo suma_numeros
Start //inicio del programa
Var
Entero: N, TOTAL
Real: NUMERO, SUMA
Leer (N)
{leer NUMERO TOTAL N}
TOTAL = N
        SUMA = 0
        Mientras TOTAL > 0 hacer
Leer (NUMERO)
SUMA = SUMA + NUMERO
TOTAL = TOTAL – 1
Fin mientras
Escribir (‘La suma de los’ N ‘números es:’
SUMA)
End // fin del programa
```

Diagrama de Flujo

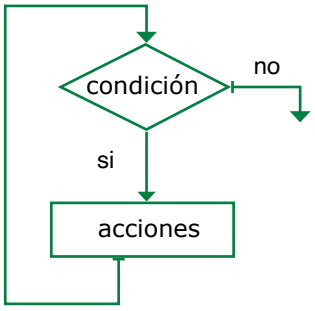


Figura 5.11. Diagrama de flujo. Fuente: (Joyanes, 2008, p. 161).

5.3.3.2. Iteración con postcondición (HACER MIENTRAS)

La segunda estructura de control iterativa a revisar es **mientras** o **while** con postcondición, el bucle visto en el subtema anterior está definido con precondición de evaluación para realizar sus acciones, lo que significa que antes de realizar cualquier tipo de iteración, se deberá realizar la evaluación de condición previamente, a este tipo de bucles se les denomina bucles controlados por la entrada.

En este nuevo caso, la función del bucle es básicamente la misma, solo con la diferencia del momento en el cual se realiza la evaluación de la condición, la cual será al final de ejecutar las acciones del bucle, esto implica que, sin importar el tipo de circunstancias las acciones del bucle se ejecutarán una vez, después se realiza la evaluación de la condición para validar si el bucle se repite o se realiza la finalización de este.

En este caso, se observa que las condiciones de la solución y del algoritmo requieren que se ejecuten las instrucciones o acciones por lo menos una vez, y ya que ha sucedido esto, se evalúa si es necesario realizar estas acciones o se termina esta sección de código.

A este tipo de bucles, en el cual se ejecutan una vez las instrucciones y después se realiza la evaluación, se les denomina bucles controlados por la salida, ya que la salida indica si el bucle continúa o finaliza.

En algunos lenguajes de programación como C, JAVA o Python este tipo de estructuras se realizan con el bucle ***do-while***.

A nivel de código fuente, sin importar el lenguaje de programación a utilizar, en el bucle ***do-while*** la primera sección del código (do) contendrá las acciones o instrucciones

que se ejecutarán al menos una vez, cuando finaliza esta sección de código se realiza la evaluación de la condición (***while***), si la condición resulta verdadera se seguirá realizando tantas veces sea necesario y la condición sea falsa.

A continuación, podrás ver un ejemplo con su representación en pseudocódigo y su análisis correspondiente:

Objetivo: Obtener un algoritmo que lea un número (por ejemplo, 198) y obtenga el número inverso (por ejemplo, 891). El siguiente ejemplo es presentado por Joyanes (2006, p. 167):

Pseudocódigo

```
algoritmo invertirnummero
START//inicio del programa
var
Entero: num, digitoSig
num = 198
escribir ('Número: = ', num)
escribir ('Número en orden inverso: ')
hacer
   digitoSig = num MOD 10
escribir(digitoSig)
num = num DIV 10
   mientras num > 0
END// fin del programa
```

En el ejemplo anterior, se observa que en cada iteración se obtiene el digito más a la derecha, esto derivado de que el resto de la división del valor del número (num) por 10. Así en la primera iteración digitoSig vale 8, ya que es el resto de la división entera de 198 entre 10 (cociente 19 y resto 8). Se visualiza el valor 8. A continuación se divide 198 entre 10 y se toma el cociente entero 19, que se asigna a la variable num.

En la siguiente iteración se divide 19 por 10 (cociente entero 1, resto 9) y se visualiza, por consiguiente, el valor del resto, ***digitoSig***, es decir el dígito 9; a continuación, se divide 19 por 10 y se toma el cociente entero, es decir, 1. En la tercera y última iteración se divide 1 por 10 y se toma el resto (***digitoSig***) que es el dígito 1. Se visualiza el dígito 1 a continuación de 89 y como resultado final aparece 891. A continuación, se efectúa la división de nuevo por 10 y entonces el cociente entero es 0 que se asigna a ***num*** que, al no ser ya mayor que cero, hace que se termine el bucle y el algoritmo correspondiente.

5.3.3.3. Iteración para número determinado de veces (DESDE HASTA)

Otro tipo de estructura de control iterativa es **desde hasta** o ***for***, en la cual se define cuantas veces se realizarán las iteraciones, en estos casos se conoce desde el inicio cuántas veces será necesario ejecutar las acciones o instrucciones dentro del bucle, el número de iteraciones será fijo.

En este tipo de estructura de control iterativa, las acciones a ejecutar dentro del cuerpo del bucle estarán definidas por una constante o variable determinada antes de la ejecución del bucle y de modo automático este tipo de estructura controla la cantidad de iteraciones sin que puedan ser modificada de forma interactiva con el usuario o las condiciones del programa.

A continuación, se muestran los ejemplos de representación en pseudocódigo y diagrama de flujo de la condición for, el ejemplo es presentado por Joyanes:

Pseudocódigo:

desde v ← vi hasta vf [*incremento* incr] hacer

<acciones>

.

.

.

.fin_desde

v: variable indice

vi, vf: valores inicial y final de la variable

Diagrama de flujo:

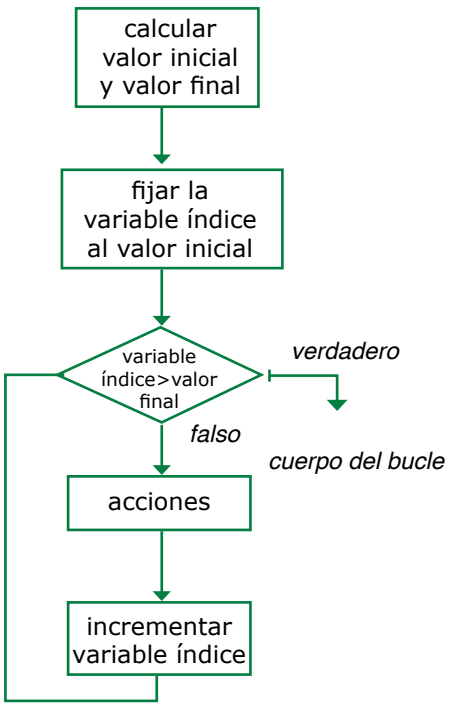


Figura 5.12. Diagrama de flujo. Fuente: (Joyanes, 2006, p.171).

5.3.3.4.1.

Contadores

Este tipo de estructura también se caracteriza por iterar instrucciones en función de una condición que debe cumplirse en un momento conocido, está representada por la instrucción para (*for*). En esta estructura se evalúa el valor de una variable a la que se asigna un valor conocido al inicio de las iteraciones; este valor sufre incrementos o decrementos en cada iteración y suspende la ejecución de las instrucciones asociadas una vez que se alcanza el valor esperado.

En algunos lenguajes, se puede definir el incremento que tendrá la variable; sin embargo, se recomienda que el incremento siempre sea con la unidad.

Pseudocódigo que representa la estructura:

Para <variable> = <valor inicial> hasta <valor tope> [paso <incremento>] hacer

Instruccion1

Instruccion2

...

Instruccion n

Fin para <variable>

Se ha podido observar entre los símbolos [ ] la opción que existe para efectuar el incremento a la variable con un valor distinto de la unidad.

A continuación, se muestra el diagrama asociado a esta estructura:

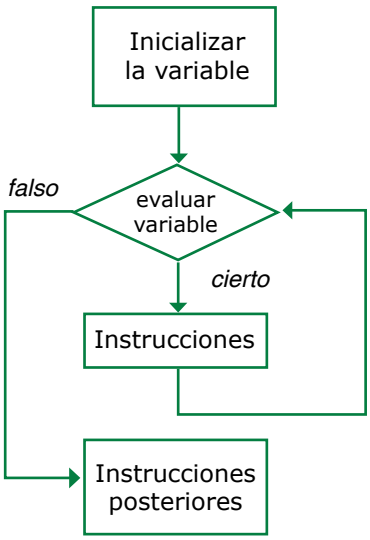


Figura 5.13. Diagrama estructura.

Para reforzar el aprendizjaje, se utilizará el ejemplo anterior implementado con la función ***for***:

suma=0 (inicializamos la variable suma)

para n=0 hasta n=5, [n+1] hacer (indicamos que comenzaremos en n=0 y repetiremos hasta que n alcance el valor de 5, con un incremento del valor de n en 1)

Suma=suma+n

fin para n

5.3.3.4.2. Acumuladores

Los acumuladores son variables que tienen como propósito almacenar valores incrementales o decrementales a lo largo de la ejecución del algoritmo; utilizan la asignación recursiva de valores para no perder su valor anterior.

Su misión es arrastrar un valor que se va modificando con la aplicación de diversas operaciones y cuyos valores intermedios, así como el final, son importantes para el resultado global del algoritmo. Este tipo de variables generalmente almacenan el valor de la solución arrojada por el algoritmo.

La asignación recursiva de valor a este tipo de variables se ejemplifica a continuación:

<variable> = <variable> + <incremento>

En los ejemplos anteriores, podemos observar el uso de los acumuladores en la variable *n*.

Condicionales

Esta clase de estructura se utiliza para ejecutar selectivamente secciones de código de acuerdo con una condición definida. Y solo tiene dos opciones: si la condición se cumple, se ejecuta una sección de código; si no, se ejecuta otra sección, aunque esta parte puede omitirse.

Es importante mencionar que se pueden anidar tantas condiciones como lo permita el lenguaje de programación en el que se implemente el programa.

El pseudocódigo básico que representa la estructura if es el siguiente:

si <condición> entonces
Instruccion1
Instruccion n
[si no
Instruccion 3
Instruccion n]
Fin si

Dentro del bloque de instrucciones que se definen en las opciones de la estructura, sí se pueden insertar otras estructuras condicionales anidadas. Entre los símbolos [ ] se encuentra la parte opcional de la estructura.

El diagrama asociado a esta estructura queda de la siguiente manera:

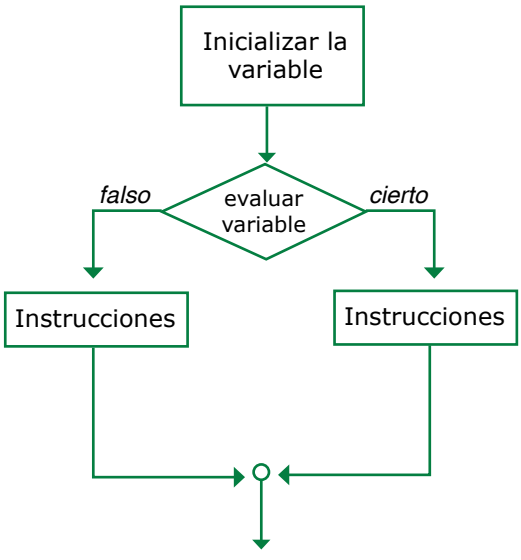


Figura 5.14. Cumplimiento de condicionales.

Ahora, se empleará la estructura if para solucionar el mismo ejemplo que se ha estado manejando.

suma=0 (inicializamos la variable suma)
n=0 (inicializamos el acumulador)
si n=5 entonces (establecemos la condición)
imprime suma
de lo contrario (establecemos la alternativa de la condición)
Suma=suma+n
n=n+1
fin si

En este caso, la condición se cumplirá hasta que n alcance el valor de 5; mientras no lo haga, realizará la suma de los valores hasta alcanzar la condición.

5.3.3.4.3. Banderas

Otro tipo de variable que podemos utilizar, son aquellas denominadas banderas, las cuales son de tipo lógico o booleano, lo que quiere decir que su funcionamiento estará dado en un valor verdadero o falso.

Dentro de un programa estas variables nos ayudarán a indicar qué acción realizar o qué flujo seguir, dependiendo si cumplen una condición verdadera o falsa. Normalmente se utilizan para salir de un bucle.

A continuación, se muestra algunos ejemplos de la utilización de este tipo de variables y cómo finalizar bucles:

Objetivo: Desplegar un mensaje de finalización al usuario cuando haya ingresado 10 números.

Pseudocódigo:

```
Algoritmo desplegar_mensaje
Start //inicio del programa
Var
Entero: N
Real: NUMERO, REGISTRO
Booleano: BANDERA
Leer (N)
REGISTRO = 0
{leer NUMERO, N}
REGISTRO = REGISTRO + 1
Si REGISTRO = 10
  BANDERA = true
Finaliza
Despliega Mensaje (´A ingresado 10 números´)
De lo contrario
  BANDERA = false
Despliega Mensaje (´vuelve a ingresar otro número´)
End // fin del programa
```

Como se observa en el ejemplo anterior, las variables de tipo bandera nos permiten terminar un flujo o una sentencia de control iterativa a través de la evaluación y asignación que se indique dentro del código. Este tipo de variables podrían utilizarse como alternativas para finalizar bucles.

Otro ejemplo sería al utilizar menús en los cuales se indicaría un valor false a la bandera para poder finalizar el menú.

Objetivo: Mostrar un programa que despliegue un menú con varias opciones.

Pseudocódigo:

```
Algoritmo menú_ejemplo
Start //inicio del programa
Escribir “---MENU---”
Escribir “Elige una opción”
Escribir “1. Desplegar fecha actual”
Escribir “2. Desplegar hora actual”
Escribir “3. Salir”
leer opcion_menu
Segun opcion_menu Hacer
1:
Desplegar “La fecha actual es:” llamada al sistema para obtener fecha
Fin
2:
Desplegar “La fecha hora es:” llamada al sistema para obtener hora
Fin
3:
Desplegar “ha decidido salir del programa”
Salir = verdadero
Fin
Fin Mientras // Fin de menú
End // fin de programa
```

5.3.3.4.4. Centinelas

Las variables de tipo de centinelas, al igual que las variables banderas, son de tipo booleano o de tipo lógico, la diferencia entre ambas es el momento de asignación del valor, para una variable de tipo bandera se asigna su valor verdadero o falso después de realizar o ejecutar una serie de acciones o instrucciones. Para las variables de tipo centinela, la asignación se realiza previamente y después de realizar un conjunto de instrucciones se realizará el cambio correspondiente entre verdadero a falso o de forma inversa.

A continuación, se presenta un ejemplo de la utilización de variables centinelas:

Objetivo: Desplegar un mensaje al usuario si se encuentra un alumno reprobado o si todos los alumnos aprobaron.

Pseudocódigo:

```
Algoritmo desplegar_mensaje_alumnos
Start //inicio del programa
Var
Real: NOTAS, CONTADOR
Booleano: CENTINELA
NOTAS = { 5.65, 8.75, 6.23, 4.3, 10, 7.55}
CENTINELA = false
CONTADOR=0
Mientras CONTADOR < LEN (NOTAS) y
NOT CENTINELA
  Si NOTAS < = 5
        CENTINELA = true
  CONTADOR = CONTADOR +1
Si CENTINELA = true
  Desplegar (“Hay un alumno reprobado”)
De lo contrario
  Desplegar (“Todos los alumnos aproba-
ron”)
End // fin del programa
```

En el ejemplo anterior, se observa la asignación del valor de la variable tipo CENTINELA desde el inicio del algoritmo y al avanzar las instrucciones o acciones del programa este podría cambiar.

RESUMEN DE LA UNIDAD



En esta unidad se mostraron las diferentes estrategias, así como las herramientas de diseño de algoritmos, considerando en todo momento la mejor alternativa dependiendo del problema planteado o solución a la que se necesite llegar.

Como se mencionó, en cada problema que tengamos que resolver se presentarán circunstancias y condiciones a considerar para elegir la mejor estrategia de diseño de algoritmos, esto representa una serie de ventajas en un proyecto de desarrollo de software, desde mejorar el rendimiento del programa, ya que estará basado en la solución más óptima, lo cual tendrá un impacto directo en tiempos de ejecución, soluciones alineadas a los objetivos del proyecto, mantenimiento del software, etcétera.

También se presentaron las estructuras básicas de control que utilizan los lenguajes de programación, lo cual es una representación y reflejo de lo expresado en un diagrama de flujo y/o un pseudocódigo. En las estructuras de control básicas vistas en la unidad se presentó la forma en la cual un problema se puede expresar bajo ciertas condiciones y cómo expresar desde sentencias secuenciales o lineales, así como expresar evaluaciones, repetir o hacer iterativas ciertos fragmentos del código y algoritmo hasta cumplir una condición deseada, considerando alterar el flujo de un programa bajo las condiciones que podrían presentar un problema y obtener la mejor solución.

ACTIVIDADES DE APRENDIZAJE

Unidad 5, actividad inicial. ***Adjuntar archivo.*** A partir del estudio de del contenido de la unidad, elabora un mapa conceptual u organizador gráfico sobre los temas vistos.

1. **Unidad 5, actividad 1.** ***Adjuntar archivo.*** Después de haber dado lectura a la unidad, realiza lo siguiente: Investiga 3 ejemplos en los cuales puedes aplicar las estrategias de diseño de algoritmos; **Divide y Vencerás, Recursividad, Algoritmos voraces** y ***Backtracking***.
2. **Unidad 5, actividad 2.** ***Adjuntar archivo.*** Después de haber dado lectura a la unidad, realiza lo siguiente: A partir del siguiente problema realiza su representación en pseudocódigo. Se desean convertir las calificaciones alfabéticas A, B, C, D, E y F a calificaciones numéricas 4, 5, 6, 7, 8 y 9 respectivamente. Diseña el algoritmo para poder realizar la conversión.
3. **Unidad 5, actividad 3.** ***Adjuntar archivo.*** Después de haber dado lectura a la unidad, realiza lo siguiente: A partir del siguiente problema realiza su representación en diagrama de flujo.

   Diseña un algoritmo que escriba los nombres de los días de la semana en función del valor de una variable DIA introducida por teclado. Los días de la semana son 7; por consiguiente, el rango de valores de DIA será 1 .. 7, y en caso de que DIA tome un valor fuera de este rango se deberá producir un mensaje de error advirtiendo la situación anómala.
4. **Unidad 5, actividad complementaria 1.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.
5. **Unidad 5, actividad complementaria 2.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.
6. **Unidad 5, actividad complementaria 3.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

ACTIVIDAD INTEGRADORA

Adjuntar archivo.

En esta unidad aprendiste sobre el diseño de soluciones algorítmicas de problemas con el apoyo de los elementos sintácticos de un lenguaje de programación. A partir de los conocimientos adquiridos, desarrolla la siguiente actividad integradora:

Realiza un organizador gráfico en el que destaques las características de cada estrategia de diseño de algoritmos.

CUESTIONARIO DE REFORZAMIENTO

Adjuntar archivo.

Contesta las siguientes preguntas.

1. Describe bajo qué tipo de problemas utilizarías la estrategia de diseño de algoritmos *Backtracking*.
2. Describe bajo qué tipo de problemas utilizarías la estrategia de diseño de algoritmos Divide y vencerás.
3. Describe bajo qué tipo de problemas utilizarías la estrategia de diseño de algoritmos Recursividad.
4. ¿En qué condiciones se utilizan las estructuras de control iterativas?
5. ¿En qué condiciones se utilizan las estructuras de control selectivas?
6. Explica con tus palabras el funcionamiento de la estructura de control DESDE HASTA.
7. Explica con tus palabras el funcionamiento de la estructura de control SI ENTONCES DE LO CONTRARIO.
8. Explica con tus palabras el funcionamiento de la estructura de control HACER MIENTRAS.
9. Explica con tus palabras el funcionamiento de la estructura de control EN CASO DE.
10. Menciona las ventajas de utilizar pseudocódigos como herramienta de diseño de algoritmos.

AUTOEVALUACIÓN

I. Responde verdadero (V) o falso (F) según corresponda.

1. El bucle mientras evalúa la expresión al final del bucle de repetición

2. Las rutinas en las cuales el texto contiene al menos dos llamadas recursivas se denominan algoritmos de divide y vencerás.

3. El diseño de **algoritmos *Backtracking*** no se utiliza como estrategia para solucionar problemas.

4. Los problemas que deben satisfacer un determinado tipo de restricciones son completos, donde el orden de los elementos de la solución no importa, se resuelven mejor con la estrategia de diseño de algoritmos Divide y vencerás.

5. La principal diferencia entre un diagrama de flujo y un diagrama estructurado N/S, es la omisión de las flechas de unión y las cajas son contiguas

II. Correlaciona la respuesta correcta con la sentencia adecuada. Elige el inciso que responda correctamente.

a) Desde hasta	b) En caso de	c) Si entonces	d) Mientras	e) Hacer mientras

1 Estructura alternativa que ejecuta una determinada acción cuando se cumple una determinada condición ◯

2. La estructura ejecuta las acciones del cuerpo del bucle un número específico de veces y de modo automático controla el número de iteraciones o pasos a través del cuerpo del bucle. ◯

3. Es el bucle que evalúa la expresión al comienzo del bucle de repetición; siempre se utiliza para crear bucle pre-test. ◯

4. Es la estructura repetitiva en que el cuerpo del bucle se repite mientras se cumple una determinada condición. ◯

5. Con frecuencia en la práctica es necesario que existan más de dos elecciones posibles. ◯

RESPUESTAS DE AUTOEVALUACIÓN

En este apartado encontrarás las respuestas al examen por unidad.

UNIDAD 5	
I. Solución	
1.	F
2.	V
3.	F
4.	F
5.	V

UNIDAD 5	
II. Solución	
1.	c
2.	a
3.	e
4.	d
5.	b

REFERENCIAS

Bisbal, R. (2009). *Manual de Algorítmica. Recursividad, complejidad y diseño de algoritmos*. Editorial UOC.

Cairó, B. (2006). *Fundamentos de programación. Piensa en C.* Pearson.

Joyanes, L. (2006). *Programación en C + +.* McGraw Hill.

Joyanes, L. (2008). *Fundamentos de programación. Algoritmos, estructuras de datos y objetos*. McGraw Hill.

UNIDAD 6

DESARROLLO DE PROGRAMAS

OBJETIVO PARTICULAR

Al finalizar la unidad, el alumnado traducirá las soluciones algorítmicas a un lenguaje de programación.

TEMARIO DETALLADO

(16 horas)

6. Desarrollo de programas
6.1. Programas secuenciales
6.2. Programas de selección simple, doble, anidada y múltiple
6.3. Programas iterativos

ACTIVIDAD DIAGNÓSTICA



Adjuntar archivo

Con base a lo visto en la unidad anterior, realiza una tabla comparativa destacando las características y diferencias entre las diversas estrategias de diseño de algoritmos. Por último, desarrolla un ejemplo en pseudocódigo con cada estrategia vista.

INTRODUCCIÓN



En esta unidad se presentarán los elementos básicos para escribir un programa de software y la implementación de las estructuras básicas de programación vistas en la unidad anterior. Se mostrarán las diferentes formas de implementar las estructuras de control en diferentes lenguajes, así como las diferencias que existen dependiendo de las características de estos, como lo es su nivel, paradigma o enfoque.

Asimismo, podrás observar la transformación que sucede desde el pseudocódigo realizado a un código fuente o codificación en lenguaje de programación, lo cual ampliará tus conocimientos y tendrás una visión más amplia de cómo una sentencia en un pseudocódigo, un flujo dentro de un diagrama, se representa en un lenguaje similar al humano, pero entendible para una computadora o equipo de cómputo.

El lenguaje de programación a utilizar dentro de la unidad será Python, el cual es un lenguaje de alto nivel y multiplataforma, lo que permite su ejecución en una amplia gama de sistemas operativos, de código abierto multipropósito, lo cual permite utilizarlo para implementar prácticamente cualquier tipo de solución a todo tipo de problemas.

Al final de la unidad serás capaz de entender las características y diferencias, a nivel de código fuente, entre las estructuras de control, partiendo desde sentencias secuenciales hasta estructuras iterativas, selección con sus variantes y las posibles opciones de implementación.

6.1.
Programas secuenciales

En las unidades anteriores, se ha observado cómo realizar la conversión de un algoritmo a un pseudocódigo o a un diagrama de flujo que es una actividad clave dentro del desarrollo de la solución de un problema, además de ser parte de ciclo de vida de desarrollo de software.

La actividad de transformar un pseudocódigo a código fuente es crucial en muchos sentidos, en dicha transformación se debe poner atención en que, sin importar la herramienta de diseño y su representación, esta debe ser lo más clara y estructurada posible, además de que la lectura de cualquier pseudocódigo deberá permitir que todos los interesados en las fases de análisis, diseño y desarrollo sean capaces de entender y así facilitar su posterior codificación a un lenguaje de programación.

Como se observó en las unidades anteriores, los algoritmos presentados en pseudocódigo son y deben ser escritos con una similitud a un lenguaje de programación, si bien estos no incluirán palabras reservadas, semánticas y sintaxis de un lenguaje de programación específico, deben tener un parecido para interpretar las estructuras de control.



Dentro de las características que se han visto de un algoritmo se han destacado que son una serie de pasos para resolver un problema y/o alcanzar un objetivo, son finitos, lo cual indica que tienen un principio y un fin, deben dar siempre el mismo resultado. Además de las características mencionadas, a nivel de estructura está formado por dos componentes: la cabecera es normalmente representada con una instrucción y/o acción sencilla que indica el inicio del algoritmo, seguido del nombre del algoritmo o del programa. El bloque de algoritmo está constituido por todas las instrucciones y/o acciones del programa, este componente se subdivide en dos secciones, la primera son las acciones de declaraciones como declaración de variables y constantes, la segunda sección son las acciones ejecutables o instrucciones que realizan un tarea o función en específico (Alcaraz, 2019, p. 21).

Como se muestra en la siguiente figura.

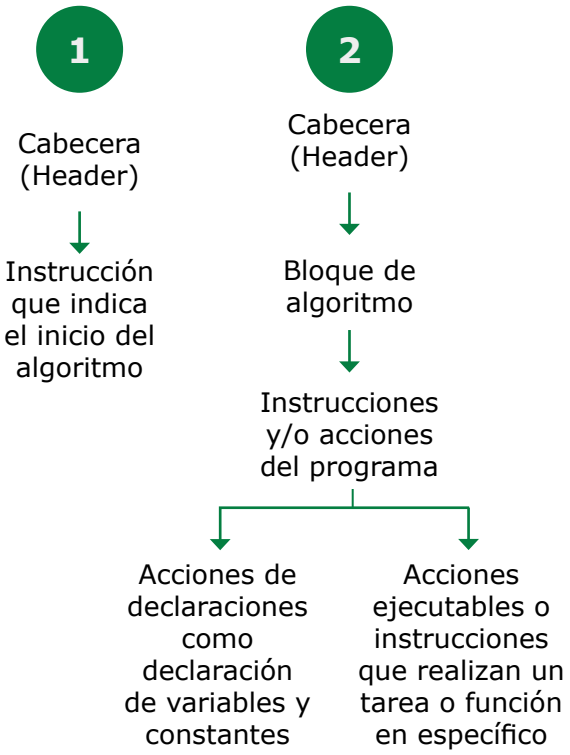


Figura 6.1. Componentes de la estructura.

Tanto los programas de *software* como los algoritmos deben iniciar e indicar en su cabecera el identificador o nombre con el cual se define el mismo, esto normalmente se definirá con las palabras reservadas propias del lenguaje de programación que se esté utilizando. En el caso concreto para un pseudocódigo se puede presentar de la siguiente forma:

algoritmo Programa1

Después de realizar la cabecera, se deberá colocar la sección de declaraciones del componente "bloque de algoritmo", en esta sección se escriben todas las variables y constantes que serán utilizadas dentro del programa. Estas variables y constantes serán definidas por sus nombres, así como el tipo de variable (numérica, carácter, cadena de texto, booleano). Dependiendo de la estructura y reglas definidas por el equipo, se puede especificar esta sección con la palabra "var" y después listar todas las variables y constantes a utilizar.

Otra forma sería seccionar con doble diagonal indicando en forma de comentario y en cada variable y constante iniciar con la palabra "var". A continuación, se muestra un ejemplo de la sección de declaración:

```
var //inicio de la sección
entero: id_empleado // id de empleado
real: horas, sueldo, impuestos // valores
reales para horas, sueldo e impuestos
```

Para la sección de constantes se puede declarar de la siguiente forma:

```
const //sección de constantes
iva=.16 //declaración de la tasa de impuestos
IVA
sueldo_base= 164 //declaración del sueldo
base diario
```

Una observación importante es que las constantes definidas no podrán cambiar durante la ejecución del programa, a comparación de las variables las cuales pueden tomar distintos valores conforme se ejecute el programa.

Como se observó en las secciones anteriores, además de realizar la declaración de las acciones, variables y constantes, se realizan comentarios sobre cada sección o cada línea, lo cual será definido por los analistas y programadores, estos comentarios son importantes ya que forman parte de la documentación del proyecto y principalmente del programa. Esta documentación se deberá

realizar tanto de forma **interna** (dentro del código fuente), como de forma **externa**, esto facilitará la compresión del programa, así como su posterior mantenimiento, la documentación deberá realizarse desde el inicio del proyecto y deberá incluir desde las sesiones de inicio de entendimiento del proyecto, así como las fases de análisis, diseño, codificación, verificación, etcétera.

Si bien parte de la documentación que se ha visto y mencionado está relacionada directamente con cuestiones técnicas, esta no deberá ser limitada a este rubro. Por otra parte, la documentación administrativa o documentación a nivel de negocio representa una parte medular para la compresión tanto de requerimientos funcionales como no funcionales que en fases de mantenimiento e incidencias serán clave para intervenir (Moreno, 2014, p. 72).

Una vez identificadas las diferentes secciones de un programa se mostrará un ejemplo de ejecución secuencial, en el cual solamente se ejecutará línea tras línea del código fuente.

En el siguiente ejemplo, se observa un de programa que realiza la suma de dos números y muestra el resultado al usuario:

```
num1 = 15// declaración de primera constante
num2 = 12// declaración de segunda constante

sum = num1 + num2

print("Sum of {0} and {1} is {2}" .format(num1,
num2, sum))
```

En el programa anterior se puede observar la declaración de dos constantes y después la suma aritmética de dichas constantes, por último, la función “print” nos da la posibilidad de presentar el resultado en pantalla.

6.2. Programas de selección simple, doble, anidada y múltiple

En la mayoría de software de programación estructurada, como C, o lenguajes de programación multiparadigma, orientado a eventos, orientado a objetos, etc., como Python o Java, las instrucciones y/o acciones se ejecutan en el mismo orden en que aparecen en el código fuente. En estos casos, las instrucciones normalmente se ejecutan una sola vez, como se observó en los ejemplos del tema anterior o en programas de software con solo estructuras de control secuencial; sin embargo, en la práctica no hay problemas que se solucionen con estructuras tan sencillas. En particular, este tipo de programas no realizan ningún tipo de validación, selección, comprobación o instrucciones que requieran un nivel más complejo de funcionalidad, lo que limita considerablemente su funcionalidad y uso.

En la práctica un programa de software requiere que se realicen comprobaciones lógicas en algunos puntos claves del código, este tipo de eventos se conocen como ejecuciones funcionales o ejecuciones de selección.

El uso de este tipo de estructuras de control nos permite proporcionar soluciones a problemas más complejos, además de mejorar en muchos casos la funcionalidad y rendimiento de los programas.

Antes de realizar la escritura de código fuente utilizando las estructuras de control de selección, se deben revisar algunos conceptos relacionados con las expresiones lógicas que nos indicarán si un evento es verdadero o falso.

Gottfried (2005) menciona y resume 4 operadores relacionales, los cuales son: "<. <=, >, >= y los dos operadores de igualdad, == y !=", estos operadores definirán la regla o condición que deseamos evaluar para decidir si una sección del código realiza una acción o realiza otro tipo de acción (p. 153).

A continuación, se muestran algunos ejemplos de expresiones lógicas:

```
contador <= 100 // la expresión es verdade-
ra cuando la variable contadora sea menor o
igual a cien
calificación > 6 // la expresión es verdadera
cuando la variable calificación sea mayor a 6
correcto == 10 // la expresión es verdadera
cuando la variable correcto sea igual a 10
opcion != 'x' // la expresión es verdadera
cuando la variable sea diferente al valor x,
esto expresado como un carácter.
```

Como puedes ver, en los tres primeros ejemplos los valores utilizados en las expresiones lógicas se utilizan valores numéricos; sin embargo, podemos realizar evaluaciones lógicas utilizando caracteres o cadenas, en el último ejemplo se realiza una evaluación lógica con un carácter, solo haciendo uso de comillas simples para indicar que se está evaluando un valor no numérico, este tipo de sintaxis podría variar entre los diferentes lenguajes de programación y sus reglas.

Ejemplo de programa con If

```
Si semáforo está en verde, cruzar la
calle. Si no, esperar.
if semaforo == verde:
    print "Cruzar la calle"
else:
    print "Esperar"
```

Ejemplo de programa con If else anidado

```
Si gasto hasta $100, pago con dinero
en efectivo. Si gasto más de $100
pero menos de $300, pago con tarjeta
de débito. Si no, pago con tarjeta de
crédito.
if compra <= 100:
    print "Pago en efectivo"
elif compra > 100 and compra < 300:
    print "Pago con tarjeta de débito"
else:
    print "Pago con tarjeta de crédito"
```

Ejemplo de programa con switch

Este bucle, se encarga de ejecutar una misma acción **"mientras que"** una determinada condición se cumpla. Ejemplo: Mientras que año sea menor o igual a 2022, imprimir la frase *"Informes del Año".*

```
# -*- coding: utf-8 -*-
anio = 2001
while anio <= 2022:
    print "Informes del Año", str(anio)
    anio += 1
```

Como sugerencia general se recomienda que dentro del código fuente se realicen los comentarios necesarios para explicar brevemente el objetivo de este tipo de estructuras de control, lo cual permitirá que, en futuros cambios, incidencias y/o mantenimientos se realicen de forma más ágil al contar con detalles de funciones.

6.3. Programas iterativos

Como se ha observado en el tema anterior, el uso de estructuras de control más complejas, como las estructuras de control selectivas con todas sus variantes, permiten realizar y codificar soluciones a problemas complejos; sin embargo, en ocasiones estas estructuras podrían ser poco eficientes ante diferentes escenarios. Tal es el caso de funciones en las cuales una acción o conjunto de instrucciones deban de realizar o repetirse en más de una ocasión. Ante este tipo de situaciones se pueden utilizar las estructuras de control **iterativas** y hacer el uso de bucles que son estructuras que nos permiten ejecutar de manera definida un conjunto de instrucciones hasta que se cumpla una condición de salida, la cual puede ser indicada por el usuario o definida dentro de la misma funcionalidad del programa de *software*.

Dentro de los diferentes lenguajes de programación, contamos con varias instrucciones que nos permiten definir bucles en los cuales un grupo de instrucciones se ejecutan de forma iterativa, hasta satisfacer una condición.

Este conjunto de instrucciones se ejecutará repetidamente mientras los valores evaluados en las condiciones sean verdaderos, de lo contrario el bucle o conjunto de instrucciones se seguirán ejecutando. Uno de los requerimientos a considerar en este tipo de estructuras de control es incluir un elemento que altere el valor de la variable que se evalúa en la condición para que cumpla con la condición de salida, ya que de no ser así, el bucle se podría ejecutar de forma infinita, lo cual generaría problemas en distintos niveles, desde que el programa de software no pueda continuar con su ejecución normal, de rendimiento, o problemas de rendimiento en el equipo de cómputo donde se ejecute, hasta provocar un incidente mayor y que genere una falla que deje sin operación el equipo donde se ejecutó.

En algunos tipos de bucle o estructuras de control iterativas se tiene la propiedad de ejecutar al menos una vez la serie de instrucciones antes de entrar al bucle o la función iterativa.

En otros casos no se ejecutará el conjunto de instrucciones sin antes haber evaluado la condición de este. Este tipo de flexibilidad y funcionalidad nos permite, dependiendo de nuestros algoritmos y solución a un problema, definir si es necesario ejecutar las instrucciones al menos una vez o evaluar si se cumplen las condiciones para realizar las acciones iterativas.

A continuación de presentan algunos ejemplos de la utilización de los bucles básicos en lenguajes de programación:

Ejemplo de programa con Do

Este bucle, se encarga de ejecutar una misma acción “mientras que” una determinada condición se cumpla. Ejemplo: Mientras que el **año** sea menor o igual a 2022, imprimir la frase *“Informes del Año*”.

```
# -*- coding: utf-8 -*-
anio = 2001
while anio <= 2022:
    print “Informes del Año”, str(anio)
    anio += 1
```

Ejemplo de programa con Do-while

Lo único en los bucles do while, es que *su código será ejecutado al menos una vez*.

```
#include <stdio.h>

int main(void)
 {
   int i = 10;
   do {
      printf(“the value of i: %i\n”, i);
      i++;
      }
  while( i < 20 );
 }
```

Ejemplo de programa con For

Bucle *for*, en Python, es aquel que nos permitirá iterar sobre una variable compleja, del tipo lista o tupla:

```
1) Por cada nombre en mi_lista,
imprimir nombre
mi_lista = [‘Juan’, ‘Antonio’, ‘Pedro’,
‘Herminio’]
for nombre in mi_lista:
    print nombre
2) Por cada color en mi_tupla,
imprimir color:
mi_tupla = (‘rosa’, ‘verde’, ‘celeste’,
‘amarillo’)
for color in mi_tupla:
    print color
```

Como se ha mencionado anteriormente, es importante incluir los comentarios necesarios en las estructuras de control selectivas como en las iterativas, en este último caso se sugiere documentar adicional a la funcionalidad de la estructura, la o las condiciones de salida de los bucles, este tipo de comentarios facilitarán la identificación de errores o modificaciones para futuros manteamientos.

RESUMEN DE LA UNIDAD

En la unidad se mostraron los cambios que suceden al realizar la conversión entre un algoritmo presentado en un pseudocódigo a un lenguaje de programación, en el cual se describió la representación de las estructuras de control secuenciales, iterativas y selectivas.

Cabe mencionar que, dependiendo del lenguaje de programación a utilizar para realizar la codificación de los algoritmos, presentará diferentes reglas como lo son la sintaxis, uso de palabras reservadas y semántica. Si bien en la actualidad, la mayoría de los lenguajes de programación de alto nivel comparten características similares, la codificación cambia entre lenguajes, pero la funcionalidad u objetivo de las estructuras de control será la misma.

Con las estructuras de control básicas vistas a lo largo de las unidades anteriores, es posible codificar cualquier algoritmo en un lenguaje de programación. Si bien, dependiendo de cada problema se presentarán n-cantidad de soluciones, las cuales podrán variar en su complejidad, estas pueden ser presentadas y/o traducidas.



Lo anterior demuestra una de las grandes ventajas de la programación de software, en la cual cualquier tipo de solución, teniendo un buen nivel de abstracción, puede ser representada de forma digital y permite automatizar cualquier actividad y/o proceso, además que a nivel de negocio esto permite a las organizaciones facilitar sus procesos de toma decisiones.

ACTIVIDADES DE APRENDIZAJE

Unidad 6, actividad inicial. ***Adjuntar archivo.*** A partir del estudio del contenido de la unidad, elabora un mapa conceptual u organizador gráfico sobre los temas vistos. Dentro del documento podrás encontrar recomendaciones de software para su elaboración.

1. **Unidad 6, actividad 1.** ***Adjuntar archivo.*** A partir del estudio de la unidad, elabora un programa que cumpla el siguiente objetivo: El programa deberá solicitar al usuario su nombre, su apellido paterno y su apellido materno por separado y al final mostrará su nombre completo en una solo línea de salida.
2. **Unidad 6, actividad 2.** ***Adjuntar archivo.*** A partir del estudio de la unidad, elabora un programa que cumpla el siguiente objetivo: El programa deberá solicitar al usuario su fecha de nacimiento, con base a los datos del usuario el programa deberá indicar el signo zodiacal del usuario.
3. **Unidad 6, actividad 3.** ***Adjuntar archivo.*** A partir del estudio de la unidad, elabora un programa que cumpla el siguiente objetivo: El programa deberá solicitar al usuario 3 números, los cuales deberán ser mayores de 3 y menores de 15. El programa deberá regresar los números ordenados de mayor a menor e indicar un error si el usuario ingresa números fuera de los parámetros establecidos.
4. **Unidad 6, actividad complementaria 1.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.
5. **Unidad 6, actividad complementaria 2.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.
6. **Unidad 6, actividad complementaria 3.** ***Adjuntar archivo.*** A partir del estudio de la unidad, realiza la actividad que tu asesor te indicará en el foro de la asignatura.

ACTIVIDAD INTEGRADORA

Adjuntar archivo.

En esta unidad aprendiste sobre la traducción de las soluciones algorítmicas a un lenguaje de programación. A partir de los conocimientos adquiridos, desarrolla la actividad integradora:

Retomando el conocimiento adquirido en la unidad, realiza la codificación de la actividad 3 en tres lenguajes diferentes (Python, C y JAVA).

CUESTIONARIO DE REFORZAMIENTO

Adjuntar archivo.

Contesta las siguientes preguntas.

1. Escribe 3 lenguajes de programación.
2. Escribe un ejemplo de una estructura de control selectiva.
3. Escribe un ejemplo de una estructura de control iterativa.
4. Escribe un ejemplo de una estructura de control secuencial.
5. ¿Cuál es la diferencia entre una estructura de control selectiva múltiple y una estructura de control selectiva anidada?
6. ¿Cuál es el objetivo de la estructura de control *For*?
7. ¿Cuál es el objetivo de la estructura de control *Do-While*?
8. ¿Cuál es el objetivo de la estructura de control *IF*?
9. ¿Cuál es el objetivo de la estructura de control *Switch*?
10. ¿Cuál es el objetivo de la estructura de control *While*?

AUTOEVALUACIÓN

I.- Elige la respuesta correcta.

a) Cabecera y bloque de algoritmo.
b) Constantes.
c) Las acciones de declaración y las acciones ejecutables.
d) Pseudocódigo.
e) Interna y externa.

1 Un algoritmo consta de dos componentes.

2. Herramienta de diseño de algoritmos que se asemeja al código de un programa de software.

3. La documentación a elaborar deberá realizarse:

4. El bloque algoritmo es el resto del programa y consta de dos componentes o secciones.

5. Los valores _____ ya no pueden variar en el transcurso del algoritmo.

II.- Responde verdadero (V) o falso (F) según corresponda.

1. El código fuente de un programa es la implementación de un pseudocódigo o un diagrama de flujo. ◯

2. Un algoritmo se puede codificar en cualquier lenguaje de programación. ◯

3. El orden al escribir un algoritmo en un lenguaje de programación no afecta su funcionamiento. ◯

4. Un programa de computadora es un conjunto de instrucciones, órdenes dadas a la máquina, que producirán la ejecución de una determinada tarea. ◯

RESPUESTAS DE AUTOEVALUACIÓN

En este apartado encontrarás las respuestas al examen por unidad.

UNIDAD 6	
I. Solución	
1.	a
2.	d
3.	e
4.	c
5.	b

UNIDAD 6	
II. Solución	
1.	V
2.	V
3.	F
4.	V

REFERENCIAS

Alcaraz, R. (2019). *Una visión práctica de la programación en C*. Ediciones de la Universidad de Castilla-la mancha.

Gottfired, B. (2005). *Programación en C*. McGraw Hill.

Moreno P. (2014). *Programación. Grado superior*. Ra-Ma.

BIBLIOGRAFÍA

Básica

Cairó, O. (2005). *Metodología de la Programación: algoritmos, diagramas de flujo y programas* (3a. Ed.) México: Alfaomega.

Capacho, R. y León, G. (2014). *Estrategias para el diseño y construcción de algoritmos: ejercicios de estrategias y prácticas para la construcción de algoritmos*. Saarbrücken: Editorial Académica Española.

Corona, A. y Ancona, M. (2011). *Diseño de algoritmos y su codificación en lenguaje C*. México: McGraw- Hill Interamericana.

Joyanes, L. (2008). *Fundamentos de Programación: algoritmos, estructura de datos y objetos.* México: McGraw-Hill Interamericana.

Lee, R. C. T., Tseng, S. S. y Chang, C. (2007). *Introducción al diseño y análisis de algoritmos: un enfoque estratégico* (Trad. H. Villagómez). México: McGraw-Hill Interamericana.

López, L. (2011). *Programación estructurada y orientada a objetos: un enfoque algorítmico (3a Ed.)*. México: Alfaomega

Mancilla, A., Ebratt, R. y Capacho, J. (2015). *Diseño y construcción de algoritmos*. Baranquilla, Colombia: Editorial Universidad del Norte.

Méndez, M. A. (2013). *Diseño de algoritmos y su programación en C*. México: Alfaomega.

Ruiz, R. (2013). *Una introducción a la programación estructurada* en C. Santa Fe, Argentina: El Cid Editor.

Villegas, E. y Guerrero, L. E. (2016). *Análisis y diseño de algoritmos: un enfoque práctico*. Bogotá, Colombia: Universidad Nacional de Colombia (Sede Manizales).

Complementaria

Deitel, H. y Deitel, P. (2004). *Cómo programar en C/C++ y Java* (Trad. J. O. García) (4a. Ed). México: Pearson Educación de México.

Jiménez, J., Jiménez, E. M. y Alvarado, L. N. (2014). *Fundamentos de programación: diagramas de flujo, diagramas N-S, pseudocódigo* y Java. México: Alfaomega.







100826